RELIEFGRID

Every Camp. Every Supply. Fairly Shared.

DISASTER RELIEF RESOURCE ALLOCATION SYSTEM

DATABASE SYSTEMS — DIGITAL ASSIGNMENT II

Course Code:	BCSE302L
Submitted by:	ABHISHEK KANDAVEL
Register Number:	24BCE5424
Programme:	B.Tech — Computer Science and Engineering
Campus:	VIT Chennai
Faculty:	PARVATHY AK
Academic Year:	2026–2027
Database Engine:	SQLite 3.50.4 (embedded; Node.js 22 `node:sqlite`)

DECLARATION

I hereby declare that this report, titled “ReliefGrid — Disaster Relief Resource Allocation System”, has been prepared by me for the Database Systems Digital Assignment II. It documents the database design, relational model, allocation methodology, data-integrity controls, transaction handling, application modules and verification results of the application that I designed and implemented.

All places, organisations and people that appear in the application and in this report (for example the “Cauvery Delta Floods 2026” incident, the relief camps, warehouses and user names) are **fictional demonstration data**. They are not official disclosures and do not describe any real emergency. Test counts, query outputs, allocation results and screenshots in this report were produced by running the submitted application and its test suite; Appendix B lists the commands used to reproduce them.

ACKNOWLEDGEMENT

I would like to thank my course faculty, Parvathy AK, and the Database Systems faculty at VIT for the opportunity to apply relational database concepts to a socially relevant problem, and for the guidance on design, normalisation and transaction processing that shaped this work. I also acknowledge the open-source projects on which the implementation relies, namely Node.js (including its built-in SQLite module), SQLite and the Express web framework, together with the published research on fair resource allocation that informed the allocation method.

ABSTRACT

I developed ReliefGrid to help relief organisations coordinate scarce supplies during floods, cyclones and similar emergencies. When requests from relief camps exceed warehouse stock, ad-hoc coordination tends to reward whoever asks first, can promise the same stock twice and leaves no explanation for those who receive less than they need. ReliefGrid addresses this with a relational database linking camps, warehouses, supply items, inventory, requests, allocation decisions, shipments and distributions, and with a priority-based allocation engine whose every decision is recorded and explained.

The database is implemented in SQLite with 22 tables, 9 views, 21 triggers and 24 indexes, created by four checksum-verified migrations. Integrity is enforced inside the database: check constraints make over-reservation of stock impossible, triggers keep the ledger, allocation history and audit log append-only and enforce a shipment state machine, and reconciliation views prove that inventory equals its replayed ledger. A Node.js and Express backend exposes 56 role-protected REST endpoints, and a browser interface provides dashboards, a situation map, explained allocation previews, shipment tracking and analytics.

The allocation method scores every open request line by urgency, the share of at-risk residents and waiting time, and then raises each line to a coverage proportional to its score (capped at 100%) using an exact piecewise-linear solution that is also aware of what earlier runs already delivered. Allocation, reservation, dispatch and delivery run as single atomic transactions. The report documents the design, normalisation, SQL, testing (71 automated tests, all passing) and measured results, and states the limitations of the prototype.

KEYWORDS

Relational Database, SQLite, ER Model, Normalization, Referential Integrity, Transactions, Concurrency Control, Triggers, Views, Disaster Relief, Resource Allocation, Max-Min Fairness, Node.js, Express.

TABLE OF CONTENTS

1. INTRODUCTION 1
2. PROBLEM STATEMENT 1
3. OBJECTIVES AND SCOPE 2
3.1 Objectives 2
3.2 Scope 2
4. EXISTING SYSTEM AND PROPOSED SYSTEM 3
4.1 Existing coordination practice 3
4.2 Proposed system 3
5. REQUIREMENTS AND TECHNOLOGY STACK 4
5.1 Functional requirements 4
5.2 Non-functional requirements 4
5.3 Technology stack 5
5.4 Users, roles and permissions 5
6. SYSTEM ARCHITECTURE AND METHODOLOGY 6
6.1 High-level architecture 6
6.2 Methodology 6
6.3 Data flow 7
6.4 Sequence of a committed allocation 8
7. DATABASE DESIGN 8
7.1 Reserved versus on-hand stock 9
7.2 Immutable decision records 9
7.3 Primary keys and foreign keys 9
8. ENTITY–RELATIONSHIP MODEL 10
8.1 Main entities 10
8.2 ER diagram 12
8.3 Cardinality summary 13
9. RELATIONAL SCHEMA AND DATA DICTIONARY 13
9.1 Table responsibilities 14
9.2 Data dictionary of the principal tables 14
9.3 Indexes 16
10. NORMALIZATION AND INTEGRITY CONSTRAINTS 17

10.1 First normal form (1NF) 17
10.2 Second normal form (2NF) 17
10.3 Third normal form (3NF) and deliberate denormalisation 17
10.4 Integrity constraints 18
10.5 Triggers (21) 18
10.6 Views (9) 19
10.7 Nulls and optional relationships 19
11. PRIORITY-BASED ALLOCATION METHODOLOGY 20
11.1 Inputs 20
11.2 Approval against a planning standard 20
11.3 Priority score 21
11.4 Rationing a scarce item 21
11.5 Whole units 22
11.6 Worked examples 22
11.7 Selecting source warehouses 23
11.8 Explainability 23
11.9 Properties and how they are verified 23
12. TRANSACTIONS, CONCURRENCY AND DATA INTEGRITY 24
12.1 Three layers of protection against double-booking 24
12.2 Atomic units of work 24
12.3 Shipment life-cycle 25
12.4 Ledger and reconciliation 25
12.5 Verified behaviour 25
13. SQL OPERATIONS AND EXAMPLE QUERIES 26
13.1 Schema definition (excerpt) 26
13.2 Data manipulation as issued by the service layer 26
13.3 Constraints and triggers rejecting invalid operations (captured output) 27
13.4 Showcase queries and their results 27
14. APPLICATION MODULES AND API DESIGN 31
14.1 Frontend modules 31
14.2 Backend API 32
14.3 Security design 34
15. TESTING AND RESULTS 34

15.1 Testing strategy 34
15.2 Environment and how to read the results 34
15.3 Automated test suite 35
15.4 Allocation scenarios (verified, fixed clock) 35
15.5 Seeded incident: end-to-end results 37
15.6 Performance of the allocation engine 39
15.7 Integrity results 39
15.8 Manual end-to-end verification in a browser 39
15.9 Summary 40
16. LIMITATIONS AND FUTURE ENHANCEMENTS 40
16.1 Limitations 40
16.2 Future enhancements 41
17. CONCLUSION 41
REFERENCES 42
APPENDIX A — APPLICATION SCREENSHOTS 43
A.1 Sign-in 43
A.2 Operations dashboard 44
A.3 Supply requests 46
A.4 Allocation engine 48
A.5 Shipments and deliveries 52
A.6 Inventory 55
A.7 Camps and distribution 56
A.8 Analytics 57
A.9 Administration 58
A.10 Mobile layout 61
APPENDIX B — REPRODUCING THE RESULTS 62
APPENDIX C — AUTOMATED TEST RESULTS 63

LIST OF FIGURES

Figure 1: ReliefGrid system architecture (presentation, application and data layers) 6
Figure 2: Sequence of interactions when an allocation run is committed 8
Figure 3: Entity-relationship diagram of the implemented schema (22 tables, 34 drawn relationships) 12
Figure 4: Allocation procedure for one run, executed inside a single transaction 22
Figure 5: Shipment state machine enforced by trg_shipments_valid_transition 25
Figure 6: Coverage of each camp by allocation method (Scenario 2, verified) 36
Figure 7: Coverage of drinking water by camp for the seeded incident (verified, fixed clock) 38
Figure 8: Sign-in page with the local demo-account shortcuts 43
Figure 9: Dashboard: KPI cards, situation map (camps coloured by most urgent open request, shipment routes) and live alerts 44
Figure 10: Dashboard (lower part): supply versus open demand, open requests by urgency and recent activity from the audit trail 45
Figure 11: Requests list with status filters, urgency filter and search 46
Figure 12: Request page: requested → approved → allocated → delivered, and the allocation decisions that were made for this request in plain language 47
Figure 13: New-request dialog (camp coordinator): urgency, items, and the live planning-cap hint per item 47
Figure 14: Request page after submission: 40,000 L of water requested, 17,250 L approved with the reason recorded 48
Figure 15: Allocation page: the priority formula and the four steps used to share a shortage 48
Figure 16: Allocation preview for drinking water: priority-score breakdown, still-needs, allocated quantity, coverage bar and source warehouses 49
Figure 17: A line expanded to show its plain-language explanation 49
Figure 18: Insulin: lines that already hold more than their fair level wait (“already holds 27 of 40”, 0 allocated) 50
Figure 19: Confirmation dialog before committing a run 50
Figure 20: Committed run: every decision, its score and the warehouse it is sourced from 51
Figure 21: Shipments planned by the committed run 51
Figure 22: Allocation history: immutable record of all committed runs 52
Figure 23: Shipments board as seen by a warehouse manager (own warehouse only) 52
Figure 24: Dispatch dialog 53
Figure 25: Shipment detail with the timeline written by a database trigger 53
Figure 26: Confirm-delivery dialog (volunteer): per-line received quantities; a smaller number records a shortfall 54
Figure 27: Volunteer view after confirming a delivery 54
Figure 28: Warehouse inventory: on hand, reserved, available, reorder level 55
Figure 29: Stock ledger viewer: append-only history of one stock line 55
Figure 30: Relief camps sorted by urgency 56
Figure 31: Camp detail: requests, stock held, shipments and distributions 56
Figure 32: Distribution dialog refusing a quantity larger than the camp holds 57
Figure 33: Analytics: coverage by urgency, time to first delivery, deliveries per day and committed stock 57

Figure 34: Analytics (lower part): largest shortages, camp coverage, allocation runs and losses in transit 58
Figure 35: User administration with role and scope 58
Figure 36: Allocation settings: urgency weights and parameters (editable by administrators) 59
Figure 37: Append-only audit trail 59
Figure 38: Data-integrity self-check: all four reconciliation views report no problems 60

LIST OF TABLES

Table 1: Ad-hoc coordination compared with ReliefGrid 3
Table 2: Technology stack 5
Table 3: Role-based permissions (as implemented) 5
Table 4: Table groups and their responsibilities 9
Table 5: Principal relationships and their integrity role 10
Table 6: Entities and representative attributes 10
Table 7: Relational schema (all tables) 13
Table 8: All 24 secondary indexes 16
Table 9: Deliberate denormalisation and how consistency is protected 17
Table 10: Constraint types and project examples 18
Table 11: All triggers 18
Table 12: All views 19
Table 13: Inputs for one request line i of a supply item 20
Table 14: Multi-round fairness: naive remaining-need split versus ReliefGrid (verified) 23
Table 15: Properties of the method and the tests that verify them 23
Table 16: Operations and what each transaction contains 24
Table 17: Captured database responses to invalid statements 27
Table 18: Result of q01 (first 6 of 12 rows) 28
Table 19: Result of q02 (first 6 of 12 rows) 28
Table 20: Result of q05 (first 6 of 12 rows) 29
Table 21: Result of q06 (8 rows) 29
Table 22: Result of q08 (4 rows) 30
Table 23: Result of q11 (3 rows) 30
Table 24: Result of q12 (5 rows) 30
Table 25: Result of q14 (3 rows) 31
Table 26: Result of q15 (4 rows) 31
Table 27: Frontend modules 31
Table 28: REST API endpoints and access rules 32
Table 29: Verification environment 34
Table 30: Automated tests by file 35
Table 31: Units / coverage per camp and method (Scenario 2) 35
Table 32: Summary metrics by method (Scenario 2) 36
Table 33: Summary metrics by method (Scenario 3) 37

Table 34: Aging example (Scenario 6): 900 units for three requests of 1,000 37
Table 35: Stock versus open demand per item at the seeded state (verified)..... 37
Table 36: Packaged drinking water: a preview run over request lines that started from zero (verified) .. 38
Table 37: Allocation engine timing on synthetic data (12 warehouses, 18 items)..... 39
Table 38: Browser verification (manual and scripted) 39
Table 39: Summary of verified outcomes..... 40
Table 40: Project size (non-blank files in each area) 62

1. INTRODUCTION

Floods and cyclones displace thousands of people within hours. They move into schools, community halls and shelters that become temporary relief camps, while food, drinking water, medicines and shelter materials are held in a small number of warehouses. The difficult part of the response is rarely the existence of supplies; it is deciding, quickly and defensibly, how limited supplies should be shared among many camps whose size, vulnerability and urgency differ, and then tracking those supplies to the people who need them.

ReliefGrid — Disaster Relief Resource Allocation System — is a full-stack web application that gives relief camps, warehouses, administrators and volunteers one shared, consistent picture of needs and stock. Camps submit supply requests with an urgency level; warehouses maintain inventory; and an allocation engine decides how much of each scarce item each request receives, from which warehouse, and why. The decision is turned into shipments that are dispatched, delivered and finally distributed to residents, with every step recorded.

From a Database Management Systems perspective the project demonstrates entity modelling, primary and foreign keys, normalisation and deliberate denormalisation, check and unique constraints, triggers, views, partial and composite indexes, versioned schema migrations, transaction-safe updates with explicit concurrency control, reconciliation queries and a range of analytical SQL (joins, window functions, recursive common table expressions and correlated subqueries). It also illustrates why auditability and integrity matter more than raw features in a system that allocates life-sustaining supplies.

2. PROBLEM STATEMENT

During an emergency, demand for a supply item frequently exceeds what the warehouses hold. Coordinating the response with spreadsheets, telephone calls and messaging groups creates several recurring risks:

- **Unfair outcomes.** First-come-first-served rewards whoever called first, not whoever needs the item most; a camp with many children, elderly residents or a critical medical need may receive nothing while an earlier, less urgent request is filled completely.
- **Double-booking of stock.** Two coordinators can be promised the same units at the same time because the stock figure they saw was out of date.
- **Over-claiming.** Requests are not checked against any per-person planning standard, so inflated requests distort shares.
- **Loss of fairness over time.** A camp served generously in one round competes again in the next as if it had received nothing.
- **No explanation or trail.** Camps that receive less than they asked for cannot see why, and administrators cannot later audit who decided what, using which numbers.
- **Weak delivery tracking.** Goods lost or damaged in transit are not fed back into the demand picture.

The problem addressed by ReliefGrid is therefore to design and implement a relational data repository and web application that (i) records camps, warehouses, stock, requests and deliveries consistently; (ii) allocates scarce supplies by a clearly defined, fair and explainable rule; (iii) makes over-allocation impossible and preserves an immutable decision history; and (iv) lets each class of user do only what their role allows.

3. OBJECTIVES AND SCOPE

3.1 Objectives

- Design a normalised relational schema for incidents, camps, warehouses, supply items, inventory, requests, allocation decisions, shipments, camp stock and distributions.
- Use primary keys, foreign keys, unique and check constraints, triggers and indexes to protect integrity and support efficient retrieval.
- Design a priority-based allocation method that considers urgency, camp population and vulnerability, waiting time and available stock, and that remains fair across successive allocation rounds.
- Make stock reservation, dispatch, delivery and cancellation transaction-safe so that stock can never be over-allocated and the ledger always reconciles.
- Keep an auditable, append-only history of stock movements, allocation decisions, shipment status changes and user actions.
- Provide authentication and role-based access control for administrators, warehouse managers, camp coordinators and volunteers.
- Expose the data through a REST API and a responsive browser interface with dashboards, shortage alerts and analytics.
- Validate the system with automated tests and realistic scenarios, and report limitations honestly.

3.2 Scope

The implemented scope is a working prototype for a single incident region. It includes managing camps, warehouses, items and inventory; submitting and reviewing supply requests; previewing and committing allocation runs; planning, dispatching, delivering and cancelling shipments; recording distributions at camps; dashboards, alerts and analytics; user and settings administration; an audit trail and database self-checks.

The application is not a production emergency-management system. It does not integrate with real government or NGO data feeds, route vehicles on real road networks, send SMS or e-mail notifications or support multi-site database replication. All demonstration data are fictional (Chapter 16 discusses limitations in detail).

4. EXISTING SYSTEM AND PROPOSED SYSTEM

4.1 Existing coordination practice

In many relief operations coordination relies on a mixture of spreadsheets, phone calls, messaging groups and paper registers. This is flexible and quick to start, but it does not enforce any consistency between what a warehouse holds, what has been promised and what has been delivered. Prioritisation is manual and typically not recorded, and there is no systematic feedback from deliveries into outstanding demand. The comparison below is a general characterisation of ad-hoc coordination, not a description of any specific organisation.

4.2 Proposed system

ReliefGrid replaces ad-hoc coordination with a normalised relational repository and a web interface in which every quantity has an owner and a state. Stock that has been promised to a planned shipment is tracked separately from stock physically present; an allocation engine applies one documented rule to all requests; and database constraints, triggers and reconciliation views prevent and detect inconsistencies.

***Table 1**: Ad-hoc coordination compared with ReliefGrid*

Aspect	Ad-hoc coordination (typical)	ReliefGrid
Stock visibility	Separate spreadsheets; may be stale	Single inventory with on-hand, reserved and available quantities per warehouse
Prioritisation	Manual judgement, rarely recorded	Score from urgency, at-risk share and waiting time, stored with each decision
Over-allocation	Possible: same stock promised twice	Prevented by transactions, conditional updates and a table CHECK constraint
Fairness over time	A well-served camp competes again as if new	Deficit-aware: cumulative coverage is levelled across rounds
Explanation	Usually none	Plain-language explanation per request line, visible to the camp
Over-claiming	Unchecked	Approved quantity capped by a per-person planning standard
Delivery tracking	Calls and messages	Shipment life-cycle with automatic timeline; short deliveries return to the queue
Traceability	Edits overwrite history	Append-only stock ledger, allocation history and audit log (trigger-enforced)
Access control	Shared files and groups	Four roles with scope limits, enforced in the API and by database constraints
Integrity checks	Manual reconciliation	Four reconciliation views that must return zero rows

5. REQUIREMENTS AND TECHNOLOGY STACK

5.1 Functional requirements

- The system shall manage incidents, relief camps (with population and at-risk counts), warehouses, supply items and per-warehouse inventory.
- Camp coordinators shall submit requests with an urgency level and item quantities; quantities shall be approved against a per-person planning standard where one exists.
- Administrators shall be able to preview an allocation without changing data and then commit it; the system shall generate shipments and reserve stock in the same transaction.
- Warehouse managers shall dispatch and cancel planned shipments; volunteers and coordinators shall confirm deliveries, recording any shortfall.
- Camp coordinators shall record distributions to residents; camp stock shall never become negative.
- The system shall provide dashboards, a situation map, shortage, SLA, low-stock and delay alerts, and analytics.
- The system shall authenticate users and restrict every operation by role and scope, and shall record an audit trail.
- Administrators shall be able to tune urgency weights and allocation parameters and run database integrity checks.

5.2 Non-functional requirements

- **Data integrity:** impossible states (negative or over-reserved stock, invalid transitions, orphan references) are rejected by the database itself.
- **Atomicity and isolation:** multi-step operations succeed or fail as a whole and cannot interleave unsafely.
- **Traceability:** history is append-only and every decision can be reproduced from stored parameters.
- **Maintainability:** schema changes are versioned migrations; the allocation algorithm is isolated as pure, unit-testable code.
- **Security:** salted password hashing, hashed session tokens, prepared statements, input validation, security headers and login throttling.
- **Usability:** responsive interface in light and dark themes, clear status and urgency indicators, and explanations written in plain language.
- **Portability:** one command to set up and run on a developer machine, with no database server to install.

5.3 Technology stack

***Table 2**: Technology stack*

Layer	Technology	Purpose
Database	SQLite 3.50.4 via Node.js built-in `node:sqlite`	Relational storage with constraints, triggers, views, partial indexes, window functions and CTEs; WAL journal
Migrations	Custom runner + SQL files	Ordered, checksum-verified schema changes; refuses to run if an applied file was edited
Backend	Node.js 22, Express 5	REST API, authentication middleware, role and scope guards
Business logic	Service modules (JavaScript)	One function per operation; each runs in one transaction
Allocation engine	Pure JavaScript module	Priority scoring, exact fair-level solver, rounding, warehouse sourcing, explanations
Security	Node `crypto` (scrypt, SHA-256)	Password hashing; hashed bearer-token sessions
Frontend	Vanilla ES modules, HTML, CSS	Single-page application with auto-escaping templates; no build step
Charts and map	Hand-written SVG	Offline analytics charts and situational map
Testing	`node:test`, scenario harness	71 automated tests; baseline comparisons and performance benchmark

5.4 Users, roles and permissions

Four roles are defined. Camp coordinators and warehouse managers are bound to a single camp or warehouse by a database CHECK constraint on the users table; every API route additionally checks that scope.

***Table 3**: Role-based permissions (as implemented)*

Capability	Admin	Warehouse manager	Camp coordinator	Volunteer
View dashboards, requests, shipments, inventory, camps, analytics, allocation history	Yes	Yes	Yes	Yes
Create / cancel a supply request; raise urgency	Yes	—	Own camp	—
Lower urgency; override approved quantity	Yes	—	—	—
Preview and commit allocation; edit allocation settings	Yes	—	—	—
Receive / adjust stock; set reorder level	Yes	Own warehouse	—	—
Assign volunteer, dispatch or cancel a planned shipment	Yes	Own warehouse	—	—
Claim an unassigned shipment	—	—	—	Yes
Confirm delivery of a shipment	Yes	—	Receiving camp	Assigned
Record distribution to residents	Yes	—	Own camp	—
Update camp census (population, at-risk)	Yes	—	Own camp	—
Manage users, audit trail, integrity checks, camp status	Yes	—	—	—

6. SYSTEM ARCHITECTURE AND METHODOLOGY

6.1 High-level architecture

The architecture separates presentation, application logic, the pure allocation engine and persistence. The browser talks only to the REST API using a bearer token. Routes perform authentication, role and scope checks and input parsing; services contain the business operations and open one transaction per operation; the allocation engine contains no input/output at all, so it can be tested with randomised scenarios in milliseconds. The database enforces integrity independently of the application code.

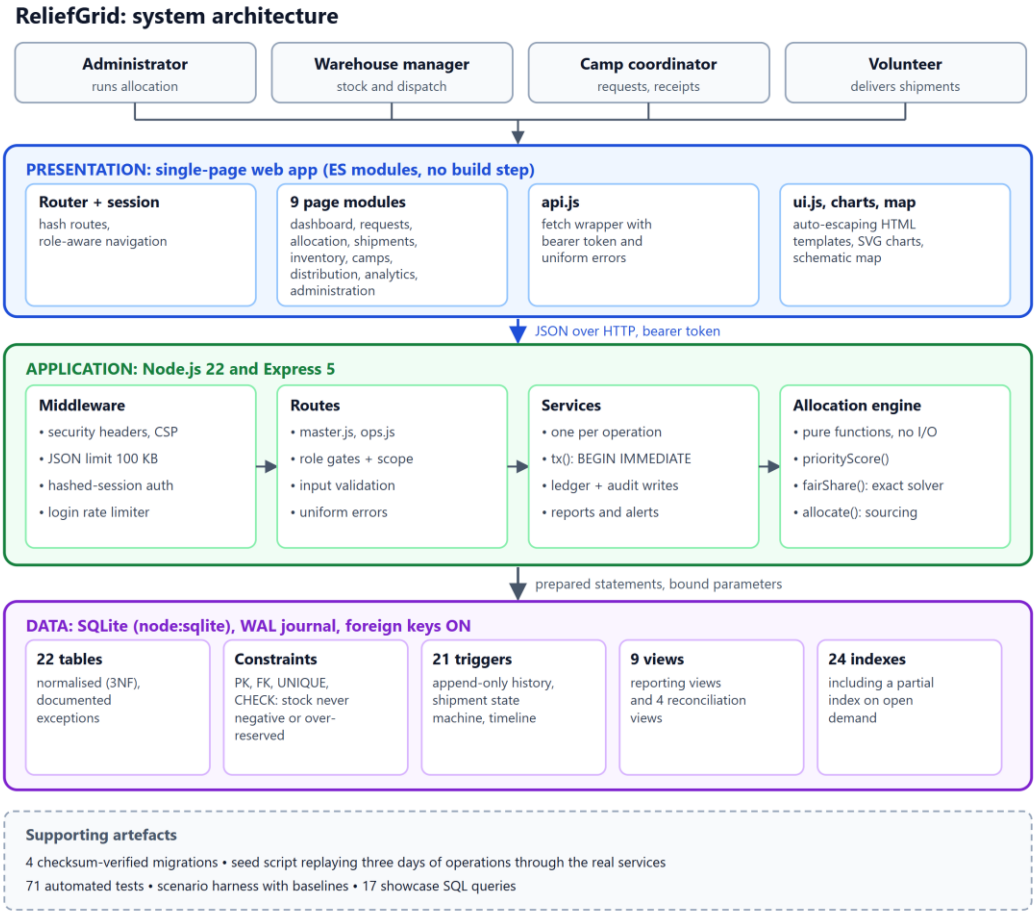


***Figure 1**: ReliefGrid system architecture (presentation, application and data layers)*

6.2 Methodology

1. **Problem analysis:** identify actors (camps, warehouses, administrators, volunteers), the supplies they exchange and the failure modes of manual coordination (Chapter 2).
2. **Schema design:** model entities and relationships, choose keys and optional relationships, decide where counters are stored and how they are reconciled (Chapters 7–10).
3. **Allocation design:** define the priority score and a fairness rule, prove its properties and implement it as a pure module (Chapter 11).

4. **Implementation:** write four ordered SQL migrations (schema, views, triggers, reference data), a migration runner and a transaction helper.
5. **Services and API:** implement one service function per business operation, then the routes with role and scope guards (Chapters 12 and 14).
6. **Seed scenario:** replay three days of fictional operations through the real services with an injected clock so that ledgers, counters and the audit log are genuinely consistent.
7. **Frontend:** build the single-page application (nine modules) on top of the API.
8. **Verification:** unit, property, database, API and scenario tests; baseline comparison; manual end-to-end checks in a browser (Chapter 15).

6.3 Data flow

A coordinator creates a request; the service computes an approved quantity for each line from the camp's population and the item's planning standard and stores request and lines in one transaction. An administrator previews an allocation: the engine reads open demand (view `v_open_demand`) and available stock and returns decisions without writing anything. On commit, the same computation is repeated inside a write transaction and its result is persisted: an allocation run, one decision per line (with the score components and an explanation), allocation lines (which warehouse supplies how much), planned shipments, shipment lines, stock reservations and ledger rows.

A warehouse manager dispatches a shipment, which converts reserved stock into stock that has left the warehouse. A volunteer or coordinator confirms delivery, entering what actually arrived; camp stock increases, request counters are updated and any shortfall becomes open demand again. Finally the coordinator records distributions to residents, which decrease camp stock. The front end never touches SQL; it only consumes the JSON API.

6.4 Sequence of a committed allocation

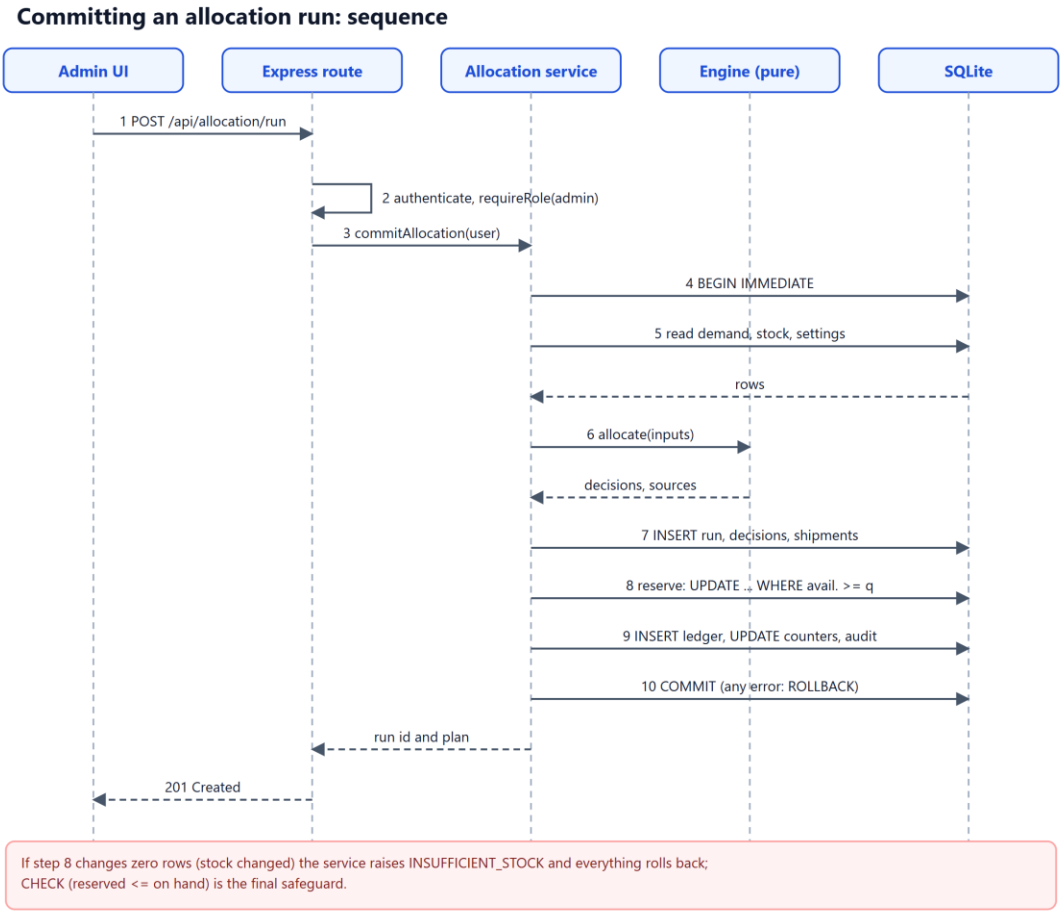


***Figure 2**: Sequence of interactions when an allocation run is committed*

7. DATABASE DESIGN

The database is organised into 22 application tables in seven groups. Each table has a primary key; foreign keys connect related concepts, while nullable foreign keys represent genuinely optional relationships such as a shipment that has no assigned volunteer yet. The schema is created by the four migration files in `db/migrations` and is identical for the application, the tests and the seed script.

Table 4: Table groups and their responsibilities

Group	Tables	Responsibility
Reference data	urgency_levels, supply_categories, supply_items, allocation_settings	Urgency weights and target hours, item catalogue with per-person planning standards, tunable allocation parameters
Places and people	disasters, camps, warehouses, users, sessions	Incidents, camps with population and at-risk counts, warehouses with coordinates, role-scoped users and hashed sessions
Demand	requests, request_items	Requests with urgency; lines carry requested, approved, allocated and delivered quantities
Warehouse stock	inventory, stock_movements	Current on-hand and reserved quantities per warehouse and item; append-only ledger that replays to them
Allocation decisions	allocation_runs, allocation_decisions, allocation_lines	Immutable record of each run: parameters, inputs, scores, outcome, explanation and source warehouses
Logistics	shipments, shipment_items, shipment_events, camp_stock, distributions	Shipments and their lines, trigger-written timeline, stock at camps and aid handed out
Audit	audit_log	Append-only record of state-changing actions

7.1 Reserved versus on-hand stock

Allocation is a *plan*; stock physically leaves a warehouse only at dispatch. The inventory table therefore keeps two counters per warehouse and item: `quantity_on_hand` and `quantity_reserved`, and defines availability as their difference. A reservation is made when an allocation run is committed, released if the shipment is cancelled and converted into a dispatch (both counters decrease together) when the shipment leaves. The table-level constraint `CHECK (quantity_reserved <= quantity_on_hand)` guarantees that no sequence of operations, however buggy, can promise more than exists.

7.2 Immutable decision records

Each allocation decision stores the inputs that produced it: the approved and previously allocated quantities, the remaining need, the network supply and demand for the item, and the urgency, vulnerability and aging factors. These values change over time, so they are deliberately copied into the decision, and database triggers forbid updates and deletes. This makes every decision explainable and auditable long after the underlying request lines have moved on.

7.3 Primary keys and foreign keys

All tables use integer primary keys, except association tables whose key is naturally composite (`inventory` and `camp_stock`) and `sessions` and `allocation_settings`, which use natural text keys. Foreign keys are declared everywhere and enforced on every connection (`PRAGMA foreign_keys = ON`). Deletes are restricted by default; only `sessions` (when a user is removed) and `request_items` (when a request is removed) cascade.

Table 5: Principal relationships and their integrity role

Relationship	Meaning	Integrity role
disasters → camps	A camp belongs to one incident	Closed incidents drop out of allocation
camps → requests → request_items	A camp raises requests; a request has item lines	No orphan lines; lines cascade with their request
supply_items → request_items / inventory / camp_stock	Every quantity refers to a catalogue item	Units and planning standards come from one place
warehouses + supply_items → inventory	One stock row per warehouse and item (composite key)	No duplicate stock rows; target of ledger foreign key
inventory → stock_movements	Each ledger row refers to an existing stock line	Ledger cannot refer to stock that does not exist
allocation_runs → allocation_decisions → allocation_lines	A run has decisions; a decision has sources	History is queryable in both directions
request_items → allocation_decisions / shipment_items	A line is considered in runs and fulfilled by shipments	Counters on the line can be reconciled against shipments
warehouses, camps → shipments	A shipment goes from one warehouse to one camp	Shipment routes are always valid places
shipments → shipment_items / shipment_events	Contents and timeline of a shipment	Timeline rows exist only for real shipments
camp_stock → distributions	Aid handed out is taken from stock held at the camp	A distribution can only refer to stock the camp holds
users → camps / warehouses	Coordinators and managers are scoped to one place	Scope enforced by a CHECK on the users table

8. ENTITY–RELATIONSHIP MODEL

The ER model captures the principal entities and relationships of the implemented schema. The diagram in Section 8.2 was generated by a script from the live schema (`PRAGMA table_info` and `PRAGMA foreign_key_list`), so it cannot drift from the migrations. Entities are coloured by group, connectors by the table they reference.

Modelling note. Cardinalities describe what the schema permits. A foreign key from `camps` to `disasters` guarantees that each camp references an existing incident; it does not require every incident to have a camp. Six actor columns (`user_id`, `actor_id`, `run_by`, `distributed_by`, `last_actor_id` and similar) also reference `users`; they are omitted from the diagram only for readability.

8.1 Main entities

Table 6: Entities and representative attributes

Entity	Purpose	Representative attributes
disasters	An incident that groups camps	id, name, type, status, started_at

Entity	Purpose	Representative attributes
camps	A relief camp	id, disaster_id, name, capacity, population, vulnerable_count, status
warehouses	A depot holding supplies	id, name, latitude, longitude, status
users / sessions	Accounts with a role and scope; hashed login sessions	id, username, role, camp_id, warehouse_id / token_hash, expires_at
urgency_levels	Urgency labels with weight and target hours	level, label, weight, sla_hours
supply_categories / supply_items	Catalogue of supplies	id, category_id, name, unit, standard_per_person_per_day
allocation_settings	Tunable method parameters	key, value, description
requests	A camp's supply request	id, camp_id, requested_by, urgency, status, created_at
request_items	A requested item with four counters	id, request_id, item_id, quantity_requested / approved / allocated / delivered
inventory	Stock of an item in a warehouse	warehouse_id, item_id, quantity_on_hand, quantity_reserved, reorder_level
stock_movements	Append-only stock ledger	id, warehouse_id, item_id, movement_type, delta_on_hand, delta_reserved
allocation_runs	A committed allocation run	id, run_by, params_json, requests_considered, units_allocated
allocation_decisions	Decision for one request line in a run	id, run_id, request_item_id, priority_score, allocated_qty, explanation
allocation_lines	Which warehouse supplies a decision, and how much	id, decision_id, shipment_id, warehouse_id, quantity, distance_km
shipments	Movement from one warehouse to one camp	id, code, warehouse_id, camp_id, status, volunteer_id, eta
shipment_items / shipment_events	Lines and status timeline of a shipment	shipment_id, request_item_id, quantity, quantity_received / status, actor_id, created_at
camp_stock / distributions	Stock at a camp; aid handed out	camp_id, item_id, quantity / quantity, people_served
audit_log	Append-only action log	id, user_id, action, entity_type, entity_id, created_at

8.2 ER diagram

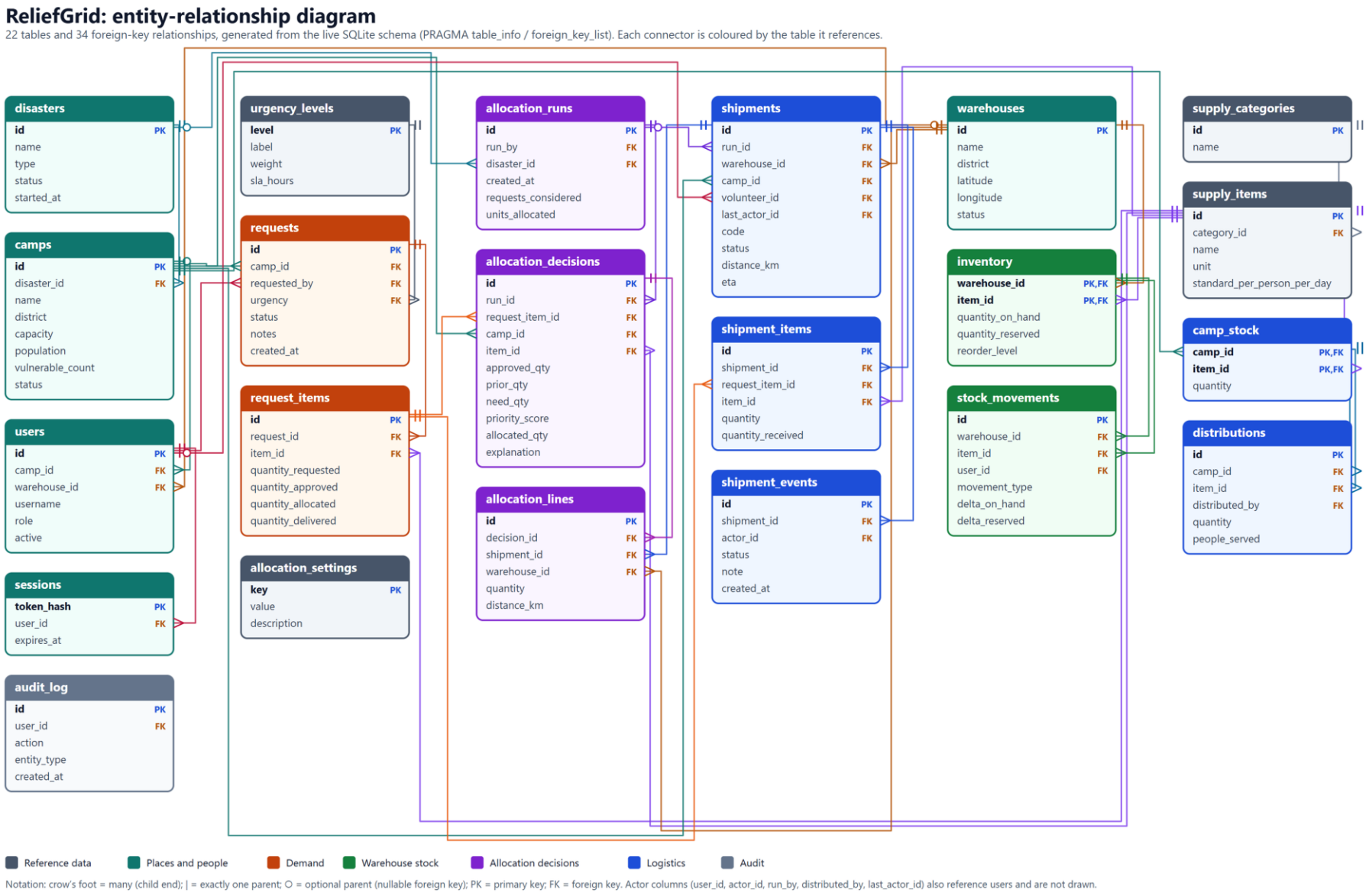


***Figure 3**: Entity-relationship diagram of the implemented schema (22 tables, 34 drawn relationships)*

8.3 Cardinality summary

- One incident has many camps; each camp references exactly one incident.
- One camp raises many requests; one request contains one or more request lines; each line refers to one catalogue item (unique per request).
- One warehouse holds many inventory rows (one per item); each inventory row has many ledger entries.
- One allocation run has many decisions; each decision refers to exactly one request line and has zero or more allocation lines (zero when nothing could be allocated).
- One shipment has one or more lines and a timeline of events; it goes from exactly one warehouse to exactly one camp and may have zero or one volunteer.
- One request line can be supplied by several shipments (different warehouses), and one shipment can carry lines for several requests of the same camp.
- A user is optionally scoped to one camp or one warehouse, depending on role.

9. RELATIONAL SCHEMA AND DATA DICTIONARY

The relational notation below is generated directly from the migrated database. PK denotes primary key and FK denotes foreign key (with the referenced table). Types follow SQLite affinity rules; exact nullability, defaults and constraints are listed in the data dictionary for the principal tables and can be read in full from `db/migrations/001_core_schema.sql`.

***Table 7**: Relational schema (all tables)*

Relation	Attributes
ALLOCATION_DECISIONS	id (PK), run_id (FK→allocation_runs), request_item_id (FK→request_items), camp_id (FK→camps), item_id (FK→supply_items), approved_qty, prior_qty, need_qty, item_supply, item_demand, urgency_weight, vulnerability_factor, aging_factor, priority_score, allocated_qty, scarce, explanation
ALLOCATION_LINES	id (PK), decision_id (FK→allocation_decisions), shipment_id (FK→shipments), warehouse_id (FK→warehouses), quantity, distance_km
ALLOCATION_RUNS	id (PK), run_by (FK→users), disaster_id (FK→disasters), params_json, created_at, requests_considered, items_scarce, units_allocated, shipments_created, note
ALLOCATION_SETTINGS	key (PK), value, description
AUDIT_LOG	id (PK), user_id (FK→users), action, entity_type, entity_id, details, created_at
CAMP_STOCK	camp_id (PK, FK→camps), item_id (PK, FK→supply_items), quantity, updated_at
CAMPS	id (PK), disaster_id (FK→disasters), name, district, latitude, longitude, capacity, population, vulnerable_count, status, contact_phone, opened_at
DISASTERS	id (PK), name, type, status, started_at, ended_at
DISTRIBUTIONS	id (PK), camp_id (FK→camp_stock), item_id (FK→camp_stock), quantity, people_served, distributed_by (FK→users), note, created_at
INVENTORY	warehouse_id (PK, FK→warehouses), item_id (PK, FK→supply_items), quantity_on_hand, quantity_reserved, reorder_level, updated_at
REQUEST_ITEMS	id (PK), request_id (FK→requests), item_id (FK→supply_items), quantity_requested, quantity_approved, quantity_allocated, quantity_delivered, approval_note

Relation	Attributes
REQUESTS	id (PK), camp_id (FK→camps), requested_by (FK→users), urgency (FK→urgency_levels), status, notes, needed_by, created_at, updated_at
SESSIONS	token_hash (PK), user_id (FK→users), created_at, expires_at
SHIPMENT_EVENTS	id (PK), shipment_id (FK→shipments), status, actor_id (FK→users), note, created_at
SHIPMENT_ITEMS	id (PK), shipment_id (FK→shipments), request_item_id (FK→request_items), item_id (FK→supply_items), quantity, quantity_received
SHIPMENTS	id (PK), code, run_id (FK→allocation_runs), warehouse_id (FK→warehouses), camp_id (FK→camps), status, volunteer_id (FK→users), distance_km, vehicle, created_at, dispatched_at, eta, delivered_at, status_changed_at, last_actor_id (FK→users), last_note
STOCK_MOVEMENTS	id (PK), warehouse_id (FK→inventory), item_id (FK→inventory), movement_type, delta_on_hand, delta_reserved, reference_type, reference_id, user_id (FK→users), note, created_at
SUPPLY_CATEGORIES	id (PK), name
SUPPLY_ITEMS	id (PK), category_id (FK→supply_categories), name, unit, standard_per_person_per_day, perishable, active
URGENCY_LEVELS	level (PK), label, weight, sla_hours
USERS	id (PK), username, full_name, phone, password_hash, role, camp_id (FK→camps), warehouse_id (FK→warehouses), active, created_at, last_login_at
WAREHOUSES	id (PK), name, district, latitude, longitude, status, contact_phone

9.1 Table responsibilities

`request_items` is the central demand table: its four counters record how much was asked, approved, promised and received. `inventory` and `stock_movements` form a balance-and-ledger pair, and `shipments`/`shipment_items` connect demand to stock. `allocation_runs`, `allocation_decisions` and `allocation_lines` are the permanent memory of every allocation. `camp_stock` and `distributions` close the loop at the camp. The remaining tables provide reference data, identity and audit.

9.2 Data dictionary of the principal tables

inventory

Column	Type	Null?	Meaning and constraints
warehouse_id	INTEGER	NOT NULL	Part of the composite primary key; references warehouses
item_id	INTEGER	NOT NULL	Part of the composite primary key; references supply_items
quantity_on_hand	INTEGER	NOT NULL	Physically in the warehouse. CHECK >= 0
quantity_reserved	INTEGER	NOT NULL	Promised to PLANNED shipments. CHECK >= 0 and <= quantity_on_hand
reorder_level	INTEGER	NOT NULL	Alert threshold; 0 means none. CHECK >= 0
updated_at	TEXT	NOT NULL	ISO-8601 UTC timestamp

request_items

Column	Type	Null?	Meaning and constraints
id	INTEGER	NOT NULL	Surrogate primary key
request_id	INTEGER	NOT NULL	Parent request; ON DELETE CASCADE; UNIQUE with item_id

Column	Type	Null?	Meaning and constraints
item_id	INTEGER	NOT NULL	Catalogue item
quantity_requested	INTEGER	NOT NULL	What the camp asked for. CHECK > 0
quantity_approved	INTEGER	NOT NULL	After the planning-standard cap. CHECK <= requested
quantity_allocated	INTEGER	NOT NULL	Reserved, in transit or delivered. CHECK <= approved
quantity_delivered	INTEGER	NOT NULL	Confirmed received. CHECK <= allocated
approval_note	TEXT	NULL	Reason when the quantity was trimmed or overridden

allocation_decisions

Column	Type	Null?	Meaning and constraints
run_id	INTEGER	NOT NULL	Run that made the decision; UNIQUE with request_item_id
request_item_id	INTEGER	NOT NULL	The request line considered
approved_qty, prior_qty, need_qty	INTEGER	NOT NULL	Snapshot; CHECK approved = prior + need
item_supply, item_demand	INTEGER	NOT NULL	Network stock and open demand for the item in this run
urgency_weight, vulnerability_factor, aging_factor	INTEGER	NOT NULL	The three components of the score
priority_score	REAL	NOT NULL	Product of the three factors
allocated_qty	INTEGER	NOT NULL	Units granted now. CHECK <= need_qty
scarce	INTEGER	NOT NULL	1 when stock was short for the item
explanation	TEXT	NOT NULL	Plain-language reasoning shown to users

shipments

Column	Type	Null?	Meaning and constraints
code	TEXT	NOT NULL	UNIQUE human-readable code (SHP-00001 ...)
run_id	INTEGER	NULL	Allocation run that planned it (NULL for manual shipments)
warehouse_id, camp_id	INTEGER	NOT NULL	Origin and destination
status	TEXT	NOT NULL	CHECK in PLANNED, IN_TRANSIT, DELIVERED, CANCELLED; transitions enforced by trigger
volunteer_id	INTEGER	NULL	Nullable until a volunteer is assigned
distance_km	REAL	NOT NULL	Estimated road distance. CHECK >= 0
dispatched_at, eta, delivered_at	INTEGER	NULL	Nullable timestamps filled as the shipment progresses
status_changed_at, last_actor_id, last_note	INTEGER	NOT NULL	Context the trigger copies into shipment_events

9.3 Indexes

***Table 8**: All 24 secondary indexes*

Index	Table	Columns	Access path served
idx_decisions_request_item	allocation_decisions	request_item_id	Decision history of a request line
idx_decisions_run	allocation_decisions	run_id	All decisions of a run
idx_lines_decision	allocation_lines	decision_id	Sources of a decision
idx_lines_shipment	allocation_lines	shipment_id	Allocation lines of a shipment
idx_audit_created	audit_log	created_at	Chronological audit listing
idx_audit_entity	audit_log	entity_type, entity_id	Audit entries of one entity
idx_camps_disaster_status	camps	disaster_id, status	Camps of an incident by status
idx_distributions_camp	distributions	camp_id, created_at	Distributions of a camp over time
idx_distributions_item	distributions	item_id	Distributions of an item
idx_inventory_item	inventory	item_id	Stock of an item across warehouses
idx_request_items_item	request_items	item_id	Lines for an item
idx_request_items_open	request_items	item_id WHERE quantity_allocated < quantity_approved	Allocator work queue: only lines that still need stock (partial index)
idx_requests_camp	requests	camp_id, created_at	Requests of a camp over time
idx_requests_status_urgency	requests	status, urgency	Open-request lists and dashboard counts
idx_sessions_expires	sessions	expires_at	Expired-session clean-up
idx_sessions_user	sessions	user_id	Sessions of a user (revocation)
idx_shipment_events_shipment	shipment_events	shipment_id, id	Timeline of a shipment
idx_shipment_items_request_item	shipment_items	request_item_id	Shipments that fulfil a request line
idx_shipments_camp	shipments	camp_id, status	Shipments to a camp by status
idx_shipments_status	shipments	status	Board of shipments by status
idx_shipments_volunteer	shipments	volunteer_id, status	A volunteer’s shipments
idx_shipments_warehouse	shipments	warehouse_id, status	Shipments from a warehouse by status
idx_movements_ref	stock_movements	reference_type, reference_id	Ledger rows caused by a run or shipment
idx_movements_wh_item	stock_movements	warehouse_id, item_id, created_at	Ledger of one stock line in time order

Primary keys and UNIQUE constraints additionally create their own implicit indexes. An automated test inspects `EXPLAIN QUERY PLAN` and asserts that four hot queries use these indexes (Section 15.3). No response-time claims are made beyond the measurements reported in Chapter 15.

10. NORMALIZATION AND INTEGRITY CONSTRAINTS

10.1 First normal form (1NF)

Every column holds an atomic value and there are no repeating groups: a request's items are rows of `request_items`, a shipment's contents are rows of `shipment_items`, and the sources of an allocation decision are rows of `allocation_lines`. The only structured text values are `allocation_runs.params_json` (a snapshot of the parameters in force) and `audit_log.details`; both are immutable snapshots that are never queried field by field.

10.2 Second normal form (2NF)

Tables with composite keys contain only attributes that depend on the whole key. In `inventory(warehouse_id, item_id)` the quantities and reorder level describe the pair; item names, units and categories live in `supply_items` and `supply_categories`, and warehouse names and coordinates in `warehouses`. `camp_stock(camp_id, item_id)` follows the same pattern. Tables with surrogate keys have no composite key, so partial dependencies cannot arise, but their functional dependencies were still checked.

10.3 Third normal form (3NF) and deliberate denormalisation

No non-key attribute depends on another non-key attribute. A request stores only an urgency *level*; the weight and target hours are in `urgency_levels`. Camp coordinates appear once, in `camps`; a shipment stores `warehouse_id` and `camp_id`, never copies of names or places. The exceptions below are intentional and documented; each has a mechanism that detects inconsistency.

***Table 9**: Deliberate denormalisation and how consistency is protected*

Denormalised data	Why	Protection
request_items.quantity_allocated / quantity_delivered (derived from shipments)	Hot path of the allocator and dashboards; avoids joins and aggregates on every read	Changed only inside the transaction that changes shipments; view v_allocation_mismatch recomputes them and must be empty
inventory.quantity_on_hand / quantity_reserved (derived from the ledger)	O(1) availability checks; a place for CHECK constraints	Views v_ledger_mismatch and v_reservation_mismatch must be empty
requests.status (derived from lines)	Cheap filtering and indexing	Recomputed from the lines inside the same transaction
allocation_decisions snapshot columns (approved, prior, need, supply, demand, factors)	A historical record: the values change later but the decision must remain explainable	Immutable by trigger
shipments.status_changed_at / last_actor_id / last_note	Lets the status trigger write the timeline row with the correct actor, time and note	Written by the same UPDATE that changes the status
camp_stock.quantity (derived from deliveries and distributions)	Direct answer to "what does the camp hold?"	View v_camp_stock_mismatch must be empty

The design is therefore presented as a third-normal-form schema with explicit, test-covered exceptions rather than as an unconditional claim of 3NF for every table.

10.4 Integrity constraints

Declarative integrity does most of the work. The schema declares 58 CHECK constraints, 40 foreign-key column references (shown as 34 connectors in Figure 3 plus six actor links to users) and 11 UNIQUE clauses, in addition to primary keys and NOT NULL constraints.

***Table 10**: Constraint types and project examples*

Constraint type	Purpose	Example in the project
PRIMARY KEY	Uniquely identifies a row	`inventory(warehouse_id, item_id)`; `requests.id`
FOREIGN KEY	Prevents orphan references	`stock_movements(warehouse_id, item_id)` → `inventory`; `distributions(camp_id, item_id)` → `camp_stock`
UNIQUE	Prevents duplicates	`request_items(request_id, item_id)`; `shipments.code`; case-insensitive `users.username`
CHECK (domain)	Valid values	`status IN (...)`; latitude between -90 and 90; `vulnerable_count <= population`
CHECK (stock safety)	No negative or over-reserved stock	`quantity_reserved >= 0 AND <= quantity_on_hand`
CHECK (lifecycle arithmetic)	Ordered quantities	`delivered <= allocated <= approved <= requested`
CHECK (record consistency)	Decision arithmetic	`approved_qty = prior_qty + need_qty`; `allocated_qty <= need_qty`
CHECK (role scope)	Users bound to the right place	coordinator ⇒ camp_id NOT NULL and warehouse_id NULL; manager the reverse
NOT NULL	Mandatory facts	`requests.camp_id`, `shipments.distance_km`
ON DELETE	Behaviour on parent removal	RESTRICT by default; CASCADE only for `sessions` and `request_items`

10.5 Triggers (21)

***Table 11**: All triggers*

Trigger	Table	Event	Purpose
trg_allocation_decisions_no_delete	allocation_decisions	BEFORE DELETE	Allocation history is immutable
trg_allocation_decisions_no_update	allocation_decisions	BEFORE UPDATE	Allocation history is immutable
trg_allocation_lines_no_delete	allocation_lines	BEFORE DELETE	Allocation history is immutable
trg_allocation_lines_no_update	allocation_lines	BEFORE UPDATE	Allocation history is immutable
trg_allocation_runs_no_delete	allocation_runs	BEFORE DELETE	Allocation history is immutable
trg_allocation_runs_no_update	allocation_runs	BEFORE UPDATE	Allocation history is immutable
trg_audit_log_no_delete	audit_log	BEFORE DELETE	Audit log is append-only
trg_audit_log_no_update	audit_log	BEFORE UPDATE	Audit log is append-only
trg_distribution_stock_guard	distributions	BEFORE INSERT	A distribution cannot exceed stock held at the camp
trg_inventory_no_delete_with_stock	inventory	BEFORE DELETE	Inventory rows holding stock cannot be deleted
trg_requests_camp_open	requests	BEFORE INSERT	No requests for a closed camp
trg_requests_urgency_no_downgrade_when_closed	requests	BEFORE UPDATE OF urgency	A fulfilled or cancelled request keeps its urgency

Trigger	Table	Event	Purpose
trg_shipment_events_no_delete	shipment_events	BEFORE DELETE	Shipment timeline is append-only
trg_shipment_events_no_update	shipment_events	BEFORE UPDATE	Shipment timeline is append-only
trg_shipment_items_immutable	shipment_items	BEFORE UPDATE OF shipment_id, request_item_id, item_id, quantity	Shipment lines cannot be edited after creation
trg_shipment_items_receipt_state	shipment_items	BEFORE UPDATE OF quantity_received	Receipts can be recorded only while in transit
trg_shipments_event_on_insert	shipments	AFTER INSERT	Writes the first timeline row when a shipment is planned
trg_shipments_event_on_status	shipments	AFTER UPDATE OF status	Writes a timeline row on every status change (actor, time, note)
trg_shipments_valid_transition	shipments	BEFORE UPDATE OF status	Shipment state machine: PLANNED→IN_TRANSIT/CANCELLED, IN_TRANSIT→DELIVERED only
trg_stock_movements_no_delete	stock_movements	BEFORE DELETE	Stock ledger is append-only
trg_stock_movements_no_update	stock_movements	BEFORE UPDATE	Stock ledger is append-only

10.6 Views (9)

***Table 12**: All views*

View	Purpose
v_allocation_mismatch	Reconciliation: request counters must equal what shipments say
v_camp_overview	Per-camp roll-up of open requests, most urgent level, active shipments and servings
v_camp_stock_mismatch	Reconciliation: camp stock must equal deliveries minus distributions
v_inventory_available	Warehouse × item with available = on hand − reserved and the below-reorder flag
v_item_supply_demand	Network-wide supply versus open demand per item (shortage alerts)
v_ledger_mismatch	Reconciliation: inventory must equal the replayed stock ledger
v_open_demand	The allocator's work queue: request lines that still need stock in open camps and incidents
v_request_progress	Per-request count of lines allocated and delivered
v_reservation_mismatch	Reconciliation: reserved stock must equal quantity on PLANNED shipments

10.7 Nulls and optional relationships

A NULL always means "unknown or not applicable" and never zero. `shipments.volunteer_id` is NULL until a volunteer is assigned; `shipment_items.quantity_received` is NULL until the delivery is confirmed (a confirmed zero is stored as 0); `allocation_runs.disaster_id` is NULL when a run covers all incidents; `users.camp_id` and `users.warehouse_id` are NULL for roles that are not scoped. The CHECK on `users` ensures that exactly the right combination of NULLs exists for each role.

11. PRIORITY-BASED ALLOCATION METHODOLOGY

The allocation method is the intellectual core of ReliefGrid. It decides, for every supply item that is in short supply, how many units each open request line receives. The design goals were that it must be (i) **fair**: higher need earns more, but nobody with a legitimate claim is ignored; (ii) **explainable**: a camp coordinator must be able to read why a quantity was granted; (iii) **repeatable**: the same data must always give the same answer; (iv) **consistent across rounds**: camps already served must not be favoured again; and (v) **safe**: it can only hand out stock that exists. It is implemented as a pure module (`server/allocation/engine.js`) and invoked by a service that supplies the data and persists the outcome.

11.1 Inputs

***Table 13**: Inputs for one request line i of a supply item*

Symbol	Meaning	Source in the database
A_i	Approved quantity of the line	request_items.quantity_approved
P_i	Quantity already allocated by earlier runs (reserved, in transit or delivered)	request_items.quantity_allocated
$d_i = A_i - P_i$	Remaining need	derived (view v_open_demand)
u_i	Urgency weight (Low 1, Moderate 2, High 3.5, Critical 6)	urgency_levels.weight
v_i	Share of residents who are at-risk (children under 5, elderly, pregnant women, people with disabilities)	camps.vulnerable_count / camps.population
h_i	Hours the request has been waiting	now − requests.created_at
S	Stock available for the item in active warehouses	inventory: quantity_on_hand − quantity_reserved

11.2 Approval against a planning standard

For items that have a per-person planning standard (for example rice 0.5 kg, packaged water 5 L, ORS 0.2 sachet and soap 0.1 bar per person per day), a request line is **approved up to** `standard × camp population × planning horizon` (3 days by default). Anything above that is trimmed and the reason is stored on the line. Items without a meaningful per-person standard (tarpaulins, insulin) are approved as requested, and administrators may override an approval but never above the requested quantity. The standards in the demonstration data are illustrative planning assumptions, not official figures.

Verified example. A camp of 1,000 people asks for 40,000 L of drinking water: 5 L × 1,000 × 3 days = 15,000 L is approved and the line records “Capped at the 3-day planning standard” (automated test `request validation and standard-based approval`). The same behaviour for a 1,150-person camp (17,250 L) is shown in Appendix A.

11.3 Priority score

$score_i = u_i \times (1 + k_v \cdot v_i) \times (1 + k_a \cdot \min(1, h_i / W))$
with vulnerability weight $k_v = 1$, aging bonus $k_a = 0.5$ and aging window $W = 48$ hours (all editable by an administrator and snapshotted into every run).

- **Urgency** is the dominant factor and is chosen by the camp. Coordinators may only raise it; only administrators may lower it.
- **Vulnerability** lifts camps with more at-risk residents, by up to a factor of 2. Camp population influences the allocation in two ways: through this share and through the planning-standard cap that limits how much a camp may claim.
- **Aging** lifts requests that have waited, by up to 1.5 after 48 hours, so that low-urgency requests cannot be postponed indefinitely.

11.4 Rationing a scarce item

Each supply item is rationed independently: stock of rice says nothing about stock of tarpaulins. Let the *coverage* of a line be $(P_i + x_i) / A_i$, where x_i is what the line receives now.

- **If $S \geq \Sigma d_i$** nobody is short and every line receives its remaining need.
- **Otherwise** the engine finds the single fair level λ for which $x_i = \text{clamp}(A_i \cdot \lambda \cdot score_i - P_i , 0 , d_i)$ and $\Sigma x_i = S$. In words: every line is brought up to a cumulative coverage of min(100%, $\lambda \times$ its priority score). Coverage is therefore proportional to priority and capped at 100%; a line that reaches 100% releases its surplus to the others; and a line that already holds more than its fair level (because an earlier run served it) receives nothing until the others catch up.

The total allocation $\Sigma x_i(\lambda)$ is a non-decreasing piecewise-linear function of λ whose breakpoints are the values at which a line starts to receive stock ($\lambda = P_i / (A_i \cdot score_i)$) or becomes capped ($\lambda = 1 / score_i$). The engine sorts the breakpoints, locates the interval that contains S and interpolates, so the solution is exact rather than iterative. With nothing allocated before ($P_i = 0$) the rule reduces to “coverage proportional to priority”. This is a priority-weighted form of max-min fairness (Bertsekas and Gallager, 1992) applied to cumulative coverage.

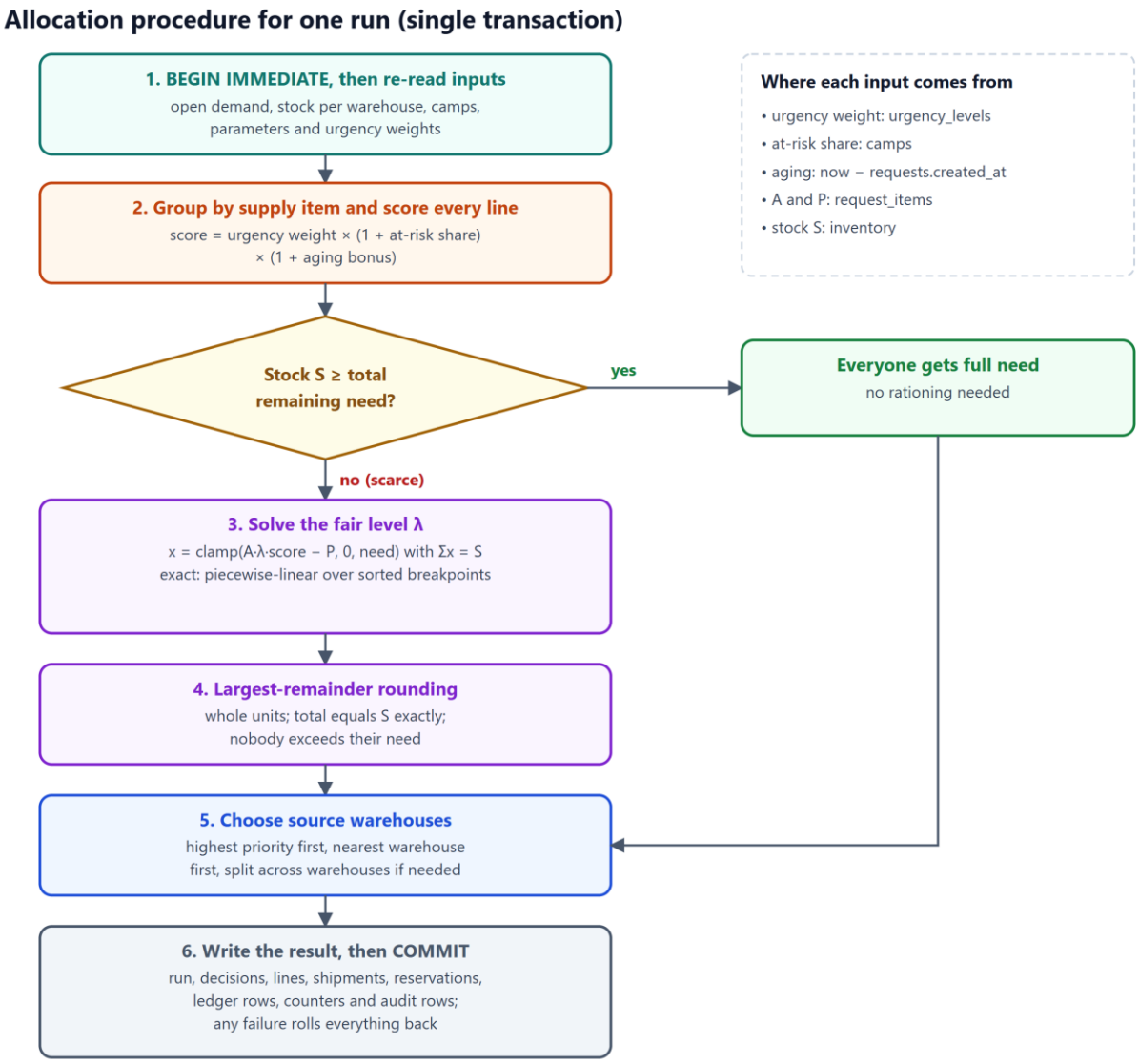


***Figure 4**: Allocation procedure for one run, executed inside a single transaction*

11.5 Whole units

Quantities are whole units. Each x_i is rounded down and the leftover units are given, one each, to the lines with the largest fractional parts (ties: higher priority, then lower id). The integer result sums exactly to S, never exceeds a line's need and lies within one unit of the exact solution.

11.6 Worked examples

Example A (analytical). Two lines of 100 units with priorities 6 (Critical) and 1 (Low) share 100 units. $100 \cdot 6\lambda + 100 \cdot \lambda = 100$ gives $\lambda = 1/7$, so coverages are 6/7 and 1/7: the allocation is **86 and 14 units** (the fraction 0.71 outranks 0.29 in the rounding). This case is an automated test.

Example B (verified by script, scenario 5 of docs/RESULTS.md). Two camps of equal priority each requested 1,000 units. In round 1, camp A alone had a request open and received 400 units. In round 2,

camp B has asked and 600 more units arrive. A method that looks only at remaining need splits the 600 by remaining need; ReliefGrid levels cumulative coverage first.

***Table 14**: Multi-round fairness: naive remaining-need split versus ReliefGrid (verified)*

Camp	Already holds	Naive share	Cumulative coverage (naive)	ReliefGrid share	Cumulative coverage (ReliefGrid)
A (served in round 1)	400	225	63%	100	50%
B (asks in round 2)	0	375	38%	500	50%

11.7 Selecting source warehouses

After shares are fixed, lines are served in priority order, each from the nearest active warehouses that still have stock (road distance = straight-line great-circle distance × a detour factor of 1.35), splitting across warehouses when one is not enough. All lines going from one warehouse to one camp are bundled into one shipment, and stock for every shipment is reserved in the same transaction. Because stock is pooled per item before sourcing, an item can always be sourced completely; the engine asserts this. The greedy choice is not a minimum-cost flow, so total kilometres are not guaranteed to be optimal (Chapter 16).

11.8 Explainability

Every decision stores a plain-language explanation. The following sentence is a real output for a Critical camp in the seeded incident:

> "Priority 8.02 = urgency 6 (Critical) × vulnerability 1.3 (30% at-risk) × waiting 1.03 (3h). Packaged drinking water is scarce: 21,000 L available for 58,535 L of open demand (36%). Every line is brought up to a coverage proportional to its priority (capped at 100%); this line's fair coverage is 91%. It receives 5,228 of 5,750 L (cumulative coverage 91%); the remaining 522 L stays queued."

Camp coordinators see the history of these sentences on the page of their request; administrators see them, with the score breakdown and sources, in the preview and in every historical run. Together with the parameter snapshot stored on each run, any past decision can be audited.

11.9 Properties and how they are verified

***Table 15**: Properties of the method and the tests that verify them*

Property	Verification
Never allocates more than the stock or more than a line needs; integer results	Property test over 600 randomised scenarios (`tests/engine.test.js`)
Uses every available unit when stock is short	Same property test
Within one unit of the exact real-valued solution	Same test, compared with an independent bisection solver
Higher priority never receives lower cumulative coverage (among lines that receive stock)	Property test over 200 randomised scenarios; demo-data test
Deterministic regardless of the order of the input	`determinism` test
Deficit awareness: lines above their fair level wait; lines are levelled	Two dedicated tests (Example B)

Property	Verification
Abundant stock gives everyone their full remaining need	`abundance` test and demo-data test
Reservation never exceeds stock even with concurrent runs; failure mid-run rolls back	API tests (Chapter 12)

12. TRANSACTIONS, CONCURRENCY AND DATA INTEGRITY

Every operation that changes more than one row runs inside `tx(db, fn)` (`server/db.js`), which issues **BEGIN IMMEDIATE**. SQLite then takes the write lock before the transaction reads anything, so everything read inside the transaction cannot be changed by another writer until it commits. Nested calls join the outer transaction, and any exception triggers ROLLBACK.

12.1 Three layers of protection against double-booking

1. **Serialised writers and a re-read inside the transaction.** `commitAllocation` loads open demand and stock after BEGIN IMMEDIATE; a plan computed earlier (the preview) is never trusted.
2. **Conditional atomic update.** Reservation is `UPDATE inventory SET quantity_reserved = quantity_reserved + :q WHERE ... AND quantity_on_hand - quantity_reserved >= :q`. If it changes zero rows the service raises INSUFFICIENT_STOCK and the whole run rolls back.
3. **Table constraint.** `CHECK (quantity_reserved <= quantity_on_hand)` rejects any statement that would over-reserve, whatever its origin.

12.2 Atomic units of work

***Table 16**: Operations and what each transaction contains*

Operation	Changes made atomically
Create request	request + lines (with approved quantities) + audit row
Commit allocation	run + decisions + allocation lines + shipments + shipment lines + conditional reservations + ledger rows + request counters and statuses + audit
Dispatch	decrement quantity_on_hand and quantity_reserved together + ledger + shipment status (timeline row by trigger) + audit
Deliver	receipts + camp stock increase + request counters (shortfall released for re-allocation) + shipment status + audit
Cancel shipment	release reservation + ledger + request counters + status
Receive / adjust stock	inventory change (adjustment cannot go below reserved) + ledger + audit
Record distribution	stock guard trigger + insert + camp stock decrement

12.3 Shipment life-cycle

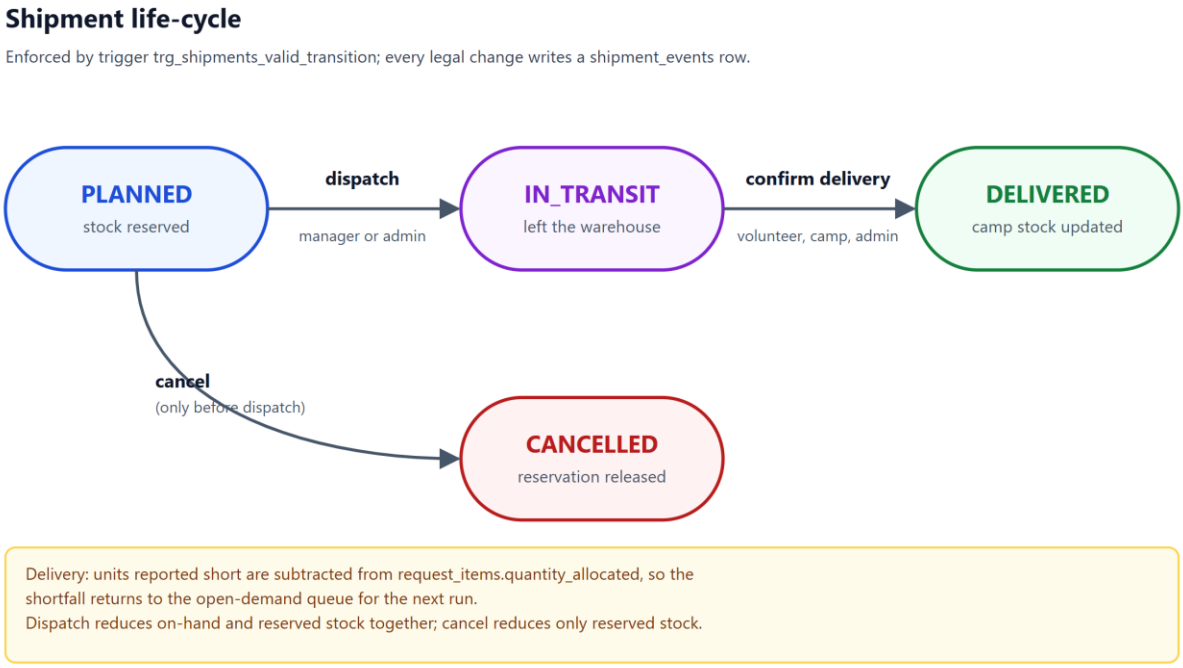


***Figure 5**: Shipment state machine enforced by trg_shipments_valid_transition*

Dispatch converts a reservation into a real stock movement; cancellation before dispatch releases only the reservation. When a delivery reports fewer units than shipped, the shortfall is subtracted from `request_items.quantity_allocated` in the same transaction, so the missing quantity becomes open demand again and is picked up by the next run.

12.4 Ledger and reconciliation

Every change to `inventory` has a matching row in the append-only `stock_movements` ledger (types OPENING, RECEIPT, ADJUSTMENT, RESERVE, RELEASE and DISPATCH, each with a change to on-hand and reserved quantities). Four reconciliation views recompute the derived figures from first principles and must return no rows: `v_ledger_mismatch`, `v_reservation_mismatch`, `v_allocation_mismatch` and `v_camp_stock_mismatch`. The Administration page runs them on demand, and the test-suite runs them after every multi-step scenario.

12.5 Verified behaviour

- **Fault injection:** a database trigger that aborts the audit insert (the last step of an allocation run) leaves zero runs, shipments, decisions, reservations and ledger rows; once the fault is removed the same run succeeds.
- **Concurrent requests:** eight simultaneous allocation requests reserve each unit exactly once (5,000 of 5,000 units, never more).
- **Real two-connection isolation:** while one connection holds a write transaction, a second connection cannot start one, sees only committed data, and cannot over-reserve (`tests/database.test.js`, against a file database).

- **Over-allocation:** after all stock is reserved, a second run allocates zero units; a direct SQL attempt to raise the reservation fails with a CHECK violation.

Honest scope note. Node.js executes the application in one thread and `node:sqlite` is synchronous, so the "eight concurrent requests" test demonstrates the correctness of the logic under interleaved requests rather than true parallelism; genuine multi-connection isolation is covered by the separate two-connection test.

13. SQL OPERATIONS AND EXAMPLE QUERIES

This chapter shows representative SQL from the project. Unlike a purely illustrative listing, **every query result in Section 13.4 was produced by running the query on the demonstration database** seeded with a fixed clock (2026-07-15 12:00 UTC), and every error message in Section 13.3 was captured by executing the statement against a database created from the project's own migration files. All 17 showcase queries are stored in `db/queries.sql` and executed by an automated test.

13.1 Schema definition (excerpt)

The inventory table and the shipment state-machine trigger, exactly as they appear in the migrations:

```
CREATE TABLE inventory (
  warehouse_id      INTEGER NOT NULL REFERENCES warehouses(id),
  item_id           INTEGER NOT NULL REFERENCES supply_items(id),
  quantity_on_hand  INTEGER NOT NULL DEFAULT 0 CHECK (quantity_on_hand  >= 0),
  quantity_reserved INTEGER NOT NULL DEFAULT 0 CHECK (quantity_reserved >= 0),
  reorder_level     INTEGER NOT NULL DEFAULT 0 CHECK (reorder_level >= 0),
  updated_at        TEXT    NOT NULL,
  PRIMARY KEY (warehouse_id, item_id),
  CHECK (quantity_reserved <= quantity_on_hand)
);

CREATE TRIGGER trg_shipments_valid_transition BEFORE UPDATE OF status ON shipments
WHEN OLD.status <> NEW.status AND NOT (
       (OLD.status = 'PLANNED'    AND NEW.status IN ('IN_TRANSIT', 'CANCELLED')) OR
       (OLD.status = 'IN_TRANSIT' AND NEW.status = 'DELIVERED'))
BEGIN SELECT RAISE(ABORT, 'invalid shipment status transition'); END;
```

13.2 Data manipulation as issued by the service layer

```
-- Reserve stock for an allocation line (fails safely if stock changed)
UPDATE inventory
   SET quantity_reserved = quantity_reserved + :qty, updated_at = :now
 WHERE warehouse_id = :wh AND item_id = :item
   AND quantity_on_hand - quantity_reserved >= :qty;      -- changes() must be 1

-- Dispatch: stock leaves the warehouse (reservation becomes a real movement)
UPDATE inventory
   SET quantity_on_hand  = quantity_on_hand  - :qty,
       quantity_reserved = quantity_reserved - :qty
 WHERE warehouse_id = :wh AND item_id = :item
   AND quantity_reserved >= :qty AND quantity_on_hand >= :qty;

-- Delivery with a shortfall: the missing units become open demand again
```

```
UPDATE request_items
   SET quantity_delivered = quantity_delivered + :received,
       quantity_allocated = quantity_allocated - (:shipped - :received)
 WHERE id = :request_item;

-- Receive stock: create the stock row on first receipt, otherwise add to it
INSERT INTO inventory (warehouse_id, item_id, quantity_on_hand, quantity_reserved, reorder_level,
updated_at)
VALUES (:wh, :item, :qty, 0, 0, :now)
ON CONFLICT (warehouse_id, item_id) DO UPDATE
   SET quantity_on_hand = quantity_on_hand + excluded.quantity_on_hand, updated_at =
excluded.updated_at;
```

A safe transaction example in the style of the project's tests (it ends with ROLLBACK so it can be run on a copy without changing data):

```
BEGIN IMMEDIATE;
UPDATE inventory SET quantity_reserved = quantity_reserved + 100
 WHERE warehouse_id = 1 AND item_id = 5 AND quantity_on_hand - quantity_reserved >= 100;
SELECT changes();                     -- 1 = reserved, 0 = not enough available stock
ROLLBACK;
```

13.3 Constraints and triggers rejecting invalid operations (captured output)

Each statement below was executed against a database built from the migrations (with a minimal set of rows); the message is the database's own.

***Table 17**: Captured database responses to invalid statements*

Attempted operation	Statement (first line)	Database response
Reserve more stock than is on hand	`UPDATE inventory SET quantity_reserved = 101 …`	IntegrityError: CHECK constraint failed: quantity_reserved <= quantity_on_hand
Skip a step in the shipment life-cycle	`UPDATE shipments SET status = 'DELIVERED' WHERE id = 1;`	IntegrityError: invalid shipment status transition
Edit the append-only stock ledger	`UPDATE stock_movements SET delta_on_hand = 999;`	IntegrityError: stock_movements is append-only
Hand out more than the camp holds	`INSERT INTO distributions (camp_id, item_id, quantity, people_served, distributed_by, created_at) …`	IntegrityError: distribution exceeds stock held at the camp
Refer to a camp that does not exist	`INSERT INTO requests (camp_id, requested_by, urgency, created_at, updated_at) …`	IntegrityError: FOREIGN KEY constraint failed

13.4 Showcase queries and their results

13.4.1 Supply versus open demand per item

View + aggregation: supply vs open demand per item, worst coverage first.

```
SELECT item_name, unit, available, open_demand, shortage, ROUND(100 * coverage) AS pct_coverable
FROM v_item_supply_demand
WHERE open_demand > 0
ORDER BY coverage, open_demand DESC
```

***Table 18**: Result of q01 (first 6 of 12 rows)*

item_name	unit	available	open_demand	shortage	pct_coverable
Insulin vials	vial	30	116	86	26
Tarpaulin sheets	sheet	380	1,320	940	29
ORS sachets	sachet	1,000	2,928	1,928	34
Packaged drinking water	L	21,000	58,535	37,535	36
Ready-to-eat meal packs	pack	1,550	3,100	1,550	50
Rice	kg	4,250	1,630	0	100

13.4.2 Camps ranked by unmet need (JOIN, GROUP BY, HAVING)

Multi-table JOIN, GROUP BY, HAVING: camps with the largest outstanding share of approved quantities.

```
SELECT c.name AS camp, c.population,
       COUNT(ri.id)                                                    AS open_lines,
       ROUND(AVG(1.0 * (ri.quantity_approved - ri.quantity_allocated) / ri.quantity_approved), 3) AS
avg_unmet_share
FROM camps c
JOIN requests r      ON r.camp_id = c.id AND r.status IN ('PENDING', 'PARTIALLY_ALLOCATED')
JOIN request_items ri ON ri.request_id = r.id AND ri.quantity_allocated < ri.quantity_approved
GROUP BY c.id
HAVING COUNT(ri.id) >= 2
ORDER BY avg_unmet_share DESC, open_lines DESC
```

***Table 19**: Result of q02 (first 6 of 12 rows)*

camp	population	open_lines	avg_unmet_share
Kuthalam Youth Hostel	230	6	0.924
Tharangambadi Town Hall	270	7	0.861
Kollidam Community Hall	460	7	0.853
Sirkazhi Road Marriage Hall	380	7	0.852
Kodiyakarai Fishermen Shelter	540	6	0.808
Mayiladuthurai Polytechnic Camp	700	7	0.762

13.4.3 Nearest warehouse still holding water, per camp (CTE and ROW_NUMBER)

CTE + ROW_NUMBER: the closest warehouse that still has water available, for every camp.

```
WITH ranked AS (
  SELECT c.name AS camp, w.name AS warehouse, i.quantity_on_hand - i.quantity_reserved AS available,
         ROW_NUMBER() OVER (
           PARTITION BY c.id
           ORDER BY (w.latitude - c.latitude) * (w.latitude - c.latitude) + (w.longitude - c.longitude) *
(w.longitude - c.longitude)
         ) AS rn
  FROM camps c
  JOIN inventory i  ON i.item_id = (SELECT id FROM supply_items WHERE name = 'Packaged drinking water')
  JOIN warehouses w ON w.id = i.warehouse_id AND w.status = 'ACTIVE'
  WHERE c.status <> 'CLOSED' AND i.quantity_on_hand - i.quantity_reserved > 0
)
SELECT camp, warehouse, available FROM ranked WHERE rn = 1 ORDER BY camp
```

***Table 20**: Result of q05 (first 6 of 12 rows)*

camp	warehouse	available
Anaikarai Panchayat Union School	Thanjavur Central Relief Depot	11,000
Kodiyakarai Fishermen Shelter	Nagapattinam Port Warehouse	8,000
Kollidam Community Hall	Nagapattinam Port Warehouse	8,000
Kuthalam Youth Hostel	Tiruvarur District Store	2,000
Marudhur Govt. Higher Secondary School	Thanjavur Central Relief Depot	11,000
Mayiladuthurai Polytechnic Camp	Tiruvarur District Store	2,000

13.4.4 Fairness check: coverage by urgency in each scarce run

Did higher urgency get higher cumulative coverage when stock was short?

```
SELECT d.run_id, ul.label AS urgency, COUNT(*) AS lines,
       ROUND(AVG(1.0 * (d.prior_qty + d.allocated_qty) / d.approved_qty), 3) AS avg_cumulative_coverage
FROM allocation_decisions d
JOIN request_items ri ON ri.id = d.request_item_id
JOIN requests r       ON r.id = ri.request_id
JOIN urgency_levels ul ON ul.level = r.urgency
WHERE d.scarce = 1
GROUP BY d.run_id, ul.level
ORDER BY d.run_id, ul.level DESC
```

***Table 21**: Result of q06 (8 rows)*

run_id	urgency	lines	avg_cumulative_coverage
1	Critical	9	0.922
1	High	15	0.527
1	Moderate	9	0.305
1	Low	3	0.136
2	Critical	12	0.763
2	High	18	0.543
2	Moderate	9	0.338
2	Low	3	0.152

13.4.5 Losses in transit by route

Aggregation over a self-consistent subset: units reported short at delivery, by warehouse and camp.

```
SELECT w.name AS warehouse, c.name AS camp,
       SUM(si.quantity) AS shipped, SUM(si.quantity_received) AS received,
       SUM(si.quantity - si.quantity_received) AS lost
FROM shipment_items si
JOIN shipments s  ON s.id = si.shipment_id AND s.status = 'DELIVERED'
JOIN warehouses w ON w.id = s.warehouse_id
JOIN camps c      ON c.id = s.camp_id
WHERE si.quantity_received < si.quantity
GROUP BY w.id, c.id
ORDER BY lost DESC
```

***Table 22**: Result of q08 (4 rows)*

warehouse	camp	shipped	received	lost
Thanjavur Central Relief Depot	Marudhur Govt. Higher Secondary School	12,189	11,335	854
Nagapattinam Port Warehouse	Thalaignayar Cyclone Shelter	12,080	11,234	846
Nagapattinam Port Warehouse	Kodiyakarai Fishermen Shelter	7,920	7,365	555
Thanjavur Central Relief Depot	Thalaignayar Cyclone Shelter	846	786	60

13.4.6 Camps with an open Critical request and no shipment on the way (NOT EXISTS)

Correlated NOT EXISTS: camps with an open Critical request and no shipment on the way.

```
SELECT c.name AS camp, COUNT(*) AS open_critical_requests
FROM camps c
JOIN requests r ON r.camp_id = c.id AND r.urgency = 4 AND r.status IN ('PENDING', 'PARTIALLY_ALLOCATED')
WHERE NOT EXISTS (SELECT 1 FROM shipments s WHERE s.camp_id = c.id AND s.status IN ('PLANNED', 'IN_TRANSIT'))
GROUP BY c.id
ORDER BY open_critical_requests DESC
```

***Table 23**: Result of q11 (3 rows)*

camp	open_critical_requests
Marudhur Govt. Higher Secondary School	3
Thalaignayar Cyclone Shelter	3
Kodiyakarai Fishermen Shelter	2

13.4.7 Units delivered per volunteer

Join across four tables: units delivered by each volunteer.

```
SELECT u.full_name AS volunteer, COUNT(DISTINCT s.id) AS shipments, SUM(si.quantity_received) AS
units_delivered
FROM users u
JOIN shipments s       ON s.volunteer_id = u.id AND s.status = 'DELIVERED'
JOIN shipment_items si ON si.shipment_id = s.id
WHERE u.role = 'volunteer'
GROUP BY u.id
ORDER BY units_delivered DESC
```

***Table 24**: Result of q12 (5 rows)*

volunteer	shipments	units_delivered
Asha Raghavan	8	29,907
Ravi Kumaran	8	25,887
Meena Sundar	8	22,871
Karthik Pandian	8	15,940
Fatima Begum	8	13,297

13.4.8 Deliveries per day (recursive CTE calendar)

Recursive CTE builds a calendar so days without deliveries still show up.

```
WITH RECURSIVE days(d) AS (
```

```
  SELECT date(MIN(delivered_at)) FROM shipments WHERE status = 'DELIVERED'
  UNION ALL
  SELECT date(d, '+1 day') FROM days WHERE d < (SELECT date(MAX(delivered_at)) FROM shipments WHERE status =
'DELIVERED')
)
SELECT d AS day, COUNT(s.id) AS shipments_delivered
FROM days LEFT JOIN shipments s ON date(s.delivered_at) = d AND s.status = 'DELIVERED'
GROUP BY d
ORDER BY d
```

***Table 25**: Result of q14 (3 rows)*

day	shipments_delivered
2026-07-13	5
2026-07-14	33
2026-07-15	2

13.4.9 Integrity report (UNION ALL of the reconciliation views)

UNION ALL of the reconciliation views: an empty result means the database is self-consistent.

```
SELECT 'ledger'      AS check_name, COUNT(*) AS problems FROM v_ledger_mismatch
UNION ALL SELECT 'reservations', COUNT(*) FROM v_reservation_mismatch
UNION ALL SELECT 'allocations',  COUNT(*) FROM v_allocation_mismatch
UNION ALL SELECT 'camp_stock',   COUNT(*) FROM v_camp_stock_mismatch
```

***Table 26**: Result of q15 (4 rows)*

check_name	problems
ledger	0
reservations	0
allocations	0
camp_stock	0

Results are an output of the seeded demonstration data and will differ if the database is re-seeded with a different clock or modified through the application.

14. APPLICATION MODULES AND API DESIGN

14.1 Frontend modules

The front end is a single-page application written as ES modules without a build step. A router maps hash routes to nine page modules; navigation is filtered by role. All dynamic text is rendered through an auto-escaping template function, which prevents cross-site scripting by default. Screenshots of each module are in Appendix A.

***Table 27**: Frontend modules*

Module	Purpose	Main users
Sign-in	Authentication with local demo-account shortcuts	All

Module	Purpose	Main users
Dashboard	KPI cards, situation map (camps coloured by urgency, shipment routes), alerts, supply versus open demand, urgency donut, recent activity; refreshes every 30 s	All
Requests	Filterable list; new-request dialog with live planning-cap hints; request page showing requested → approved → allocated → delivered and the explanation history	Coordinators, admin
Allocation	Method summary, preview with per-line score breakdown and explanations, commit with confirmation, run history and run detail	Admin (read-only for others)
Shipments	Status board, dispatch, assign, claim, cancel, confirm-delivery dialog with per-line quantities, timeline	Managers, volunteers, coordinators
Inventory	Per-warehouse and network stock, receive, adjust, reorder level, ledger viewer	Managers, admin
Relief camps	Camp cards and detail pages; census update	All
Distribution	Record aid handed out; camp stock; refusal when stock is insufficient	Coordinators, admin
Analytics	Coverage by urgency, time to first delivery, deliveries per day, committed stock, shortages, camp coverage, runs, losses	All
Administration	Users, allocation settings, audit trail, data-integrity checks	Admin

14.2 Backend API

The server exposes 56 JSON endpoints under `/api` (counted from the route definitions). Every endpoint except sign-in requires a bearer token; role gates and scope checks are applied on top, and every error is returned in a uniform structure with a stable code. The table below lists the endpoints grouped by area; “all” means every signed-in role.

***Table 28**: REST API endpoints and access rules*

Method	Endpoint	Access	Purpose
POST	/api/auth/login	public (rate-limited)	Sign in; returns a bearer token and the user
POST	/api/auth/logout	any signed-in user	Delete the session
GET	/api/auth/me	any signed-in user	Current user
GET	/api/auth/demo-accounts	public, local builds only	Sign-in shortcuts for the seeded demo users
GET	/api/dashboard	all roles	KPIs, map data, alerts, supply versus demand, activity
GET	/api/alerts	all roles	Shortage, SLA, low-stock and delay alerts
GET	/api/analytics	all roles	Coverage and delivery analytics
GET	/api/integrity	admin	Run the four reconciliation views
GET	/api/audit	admin	Audit trail
GET / POST / PATCH	/api/disasters[/:id]	all / admin / admin	List, create and change incident status
GET	/api/categories	all roles	Supply categories
GET / PUT	/api/urgency-levels[/:level]	all / admin	Read; edit weight and target hours
GET / POST / PATCH	/api/items[/:id]	all / admin / admin	Supply catalogue
GET / POST / PATCH	/api/warehouses[/:id]	all / admin / admin	Warehouses with stock totals; create; set status

Method	Endpoint	Access	Purpose
GET	/api/camps	all roles	Camp list with open requests and urgency
GET	/api/camps/:id	all roles	Camp detail: requests, stock, shipments
POST	/api/camps	admin	Create a camp
PATCH	/api/camps/:id	admin; coordinator of that camp	Update census (population, at-risk); status is admin-only
GET	/api/camps/:id/stock	all roles	Stock held at a camp
GET	/api/volunteers	admin, warehouse manager	Active volunteers (for assignment)
GET / POST / PATCH	/api/users[/:id]	admin	List, create, activate/deactivate, reset password
GET	/api/inventory	all roles	Inventory view (filter by warehouse or item)
GET	/api/inventory/movements	all roles	Stock ledger
POST	/api/inventory/receive	admin; manager of that warehouse	Receive stock
POST	/api/inventory/adjust	admin; manager of that warehouse	Adjust stock with a reason (not below reserved)
POST	/api/inventory/reorder-level	admin; manager of that warehouse	Set reorder level
GET	/api/requests	all roles	Requests (filter by status, camp, urgency)
GET	/api/requests/:id	all roles	Request detail with lines, decisions and shipments
POST	/api/requests	admin; coordinator of that camp	Create a request
POST	/api/requests/:id/cancel	admin; coordinator of that camp	Cancel (no active shipments)
POST	/api/requests/:id/urgency	admin; coordinator (raise only)	Change urgency
POST	/api/request-items/:id/approval	admin	Override an approved quantity
GET / PUT	/api/allocation/settings	all / admin	Read; edit allocation parameters
POST	/api/allocation/preview	admin	Compute an allocation without changing data
POST	/api/allocation/run	admin	Commit an allocation run (one transaction)
GET	/api/allocation/runs[/:id]	all roles	Run history and run detail
GET	/api/shipments[/:id]	all roles	Shipment list and detail with timeline
POST	/api/shipments/:id/assign	admin; manager of that warehouse	Assign a volunteer
POST	/api/shipments/:id/dispatch	admin; manager of that warehouse	Dispatch
POST	/api/shipments/:id/cancel	admin; manager of that warehouse	Cancel a planned shipment
POST	/api/shipments/:id/claim	volunteer	Claim an unassigned shipment
POST	/api/shipments/:id/deliver	admin; receiving coordinator; assigned volunteer	Confirm delivery with received quantities
GET / POST	/api/distributions	all / admin, coordinator of that camp	List and record distributions

14.3 Security design

- **Passwords** are hashed with scrypt and a per-password random salt; comparison is constant-time, and a dummy hash is verified for unknown users so timing does not reveal which usernames exist.
- **Sessions** use random 256-bit tokens stored only as SHA-256 hashes, expire after 12 hours and are deleted on logout, deactivation and password reset. Tokens travel in an Authorization header, so classic CSRF does not apply.
- **Authorisation** is declared per route (role) and per object (own camp, own warehouse, assigned shipment); the users table CHECK keeps role and scope consistent.
- **Input handling:** all SQL uses prepared statements with bound parameters; every field is validated for type, range and length; JSON bodies are limited to 100 KB; unknown enumeration values are rejected.
- **Browser protections:** Content-Security-Policy `default-src 'self'`, X-Content-Type-Options, X-Frame-Options DENY and a no-referrer policy.
- **Abuse control:** after eight failed sign-ins within ten minutes for a given address and username, further attempts return HTTP 429; every failed attempt is audited.

15. TESTING AND RESULTS

15.1 Testing strategy

Testing covers the allocation algorithm in isolation (including randomised property tests), the database schema (constraints, triggers, views, indexes, migrations and isolation), the HTTP API end to end (authentication, role-based access, lifecycle and concurrency), realistic scenarios on the seeded incident, and the SQL showcase queries. The user interface was verified manually in a browser and with scripted screenshots.

15.2 Environment and how to read the results

***Table 29**: Verification environment*

Item	Value
Operating system	Windows 11 Home (10.0.26200)
Runtime	Node.js v22.19.0
Database engine	SQLite 3.50.4 (built into Node.js)
Test runner	`node:test` (built in), spec reporter
Date of the verification run	October 2026

Reading guide. *Verified* results in this chapter were produced by running the commands listed in Appendix B on the submitted code. *Illustrative* material (diagrams, the analytical Example A in Chapter 11, and the fictional demonstration data) is labelled as such. No result in this report has been estimated or invented. Allocation tables use a fixed clock so that they are exactly reproducible; screenshots were captured from the running application seeded at capture time, so hour-dependent figures (waiting time, “hours ago”) differ slightly from the fixed-clock tables.

15.3 Automated test suite

***Table 30**: Automated tests by file*

Test file	Tests	What it verifies
tests/engine.test.js	18	Closed-form allocation cases, deficit awareness, 600-scenario property test against an independent solver, 200-scenario monotonicity test, determinism, scoring, planning cap, sourcing and distance
tests/database.test.js	14	Migrations and checksum tamper detection, foreign keys, CHECK constraints, role-scope constraint, append-only history, shipment state machine and timeline, business-rule triggers, views, reconciliation, `EXPLAIN QUERY PLAN` index use, tx() semantics, real two-connection isolation
tests/api.test.js	13	Authentication, rate limiting, role and scope matrix, validation, atomic preview/commit, over-allocation, concurrent runs, fault-injection rollback, full lifecycle with partial delivery and re-allocation, cancellation, inventory floor, dashboards, input hardening
tests/scenarios.test.js	5	Seeded incident is internally consistent; scarce items rationed by priority; Critical > High > Moderate > Low coverage; commit through the API; role journeys
tests/queries.test.js	21	All 17 showcase queries run on the demo data; integrity report clean; ledger replay ends at the inventory balance; coverage ordered by urgency
Total	**71**	**71 passed, 0 failed, 0 skipped; total duration 4.3 s**

The complete list of the 71 tests, as printed by the test runner, is in Appendix C.

15.4 Allocation scenarios (verified, fixed clock)

`npm run scenarios` runs the engine against hand-built situations and against three baseline rules: first-come-first-served, equal share of need (ignoring urgency) and strict priority (winner takes all). **Coverage** is the share of a camp's approved quantity that it holds after the run.

Scenario 2: one water convoy for six camps (19,000 L for 48,400 L of demand)

***Table 31**: Units / coverage per camp and method (Scenario 2)*

Camp	Urgency	Waited	Needs (L)	Priority	ReliefGrid	First come	Equal share	Strict priority
Riverside School	Critical	5h	12,000	8.08	7,728 / 64%	0 / 0%	4,711 / 39%	12,000 / 100%
Temple Hall	High	20h	4,500	5.58	2,002 / 44%	4,500 / 100%	1,766 / 39%	4,500 / 100%
Stadium	High	12h	14,000	5.00	5,582 / 40%	7,100 / 51%	5,496 / 39%	2,500 / 18%
Town Hall	Moderate	30h	4,000	3.20	1,020 / 26%	4,000 / 100%	1,570 / 39%	0 / 0%
Polytechnic	Moderate	8h	10,500	2.62	2,194 / 21%	0 / 0%	4,122 / 39%	0 / 0%
Youth Hostel	Low	44h	3,400	1.75	474 / 14%	3,400 / 100%	1,335 / 39%	0 / 0%

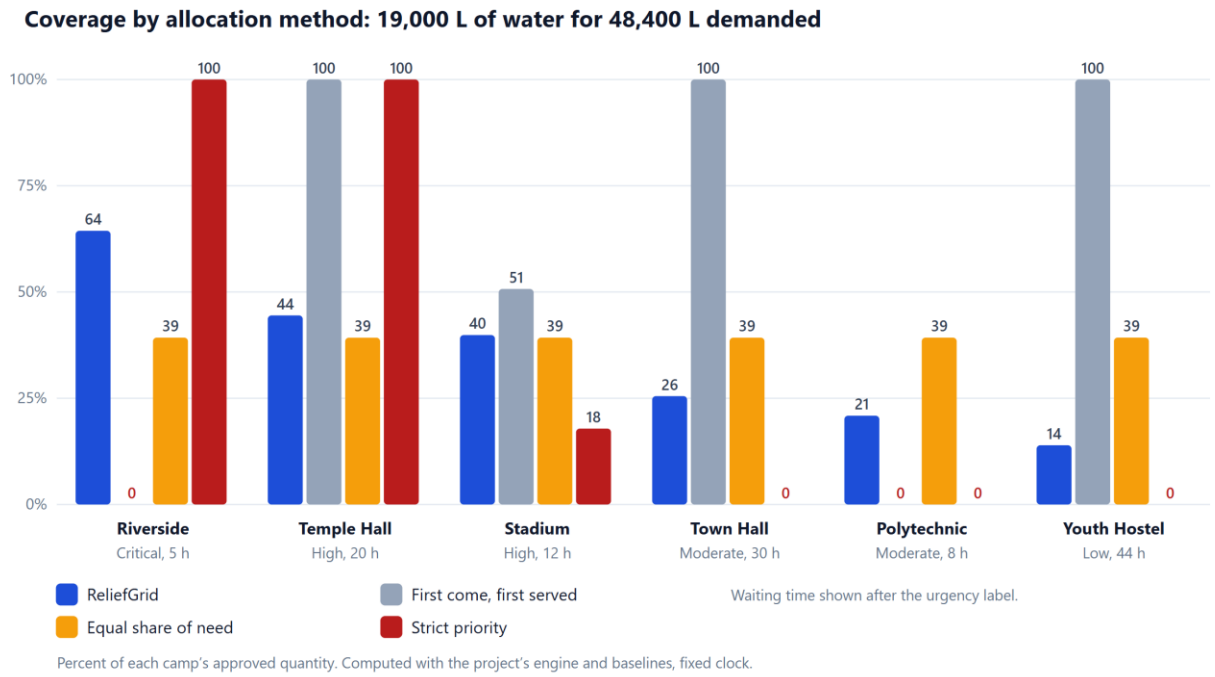


***Figure 6**: Coverage of each camp by allocation method (Scenario 2, verified)*

***Table 32**: Summary metrics by method (Scenario 2)*

Method	Critical coverage	Low coverage	Worst-off camp	Camps left with nothing	Jain index	Units used
ReliefGrid	64%	14%	14%	0	0.81	19,000
First come, first served	0%	100%	0%	2	0.63	19,000
Equal share of need (ignores urgency)	39%	39%	39%	0	1.00	19,000
Strict priority (winner takes all)	100%	0%	0%	3	0.39	19,000

First-come-first-served gives the long-waiting low-urgency hostel everything it asked for while the Critical camp, which asked last, receives nothing. Equal sharing treats a Critical camp like a Low one. Strict priority serves the top request completely and leaves the three lowest-priority camps with nothing. ReliefGrid gives the Critical camp the largest share of what it needs (64%), every camp something (worst-off 14%) and rewards waiting. The Jain fairness index is highest for equal shares by construction; ReliefGrid deliberately trades some of it for priority, so it must be read together with the Critical and worst-off columns.

Scenario 3: extreme scarcity (30 insulin vials for 116 requested)

***Table 33**: Summary metrics by method (Scenario 3)*

Method	Critical coverage	Low coverage	Worst-off camp	Camps left with nothing	Jain index	Units used
ReliefGrid	30%	–	19%	0	0.95	30
First come, first served	0%	–	0%	3	0.25	30
Equal share of need (ignores urgency)	25%	–	24%	0	1.00	30
Strict priority (winner takes all)	56%	–	0%	2	0.30	30

Scenario 6: waiting is rewarded without overturning urgency

***Table 34**: Aging example (Scenario 6): 900 units for three requests of 1,000*

Request	Urgency weight	Vulnerability	Aging	Priority	Allocated of 1,000	Coverage
Low urgency, waited 72h	1	1.20	1.50	1.80	300	30%
Low urgency, just asked	1	1.20	1.00	1.20	200	20%
Moderate, just asked	2	1.20	1.00	2.40	400	40%

15.5 Seeded incident: end-to-end results

The seed script replays three days of fictional operations through the real services (opening stock, wave of requests, two historical allocation runs, dispatches, partial deliveries and 231 distributions) and leaves 29 requests (178 request lines) in the system. At the fixed clock **91 request lines** are still open and **5 items are scarce**.

***Table 35**: Stock versus open demand per item at the seeded state (verified)*

Item	In stock	Open demand	Coverable	Status
Rice	4,250 kg	1,630 kg	100%	sufficient
Ready-to-eat meal packs	1,550 pack	3,100 pack	50%	scarce, rationed
Packaged drinking water	21,000 L	58,535 L	36%	scarce, rationed
ORS sachets	1,000 sachet	2,928 sachet	34%	scarce, rationed
Insulin vials	30 vial	116 vial	26%	scarce, rationed
Tarpaulin sheets	380 sheet	1,320 sheet	29%	scarce, rationed
Blankets	500 piece	350 piece	100%	sufficient
Sleeping mats	1,850 piece	240 piece	100%	sufficient
Soap bars	830 bar	174 bar	100%	sufficient
Sanitary napkin packs	1,150 pack	210 pack	100%	sufficient
Dry clothing sets	3,050 set	120 set	100%	sufficient
Solar lanterns	300 piece	140 piece	100%	sufficient

***Table 36**: Packaged drinking water: a preview run over request lines that started from zero (verified)*

Camp	Urgency	Priority	Approved (L)	Allocated (L)	Coverage
Kodiyakarai Fishermen Shelter	Critical	8.53	2,700	2,613	97%
Marudhur Govt. Higher Secondary School	Critical	8.16	4,100	3,796	93%
Thalaignayar Cyclone Shelter	Critical	8.02	5,750	5,228	91%
Poompuhar Community Hall	High	5.21	2,350	1,387	59%
Nagoor Indoor Stadium	High	5.02	4,750	2,702	57%
Vedaranyam Temple Mandapam	High	4.95	1,550	871	56%
Anaikarai Panchayat Union School	Moderate	3.05	3,200	1,107	35%
Sirkazhi Road Marriage Hall	Moderate	2.91	1,900	626	33%
Mayiladuthurai Polytechnic Camp	Moderate	2.88	3,500	1,145	33%
Kollidam Community Hall	Moderate	2.69	2,300	703	31%
Tharangambadi Town Hall	Low	1.43	1,350	218	16%
Kuthalam Youth Hostel	Low	1.22	1,150	159	14%

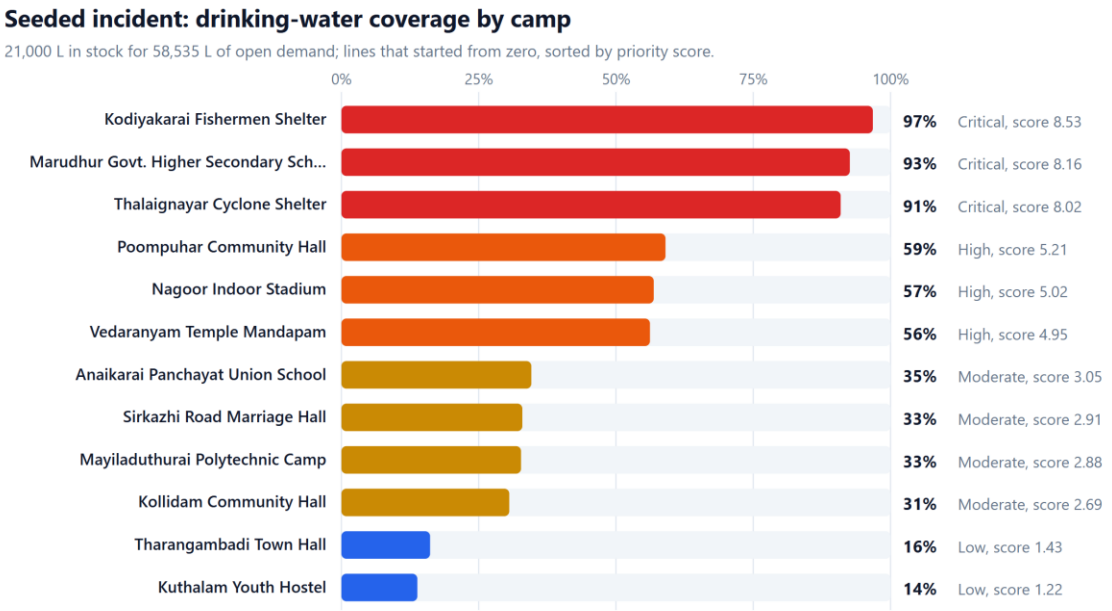


***Figure 7**: Coverage of drinking water by camp for the seeded incident (verified, fixed clock)*

Critical camps end up between 91% and 97% covered, High camps at 56–59%, Moderate camps at 31–35% and Low camps at 14–16%, while no camp receives zero. Query q06 (Section 13.4.4) confirms the same ordering in the historical runs; in run 1 the average cumulative coverage was 0.922 for Critical, 0.527 for High, 0.305 for Moderate and 0.136 for Low request lines.

Commit of a full run (verified): 5 ms in one transaction; 28 shipments planned; 26,824 units reserved; reserved stock (27,681) never exceeds stock on hand (41,852); integrity checks: all passed.

15.6 Performance of the allocation engine

***Table 37**: Allocation engine timing on synthetic data (12 warehouses, 18 items)*

Request lines	Warehouses	Allocation time (pure engine)	Never exceeds stock
100	12	2 ms	yes
500	12	6 ms	yes
2,000	12	16 ms	yes
5,000	12	40 ms	yes

Timings are for the pure engine on the machine described in Section 15.2 and will vary with hardware. Each item is solved independently in $O(n^2)$ in the number of its own request lines, so even thousands of open lines allocate in a small fraction of a second. These figures concern the algorithm only; database throughput was not load-tested.

15.7 Integrity results

- Query q15 over the four reconciliation views returned **zero problems** for ledger, reservations, allocations and camp stock on the seeded database.
- The integrity page in the running application reported “All 4 checks passed” after the full set of scripted UI operations (allocation commit, dispatch, partial delivery, new request, distribution), as shown in Appendix A.
- A test deliberately tampers with inventory and request counters and confirms that the reconciliation views detect it.

15.8 Manual end-to-end verification in a browser

***Table 38**: Browser verification (manual and scripted)*

Area	What was checked	Result
Sign-in	All four roles via demo shortcuts; sign-out; role-specific navigation	Passed
Dashboard	KPIs, situation map with routes and critical markers, alerts, supply bars, donut, activity feed	Passed
Allocation	Preview of 91 request lines; explanation expanded; confirmation; commit; run-detail page with shipment codes	Passed
Shipments	Manager dispatch; volunteer claim and partial delivery (shortfall reported); admin cancellation with reason; trigger-written timeline	Passed
Requests	Coordinator creates a request; planning-cap hint; trimmed quantity and reason on the request page	Passed
Distribution	Record a distribution; an over-stock attempt is refused with the server’s message	Passed
Camps, Inventory, Analytics, Admin	Camp grid and detail; warehouse tabs and ledger; charts; users table; settings edit changes the next preview; integrity check	Passed
Responsive layout	375 px viewport: off-canvas navigation; no horizontal overflow on any of the nine pages (measured)	Passed

Defects found and fixed during this process: card grids rendering stray commas, a map whose bounds were distorted by a closed camp in another district, misaligned supply bars, a “Low” badge on items with no reorder level, a misleading “people served” label (now “aid servings”) and two charts that mixed incompatible units. Testing multi-round scenarios also exposed a real flaw in the first version of the algorithm (an already well-served request competed again as if it had nothing); the method was made deficit-aware and is now covered by tests.

15.9 Summary

***Table 39**: Summary of verified outcomes*

Claim	Evidence	Status
All automated tests pass	71/71 passed in 4.3 s	Verified
Stock cannot be over-allocated	CHECK constraint, conditional update, 8-request concurrency test, fault-injection rollback	Verified
History is immutable	Trigger tests on ledger, decisions, lines, events and audit log	Verified
Allocation follows priority and never starves a camp	Property tests; Scenario 2 and seeded water results	Verified
Allocation is fair across rounds	Deficit-awareness tests; Scenario 5	Verified
Database self-consistency	q15 and the integrity page: zero problems	Verified
Responsive, role-aware interface	Manual and scripted browser checks; screenshots	Verified (manual)
Behaviour under heavy multi-user load or on other browsers	Not tested	Not verified

16. LIMITATIONS AND FUTURE ENHANCEMENTS

16.1 Limitations

- **Single-writer database.** SQLite serialises writers. This gives simple, strong isolation and suits one control room, but it is not designed for many write-heavy nodes; the Node.js `node:sqlite` module is also marked experimental in Node 22.
- **Greedy sourcing.** Warehouses are chosen in priority order, nearest first; this is explainable but not a global minimum-distance assignment, and there are no vehicle capacities, road closures or real routing.
- **Items are rationed independently.** A camp’s very high priority for water does not trade off against its need for blankets.
- **Declared data are trusted.** Urgency, population and at-risk counts are entered by coordinators; the planning cap, audit trail and administrator overrides are safeguards, not proof of honesty.
- **Linear aging and vulnerability.** The forms are simple, tunable parameters and have not been calibrated against real operational outcomes.
- **Schematic map.** The situation map is offline and schematic (no map tiles); distances are straight-line estimates multiplied by a detour factor.

- **No notifications or file attachments;** no TLS termination (intended to sit behind a reverse proxy); login throttling is per process and held in memory.
- **Demonstration data only.** All data are fictional; planning standards are illustrative assumptions.
- **Testing gaps.** No automated browser tests, no load tests beyond the engine benchmark, no cross-browser or assistive-technology audit.

16.2 Future enhancements

- Port the database layer to PostgreSQL (row-level locks with `SELECT ... FOR UPDATE`, trigger functions in PL/pgSQL) for multi-node deployment, and add read replicas for dashboards.
- Replace greedy sourcing with a minimum-cost flow that accounts for distance, vehicle capacity and road status, while keeping the explanation text.
- Integrate real map tiles and road routing, GPS tracking of shipments and offline-first mobile screens for volunteers.
- Add SMS/e-mail/push notifications for shortage alerts and delivery confirmations.
- Learn or calibrate urgency, aging and vulnerability weights from historical outcomes and expose a what-if simulator for administrators.
- Support cross-item trade-offs (bundles such as a family kit) and expiry-aware (first-expiry-first-out) allocation for perishable items.
- Add automated browser tests, accessibility audits and load tests; add multi-factor authentication and fine-grained audit export.
- Exchange data with government and NGO systems through documented APIs, with explicit provenance fields.

17. CONCLUSION

ReliefGrid demonstrates how database design can make a high-stakes coordination problem safer and fairer. The project organises incidents, camps, warehouses, supply items, stock, requests, allocation decisions, shipments and distributions into a normalised relational model with 22 tables, 9 views, 21 triggers and 24 indexes. Constraints, triggers and reconciliation views move integrity from convention into the database: stock cannot be over-reserved, history cannot be rewritten, shipments cannot skip states, and derived figures can always be checked against the ledgers they come from.

The allocation method turns urgency, camp population and vulnerability, waiting time and available stock into an explainable decision. By levelling cumulative coverage it stays fair across successive rounds, and by solving the fair level exactly it is deterministic and testable. Executing allocation, reservation, dispatch and delivery as single transactions with three layers of protection means that failures leave the database unchanged, and the immutable decision records mean that every outcome can be audited later.

The verified results support these claims: all 71 automated tests pass; the reconciliation views report no problems; in the seeded shortage Critical camps receive about 91–97% of their drinking-water need, Low-urgency camps about 14–16% and no camp receives nothing; and the method outperforms first-come-first-served, equal sharing and strict priority on the metrics reported. The prototype's limits — a

single-writer embedded database, greedy sourcing, fictional data and no load or browser-automation testing — are stated openly, and the future-work list shows how each could be addressed. Overall, the project applies core DBMS concepts (modelling, normalisation, constraints, triggers, views, indexing, transactions and analytical SQL) to a real-world resource-allocation problem.

REFERENCES

1. SQLite Consortium. SQLite Documentation. https://www.sqlite.org/docs.html
2. SQLite Consortium. Transaction control, including BEGIN IMMEDIATE. https://www.sqlite.org/lang_transaction.html
3. SQLite Consortium. CREATE TRIGGER; Window Functions; Write-Ahead Logging. https://www.sqlite.org/lang_createtrigger.html, https://www.sqlite.org/windowfunctions.html, https://www.sqlite.org/wal.html
4. Node.js Foundation. Node.js documentation: SQLite (`node:sqlite`), Crypto and the test runner. https://nodejs.org/api/
5. Express. Express 5.x Documentation. https://expressjs.com/
6. E. F. Codd. A Relational Model of Data for Large Shared Data Banks. Communications of the ACM, 13(6), 1970.
7. R. Elmasri and S. B. Navathe. Fundamentals of Database Systems. Pearson.
8. A. Silberschatz, H. F. Korth and S. Sudarshan. Database System Concepts. McGraw-Hill.
9. D. Bertsekas and R. Gallager. Data Networks (2nd ed.), Prentice Hall, 1992 (max-min fair allocation).
10. R. Jain, D.-M. Chiu and W. Hawe. A Quantitative Measure of Fairness and Discrimination for Resource Allocation in Shared Computer Systems. DEC Technical Report TR-301, 1984.
11. C. Percival and S. Josefsson. The scrypt Password-Based Key Derivation Function. RFC 7914, IETF, 2016.
12. The Sphere Association. The Sphere Handbook: Humanitarian Charter and Minimum Standards in Humanitarian Response (background for per-person planning standards; the standards used in ReliefGrid are illustrative). https://spherestandards.org/handbook/
13. Project documentation prepared for ReliefGrid: README.md, docs/ARCHITECTURE.md, docs/DATABASE.md, docs/ALLOCATION.md, docs/RESULTS.md and docs/TESTING.md (October 2026).

APPENDIX A — APPLICATION SCREENSHOTS

All screenshots were captured from the running application by a script that signs in as each role in turn and operates the real interface (`report/build/screens.mjs`). The database was freshly seeded, so allocation, dispatch, delivery and request actions shown here are the genuine results of those operations. The light theme is used for print; the application also has a dark theme.

A.1 Sign-in

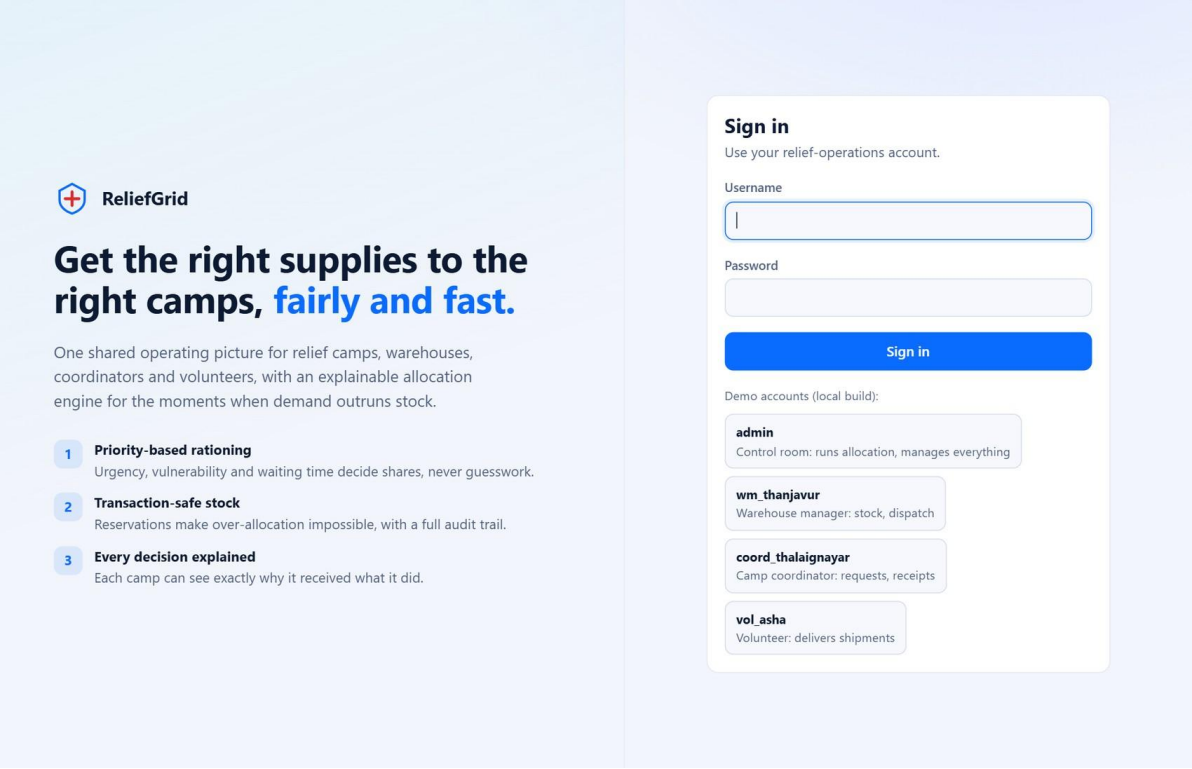


***Figure 8**: Sign-in page with the local demo-account shortcuts*

A.2 Operations dashboard

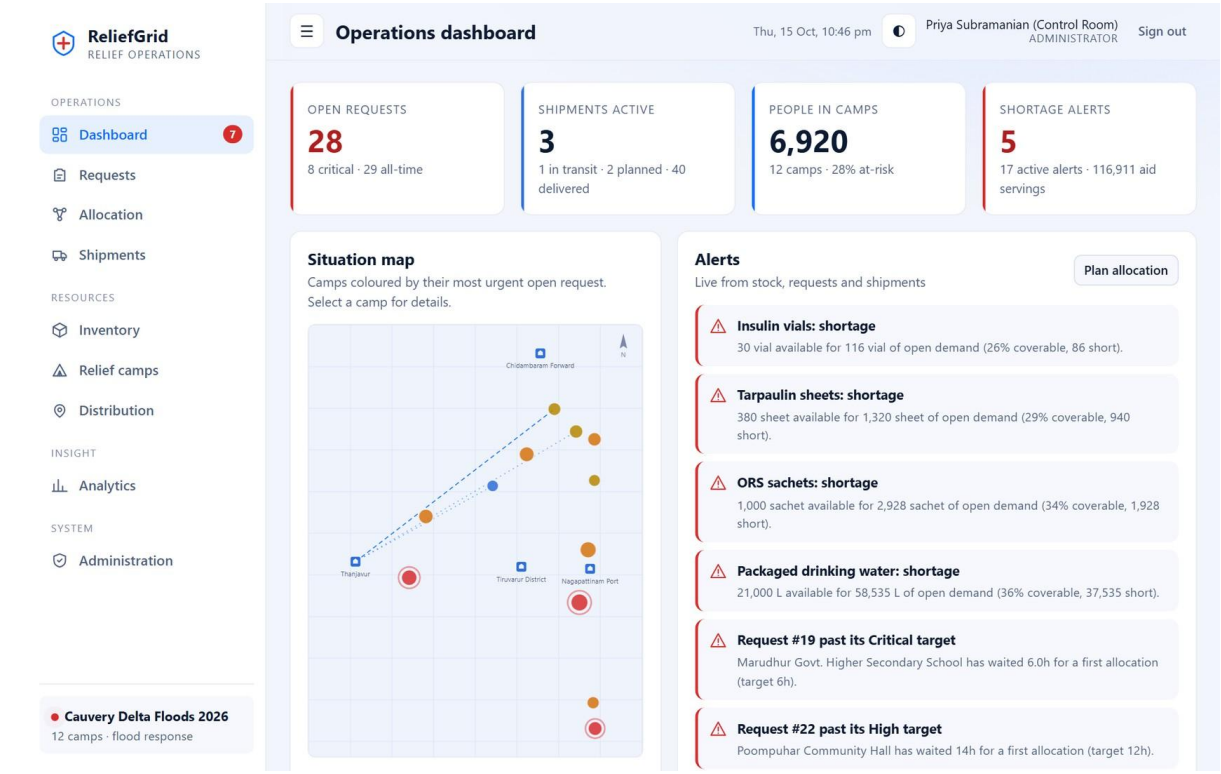


***Figure 9**: Dashboard: KPI cards, situation map (camps coloured by most urgent open request, shipment routes) and live alerts*

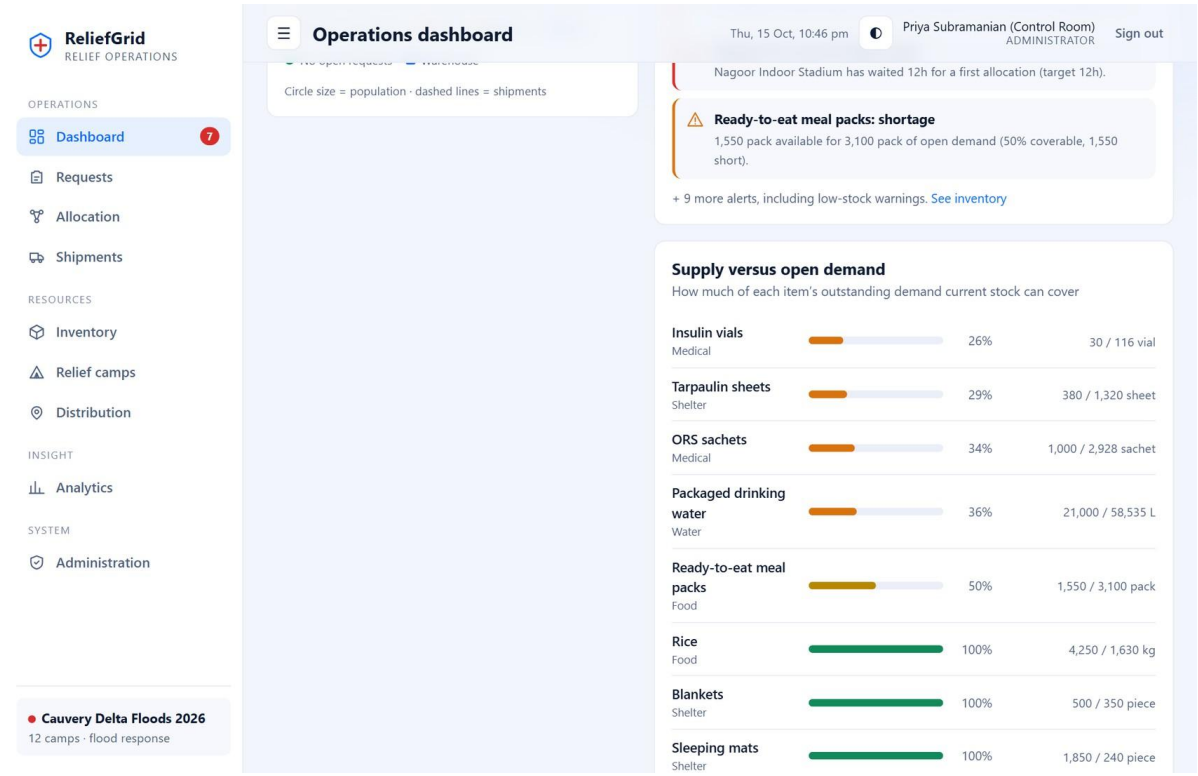


***Figure 10**: Dashboard (lower part): supply versus open demand, open requests by urgency and recent activity from the audit trail*

A.3 Supply requests

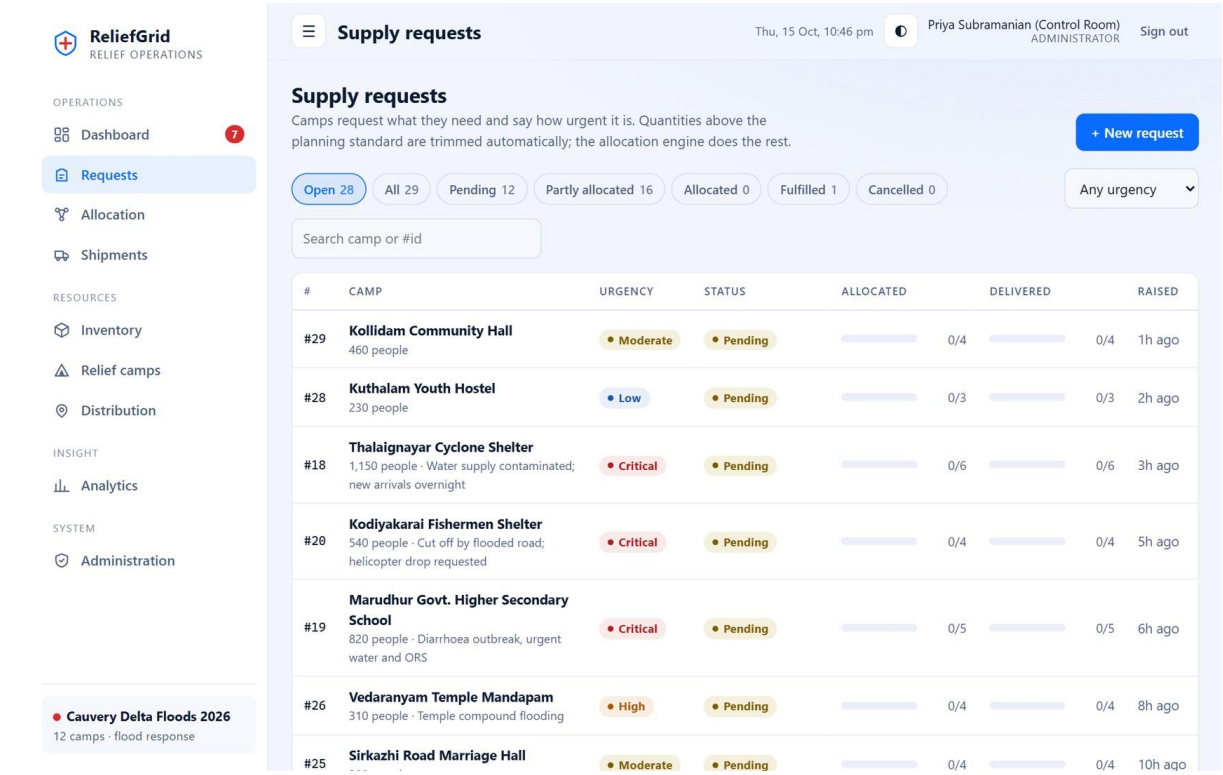


***Figure 11**: Requests list with status filters, urgency filter and search*

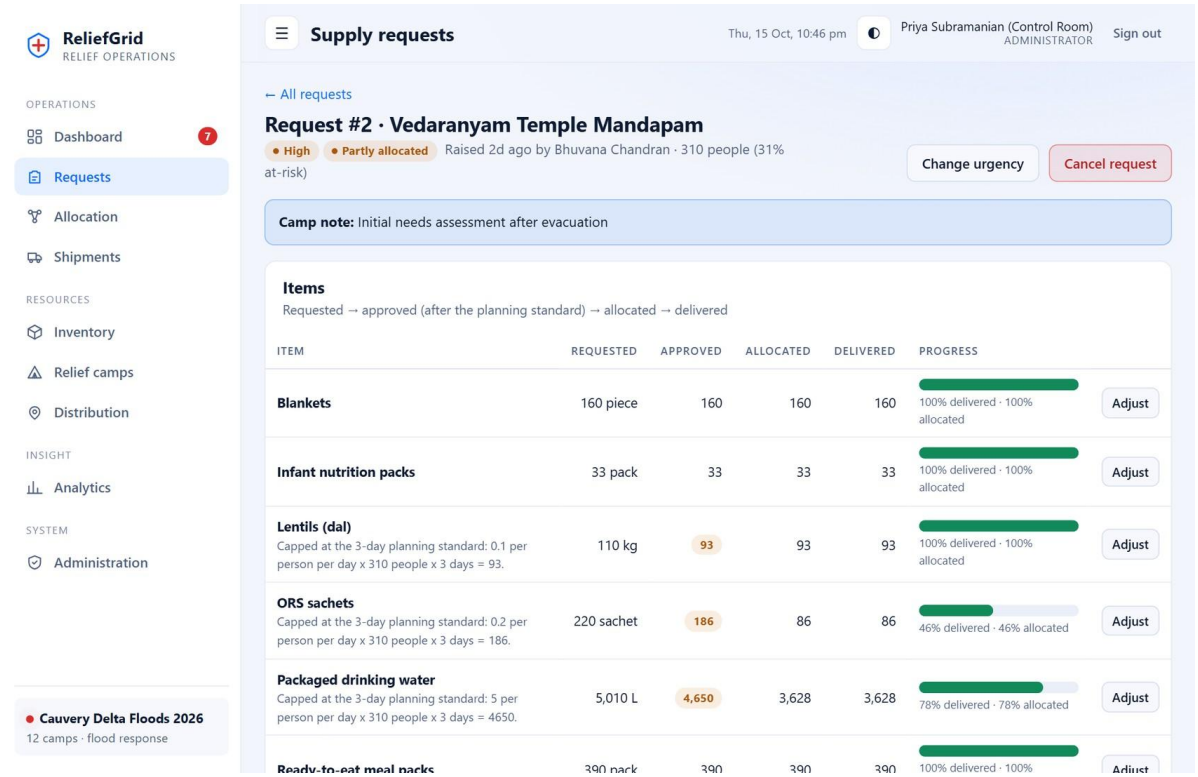


***Figure 12**: Request page: requested → approved → allocated → delivered, and the allocation decisions that were made for this request in plain language*

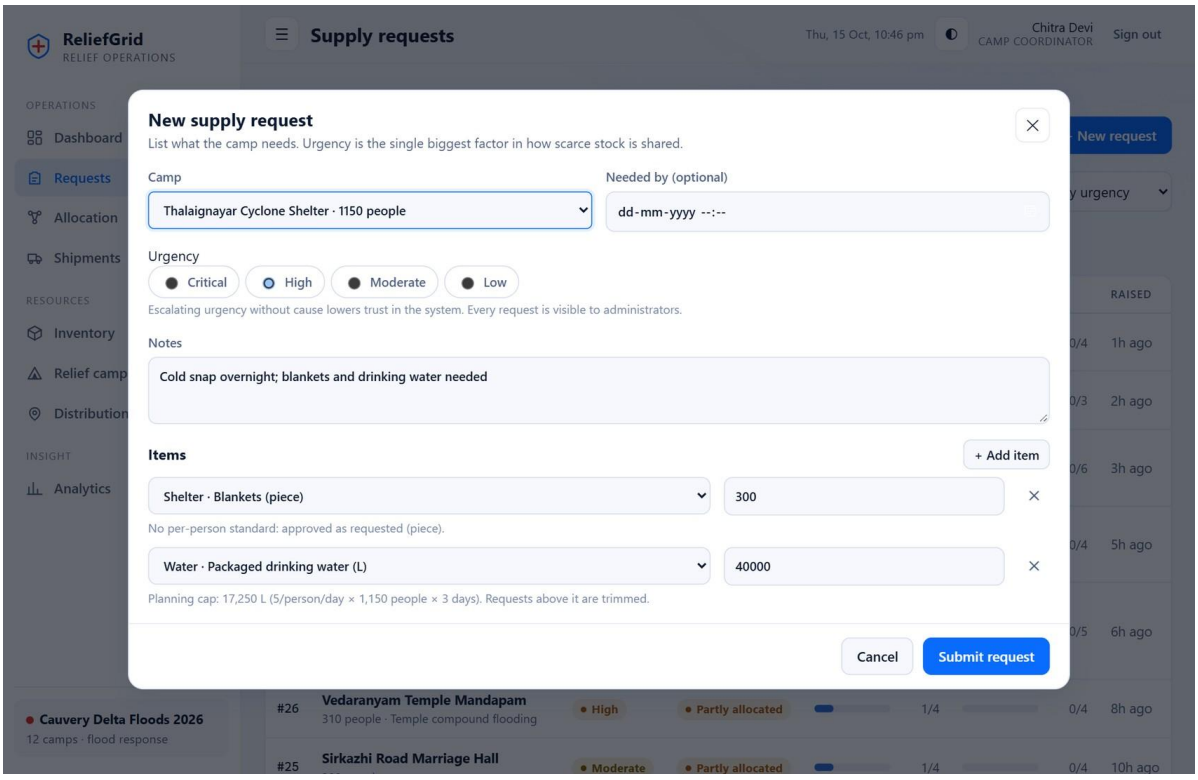


***Figure 13**: New-request dialog (camp coordinator): urgency, items, and the live planning-cap hint per item*

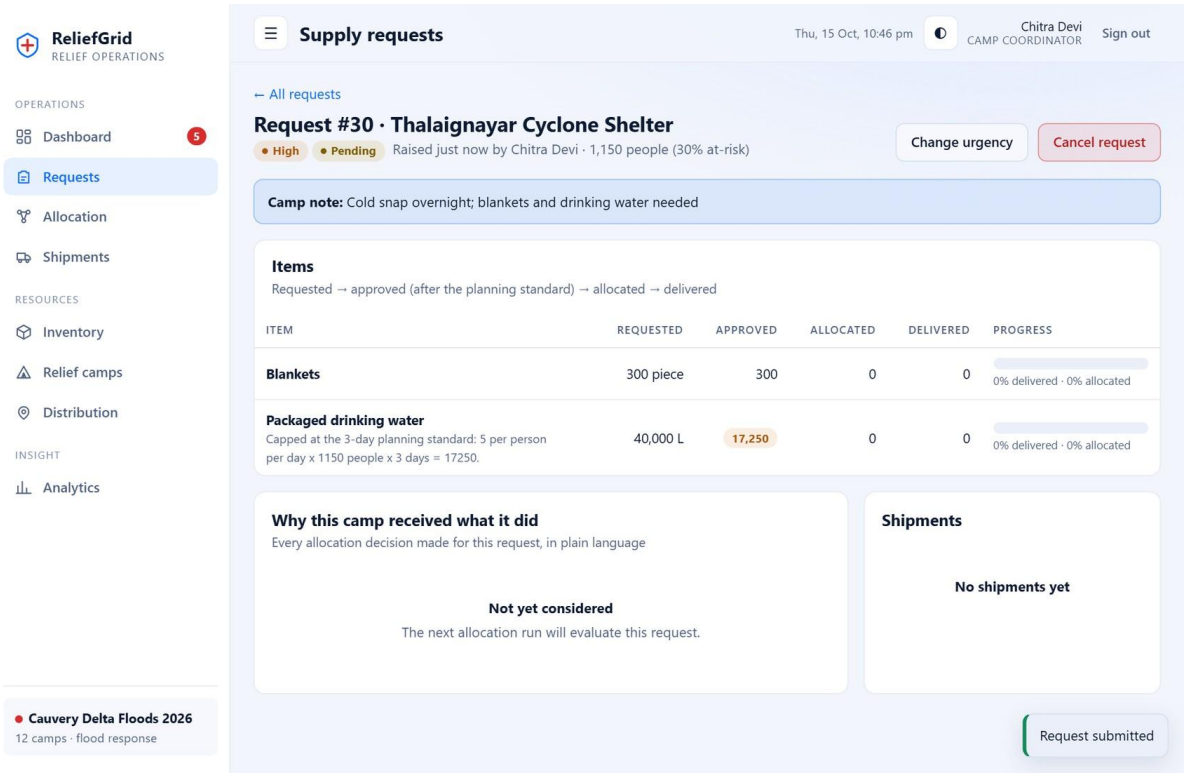


***Figure 14**: Request page after submission: 40,000 L of water requested, 17,250 L approved with the reason recorded*

A.4 Allocation engine

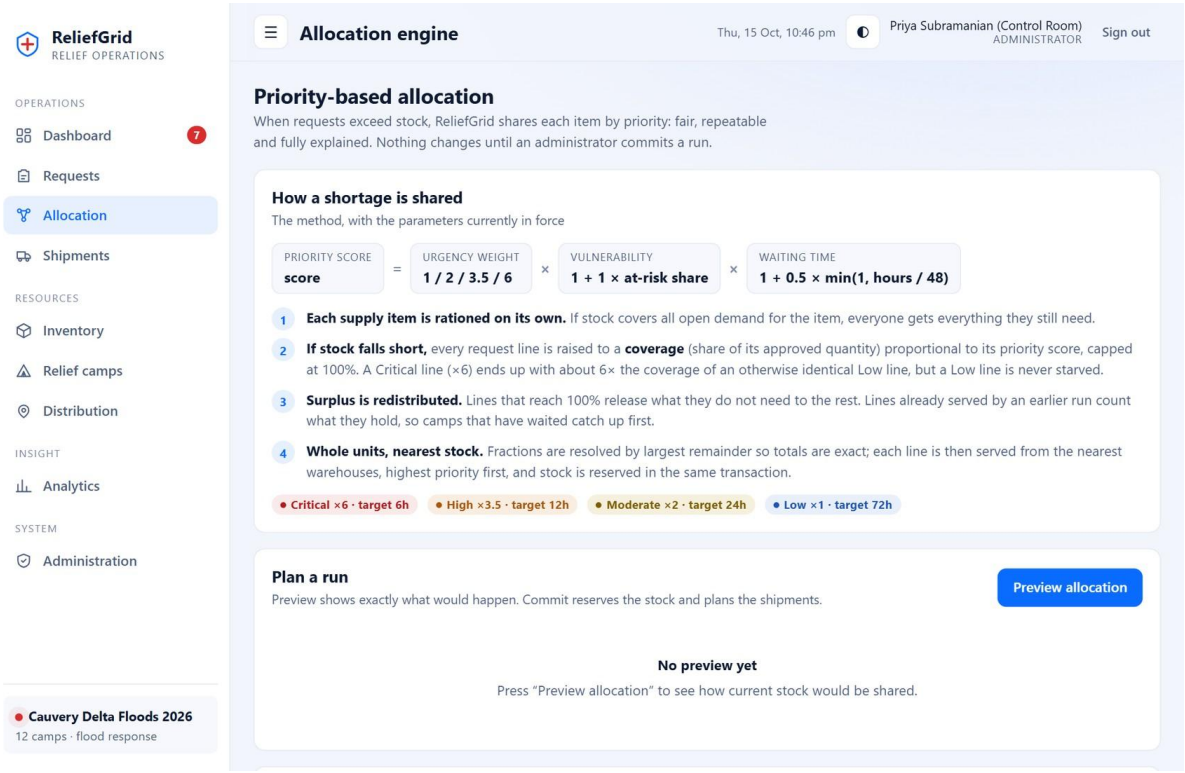


***Figure 15**: Allocation page: the priority formula and the four steps used to share a shortage*

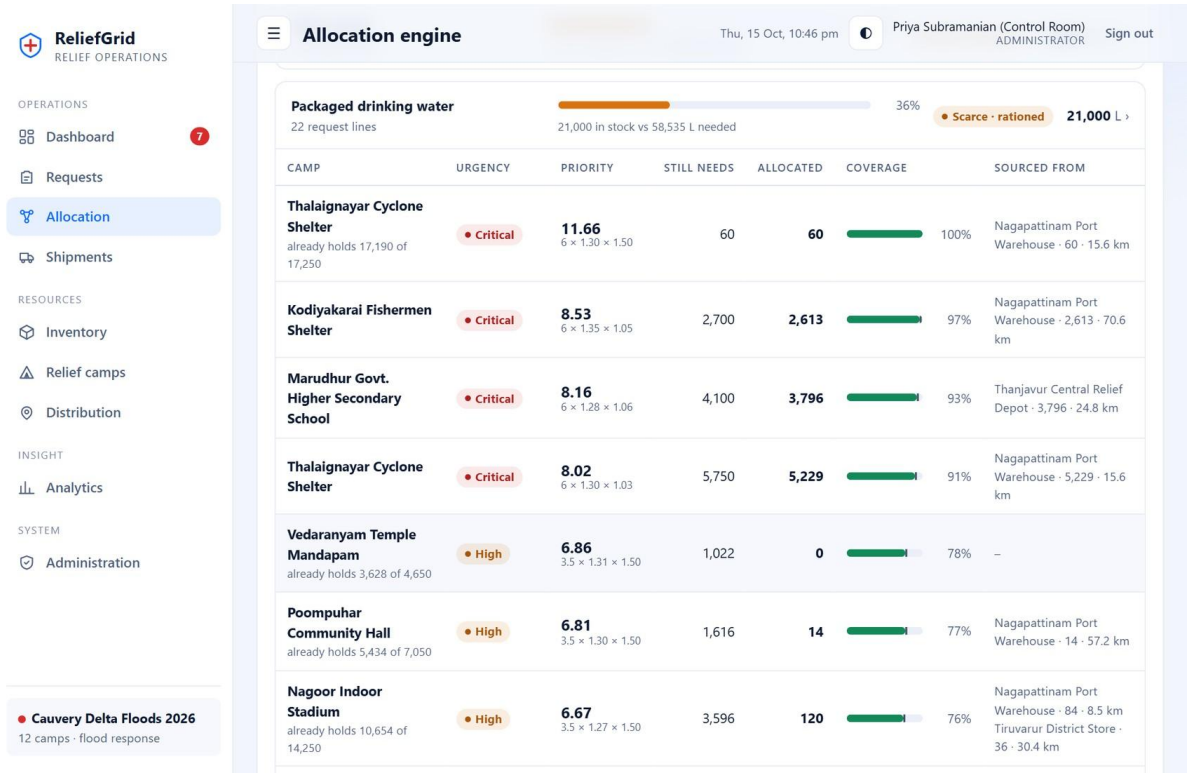


***Figure 16**: Allocation preview for drinking water: priority-score breakdown, still-needs, allocated quantity, coverage bar and source warehouses*

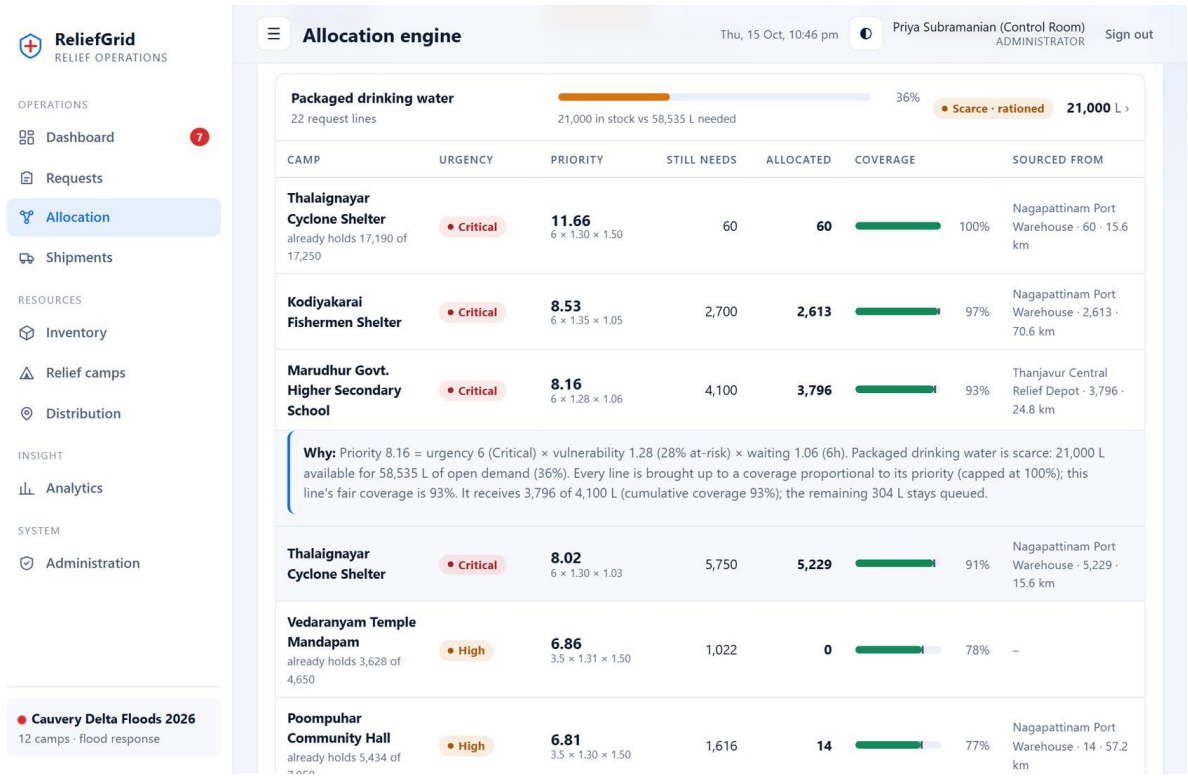


***Figure 17**: A line expanded to show its plain-language explanation*

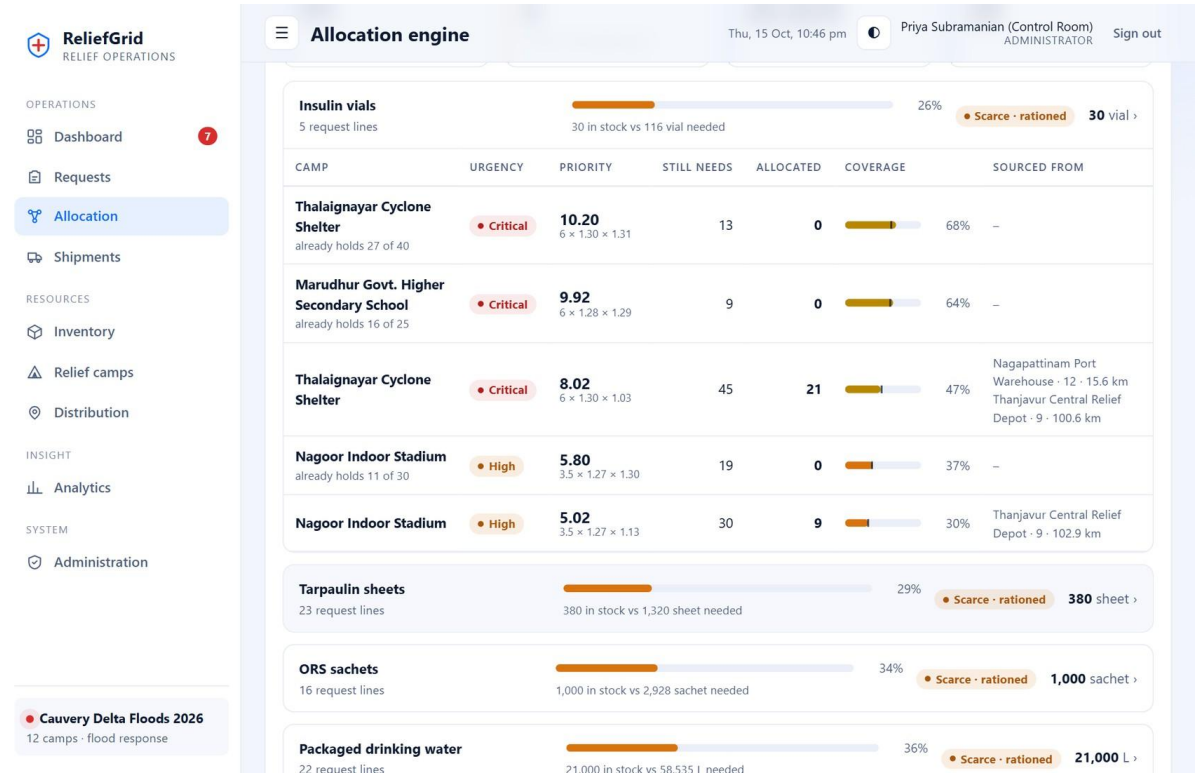


***Figure 18**: Insulin: lines that already hold more than their fair level wait ("already holds 27 of 40", 0 allocated)*

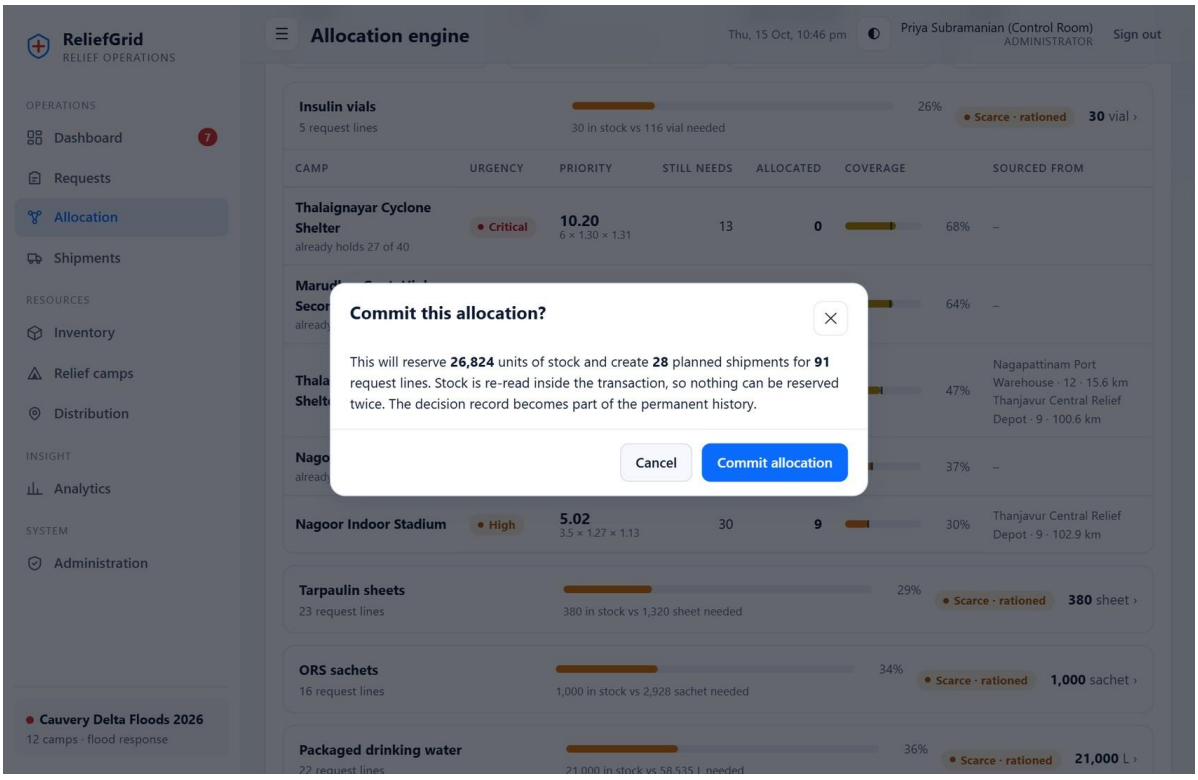


***Figure 19**: Confirmation dialog before committing a run*

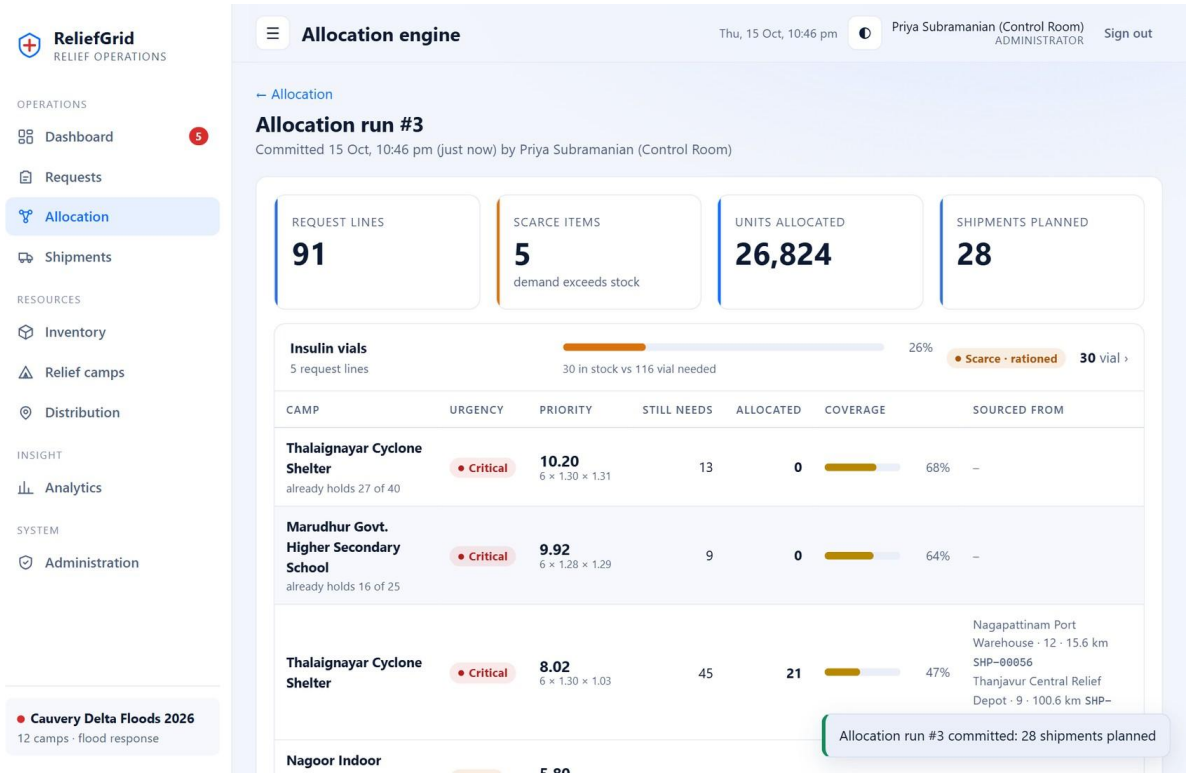


***Figure 20**: Committed run: every decision, its score and the warehouse it is sourced from*

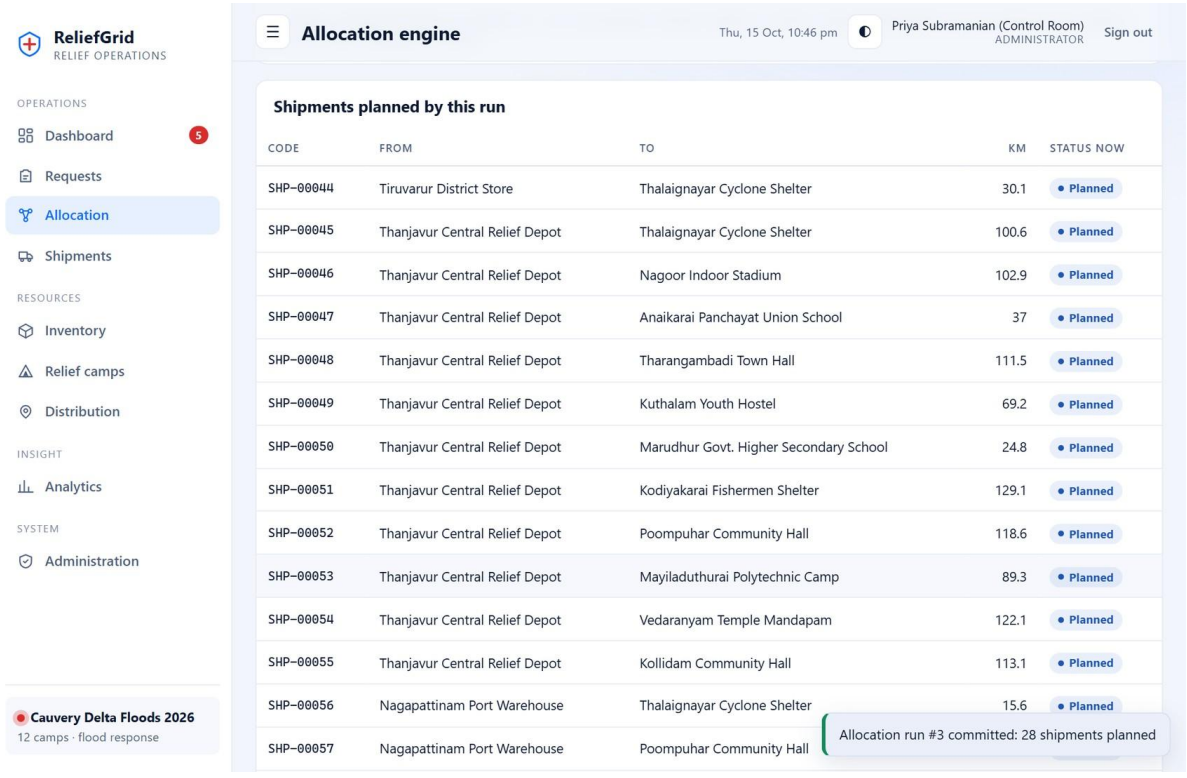


***Figure 21**: Shipments planned by the committed run*

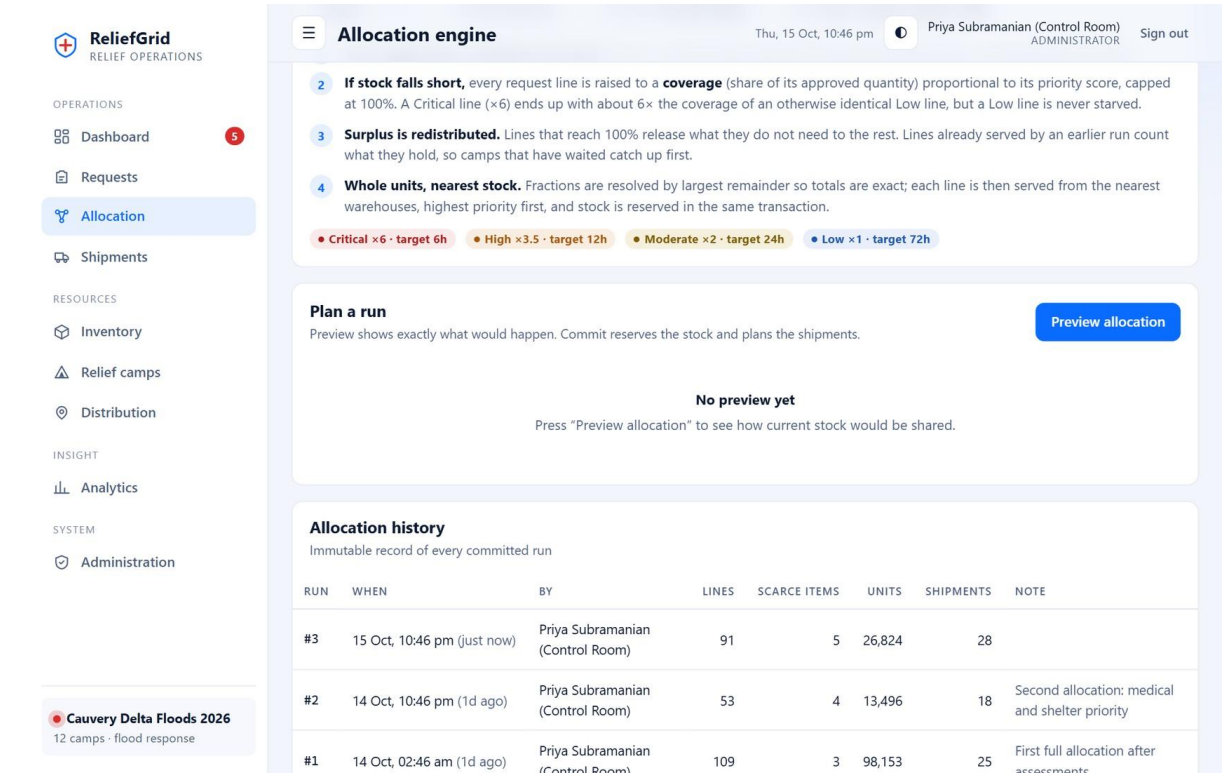


***Figure 22**: Allocation history: immutable record of all committed runs*

A.5 Shipments and deliveries

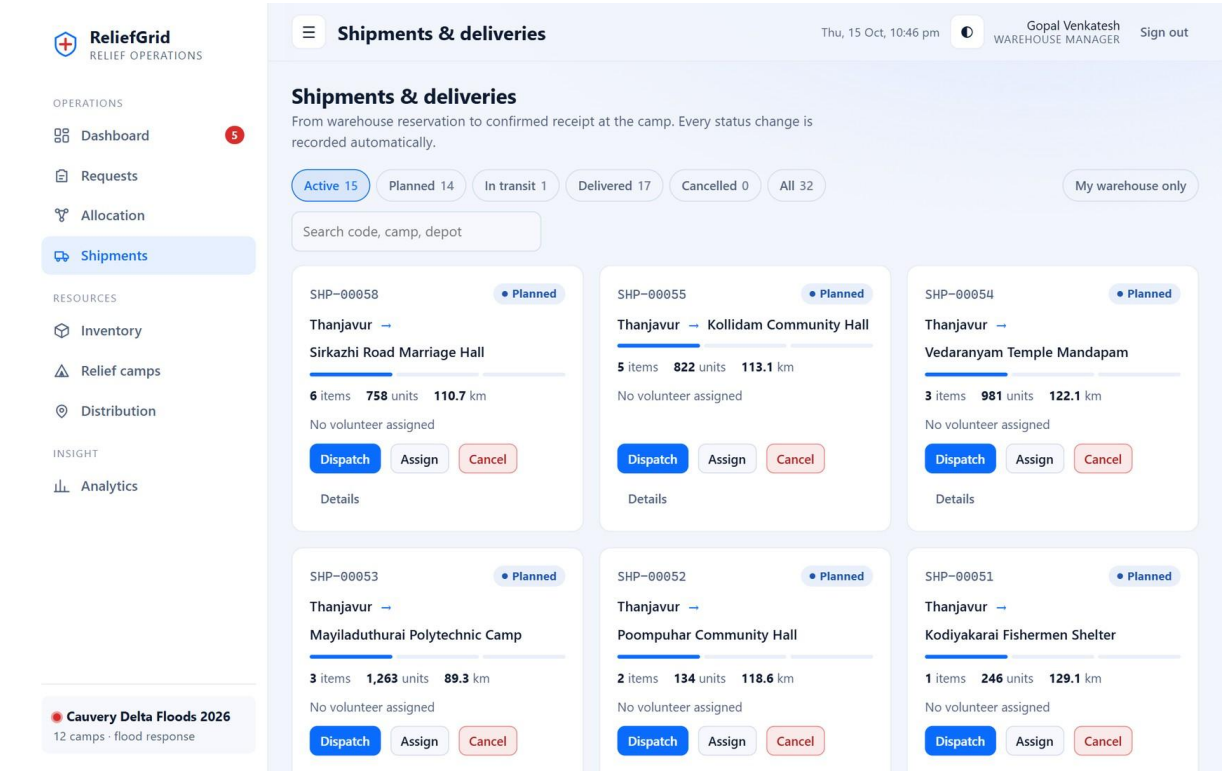


***Figure 23**: Shipments board as seen by a warehouse manager (own warehouse only)*

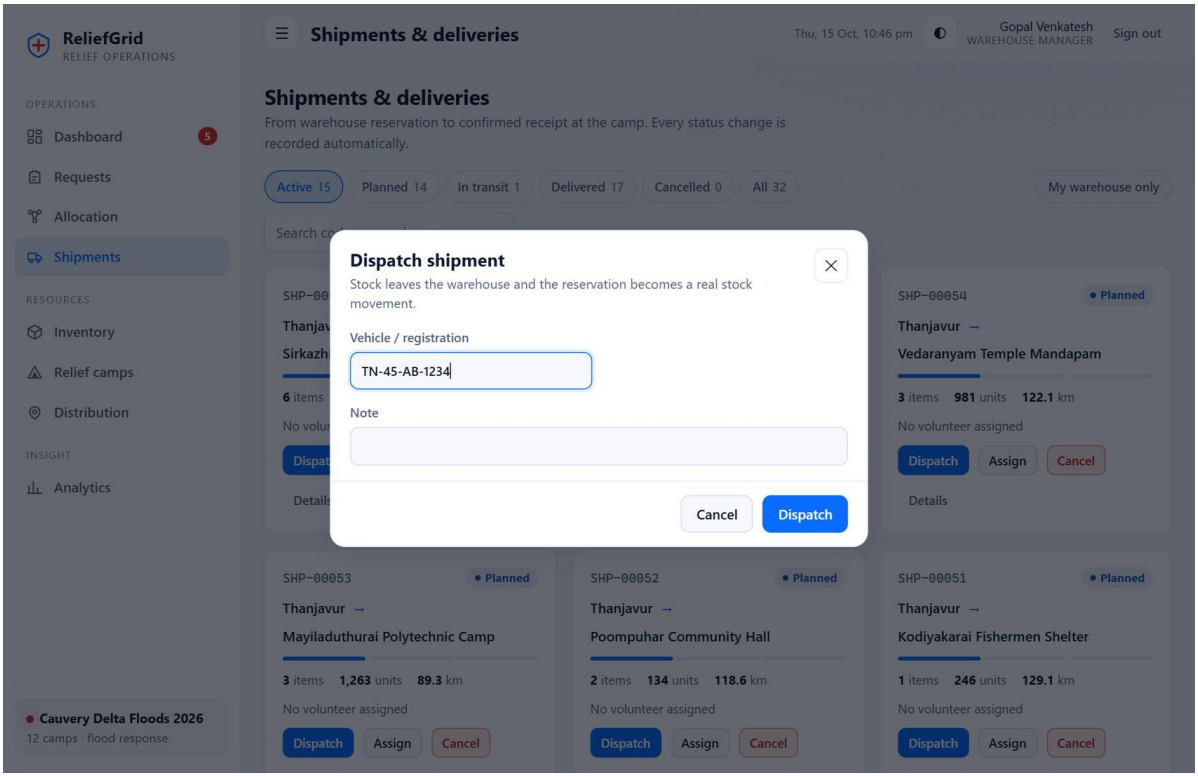


***Figure 24**: Dispatch dialog*

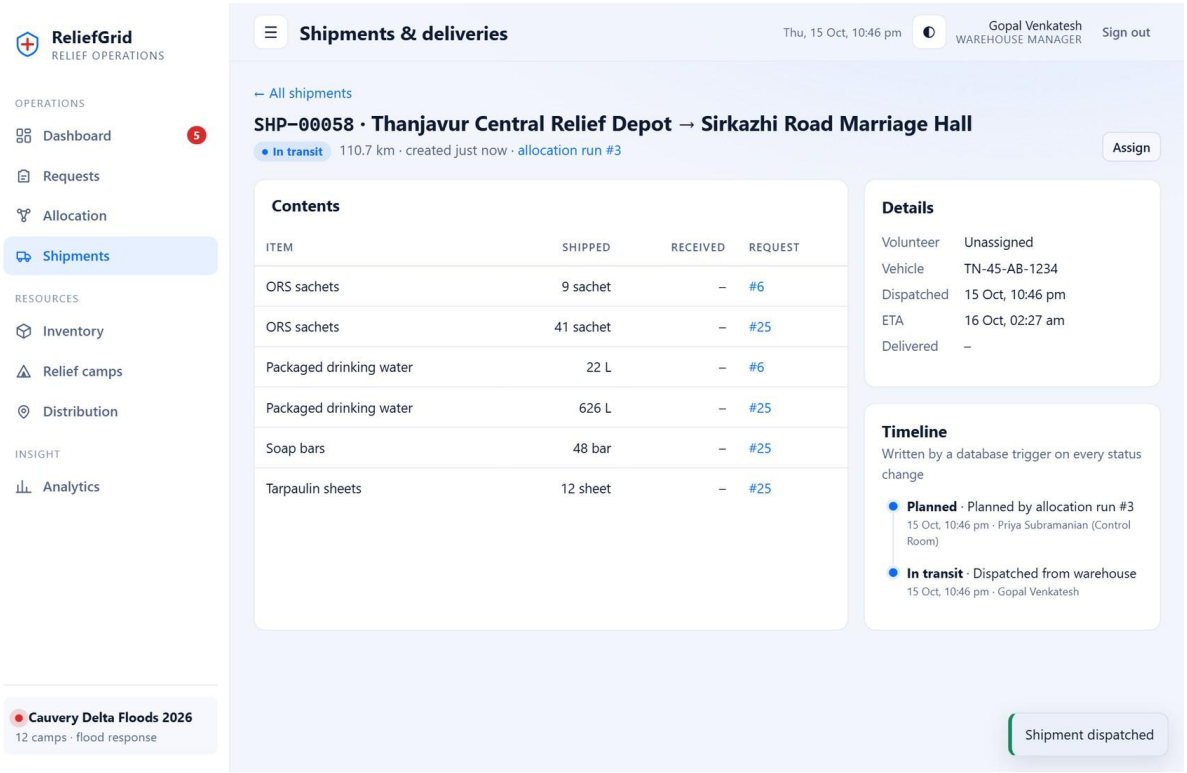


***Figure 25**: Shipment detail with the timeline written by a database trigger*

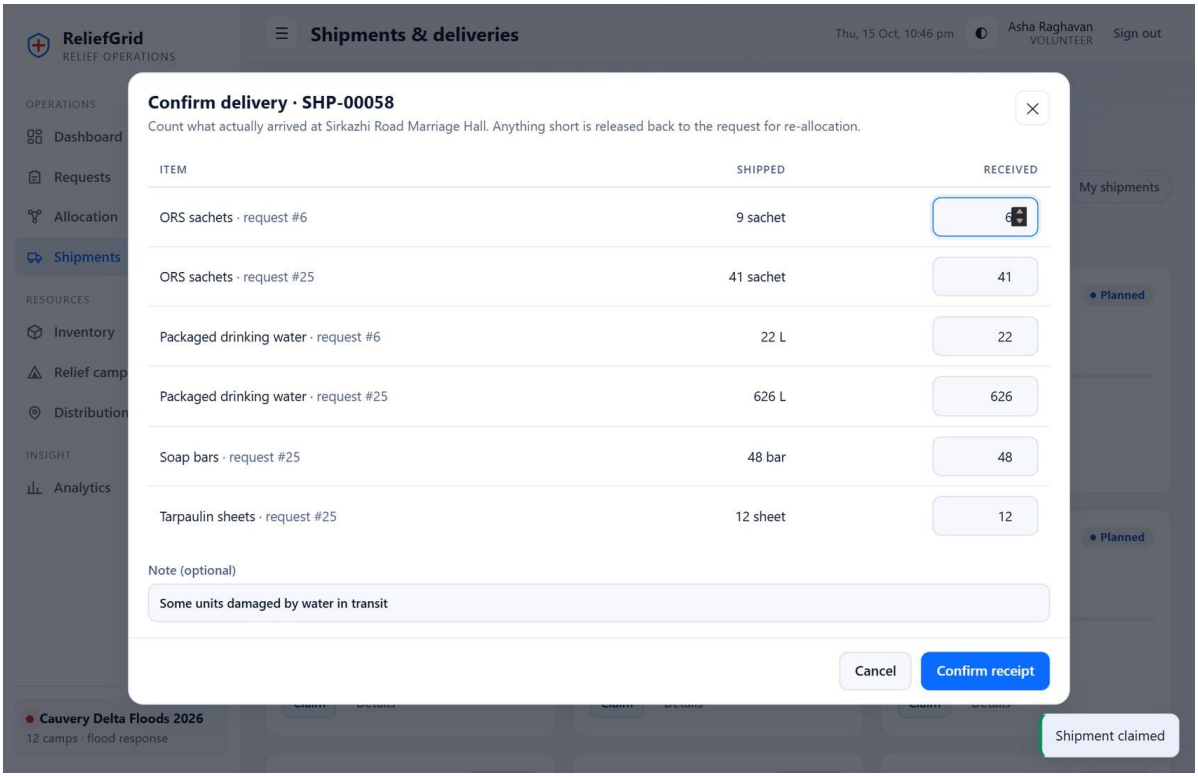


***Figure 26**: Confirm-delivery dialog (volunteer): per-line received quantities; a smaller number records a shortfall*

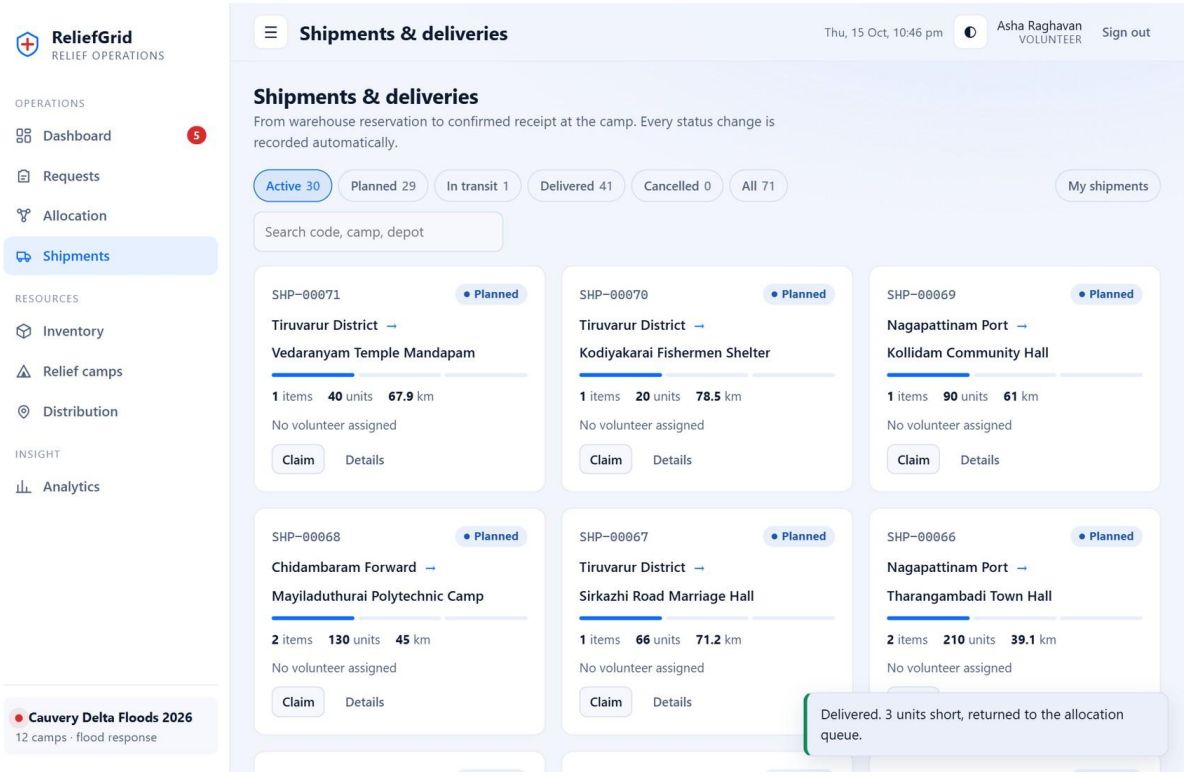


***Figure 27**: Volunteer view after confirming a delivery*

A.6 Inventory

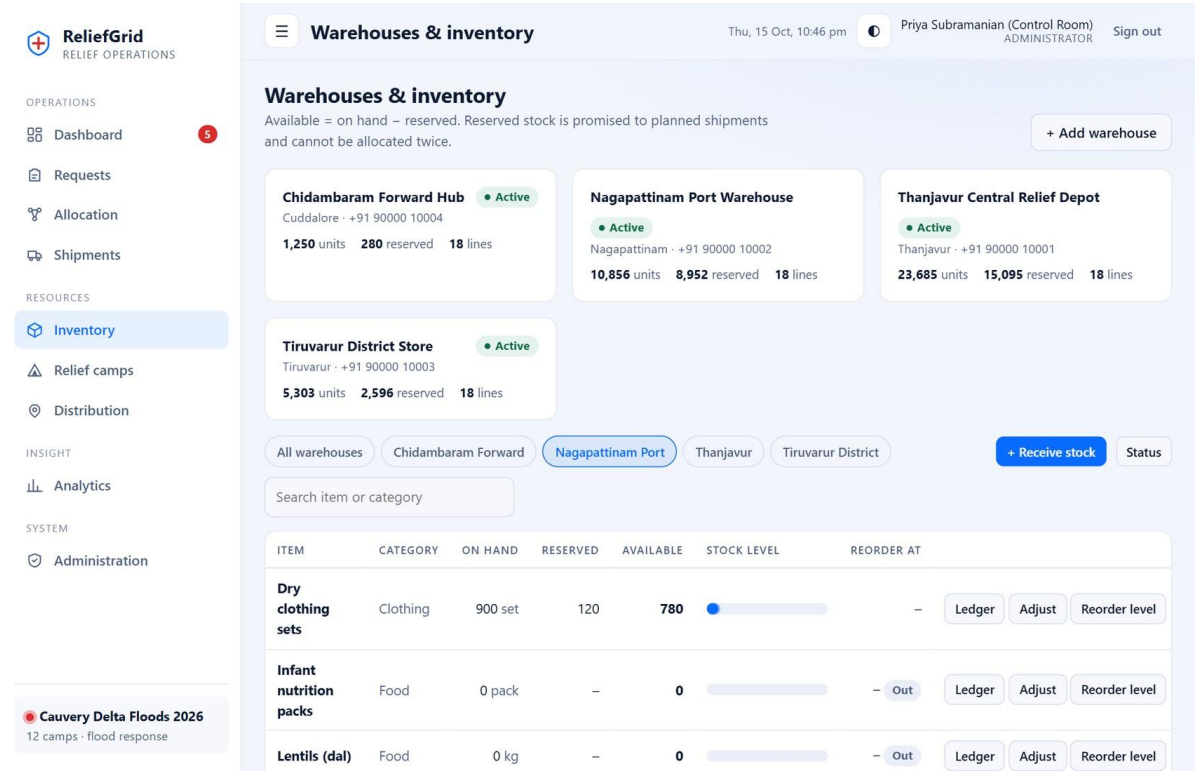


***Figure 28**: Warehouse inventory: on hand, reserved, available, reorder level*

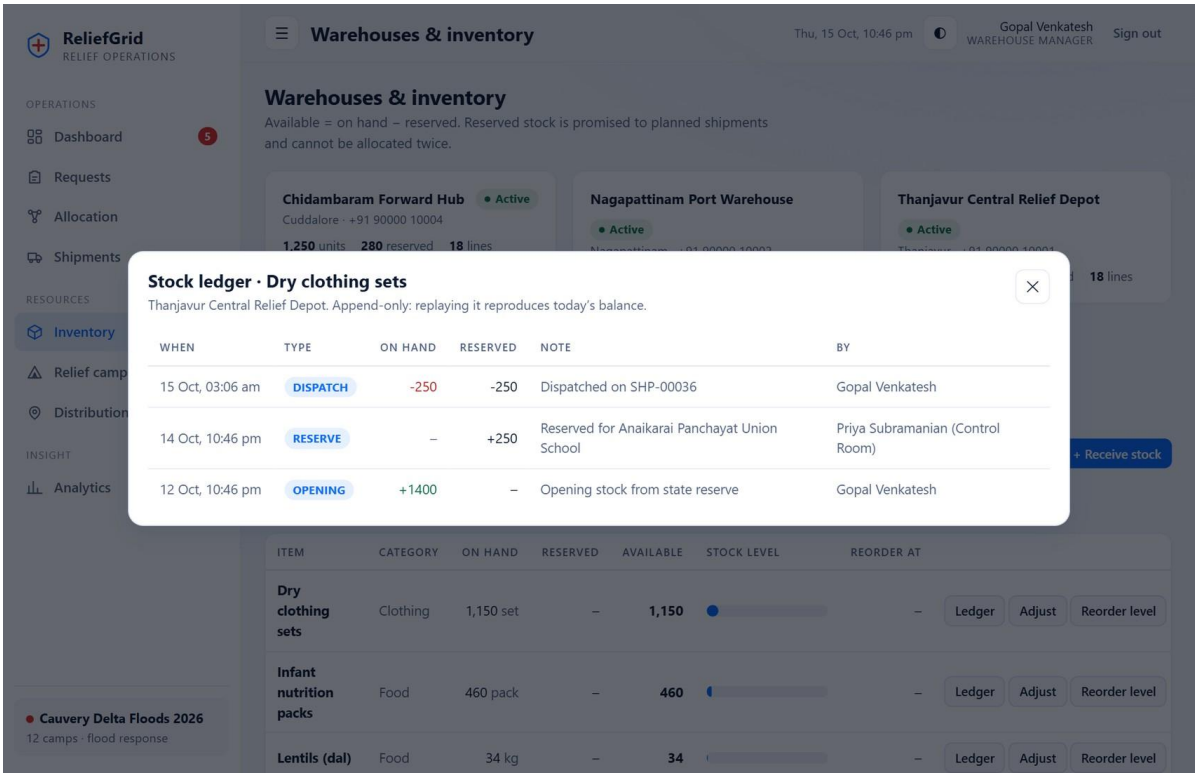


***Figure 29**: Stock ledger viewer: append-only history of one stock line*

A.7 Camps and distribution

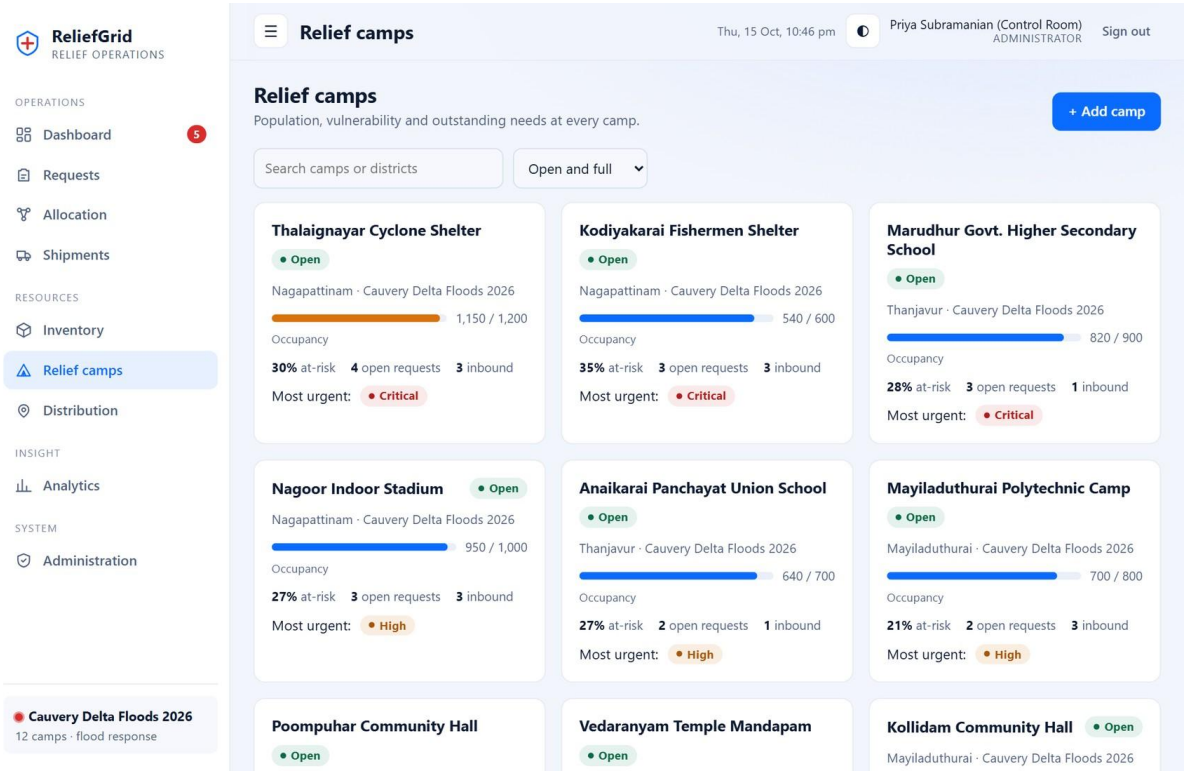


***Figure 30**: Relief camps sorted by urgency*

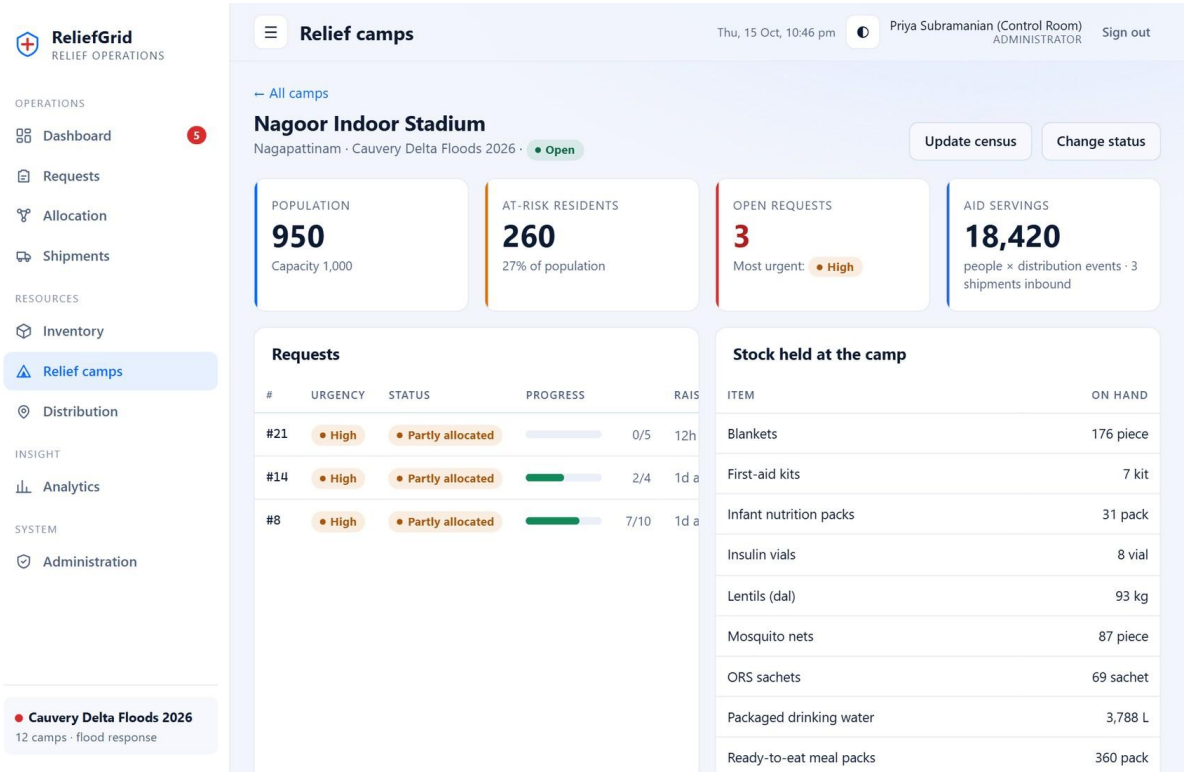


***Figure 31**: Camp detail: requests, stock held, shipments and distributions*

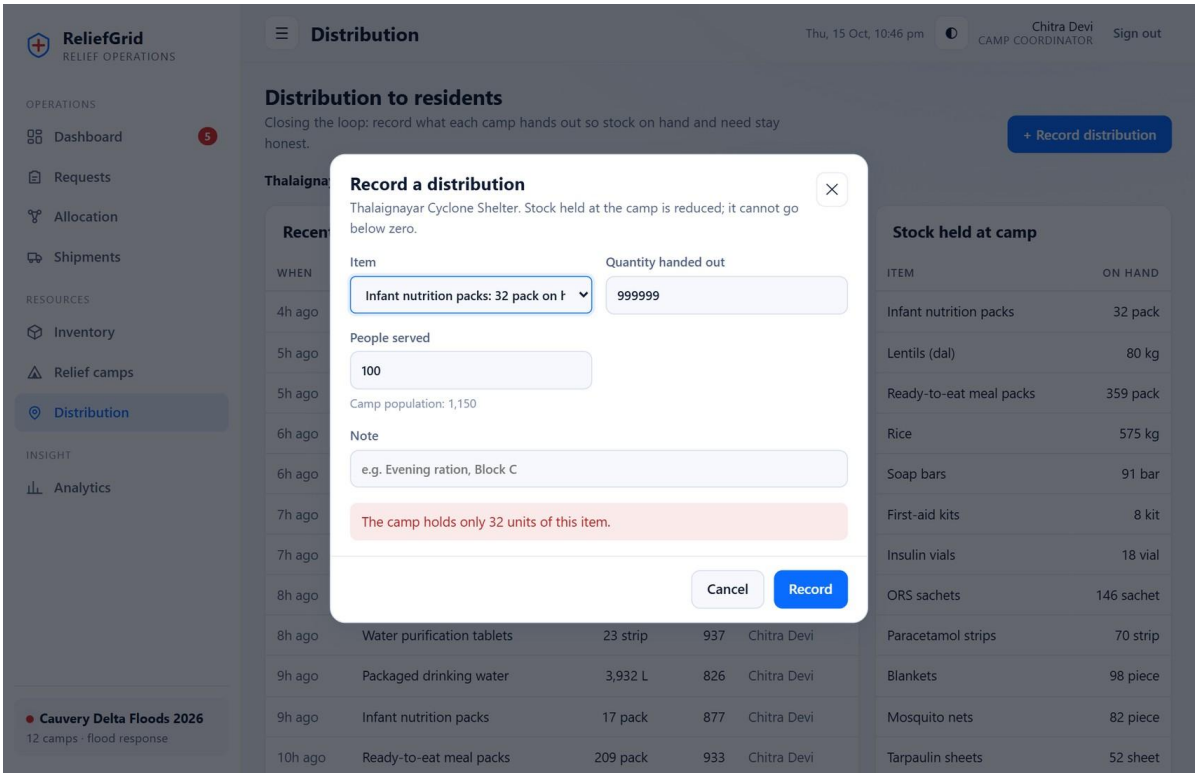


***Figure 32**: Distribution dialog refusing a quantity larger than the camp holds*

A.8 Analytics

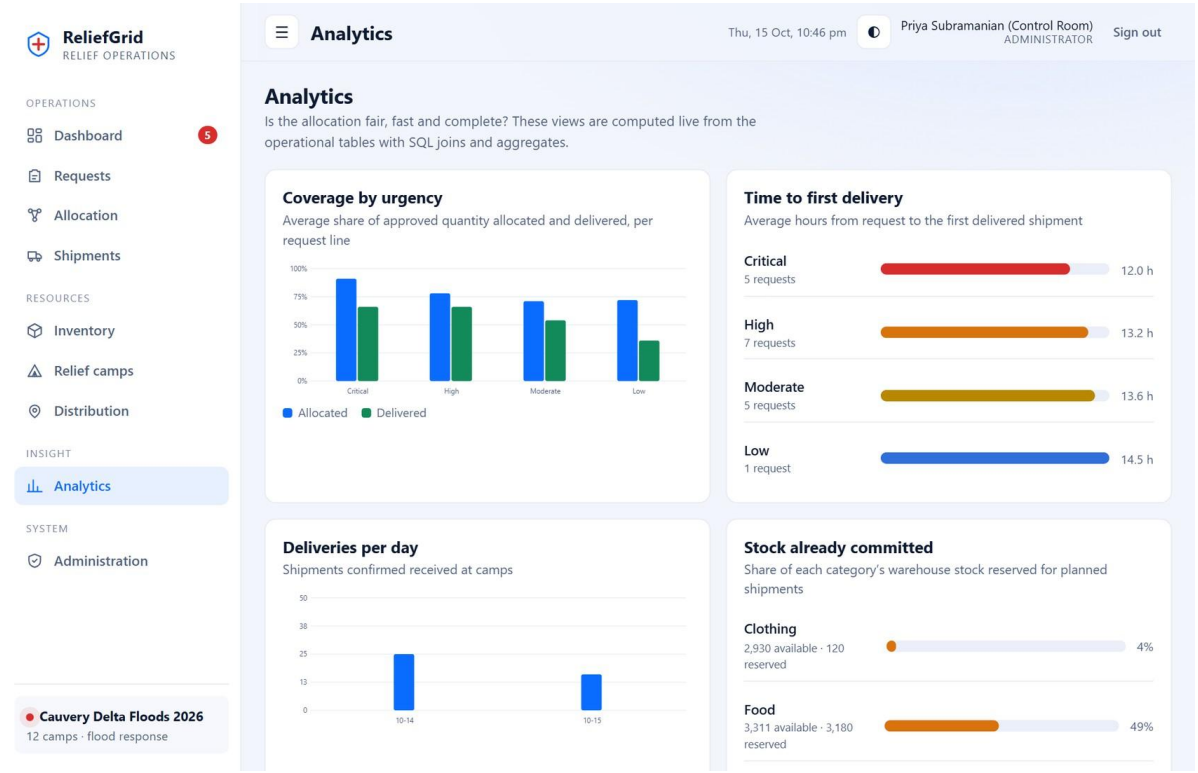


***Figure 33**: Analytics: coverage by urgency, time to first delivery, deliveries per day and committed stock*

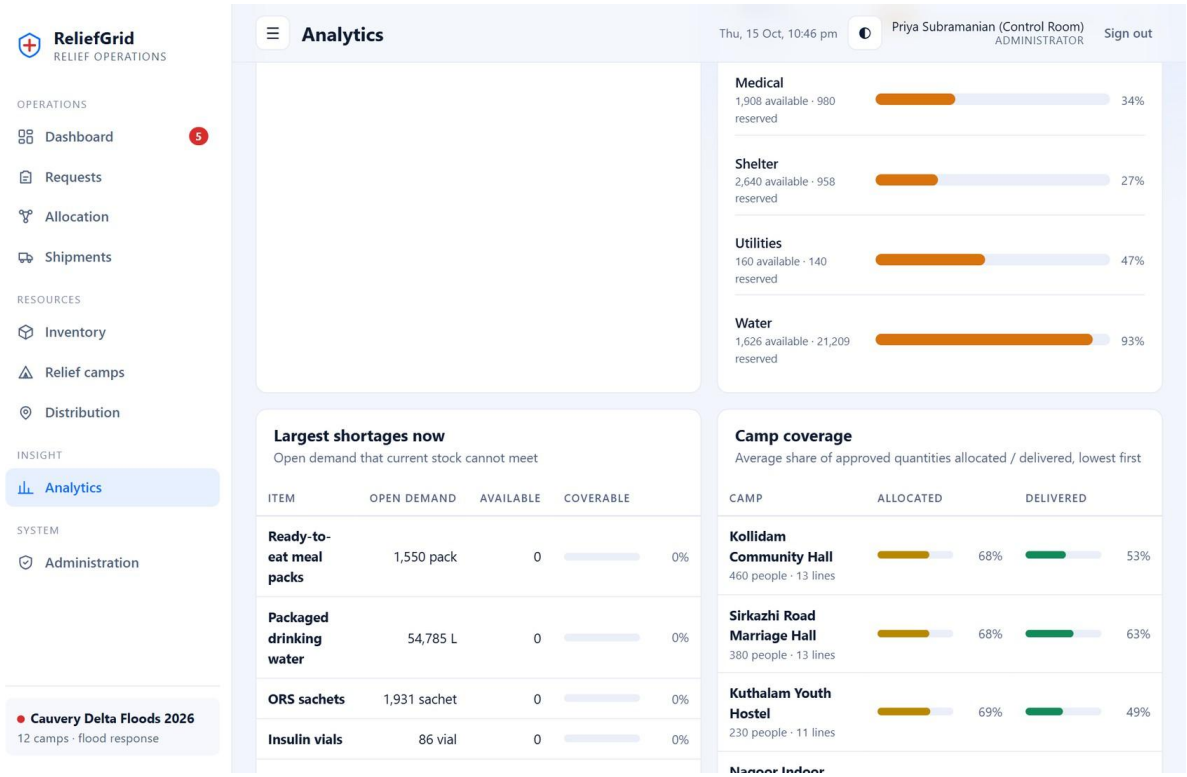


***Figure 34**: Analytics (lower part): largest shortages, camp coverage, allocation runs and losses in transit*

A.9 Administration

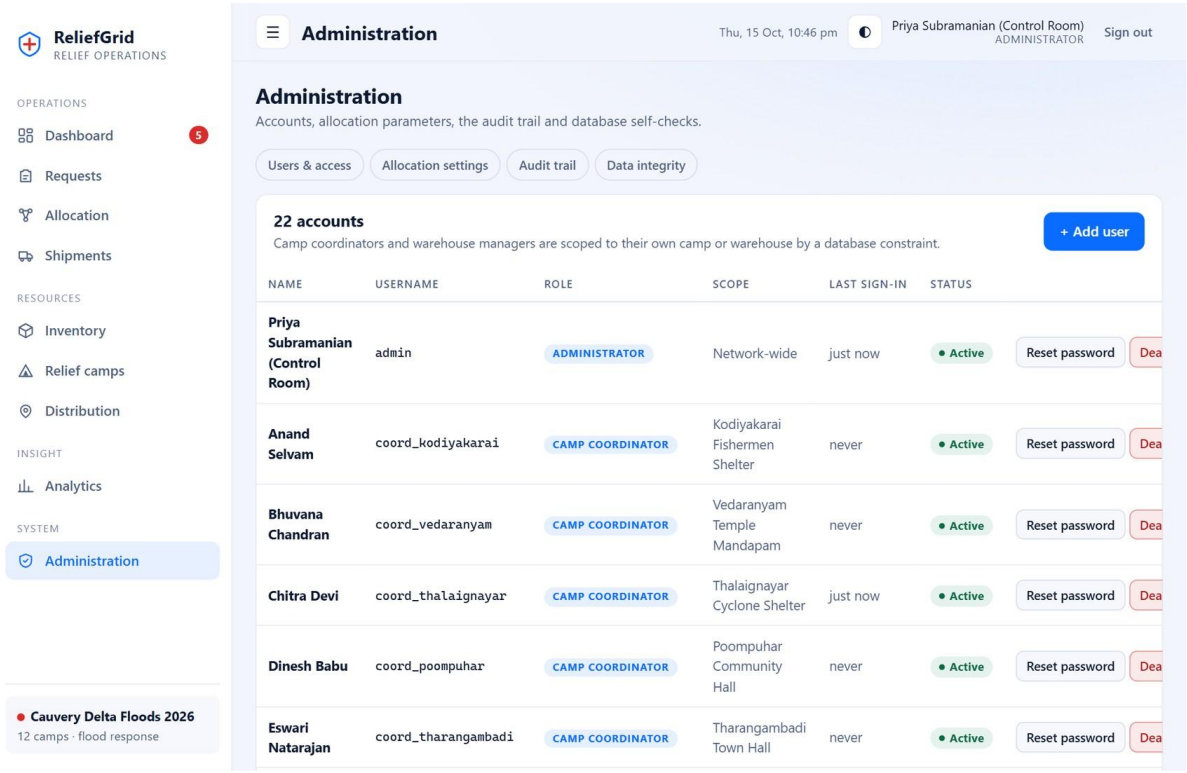


***Figure 35**: User administration with role and scope*

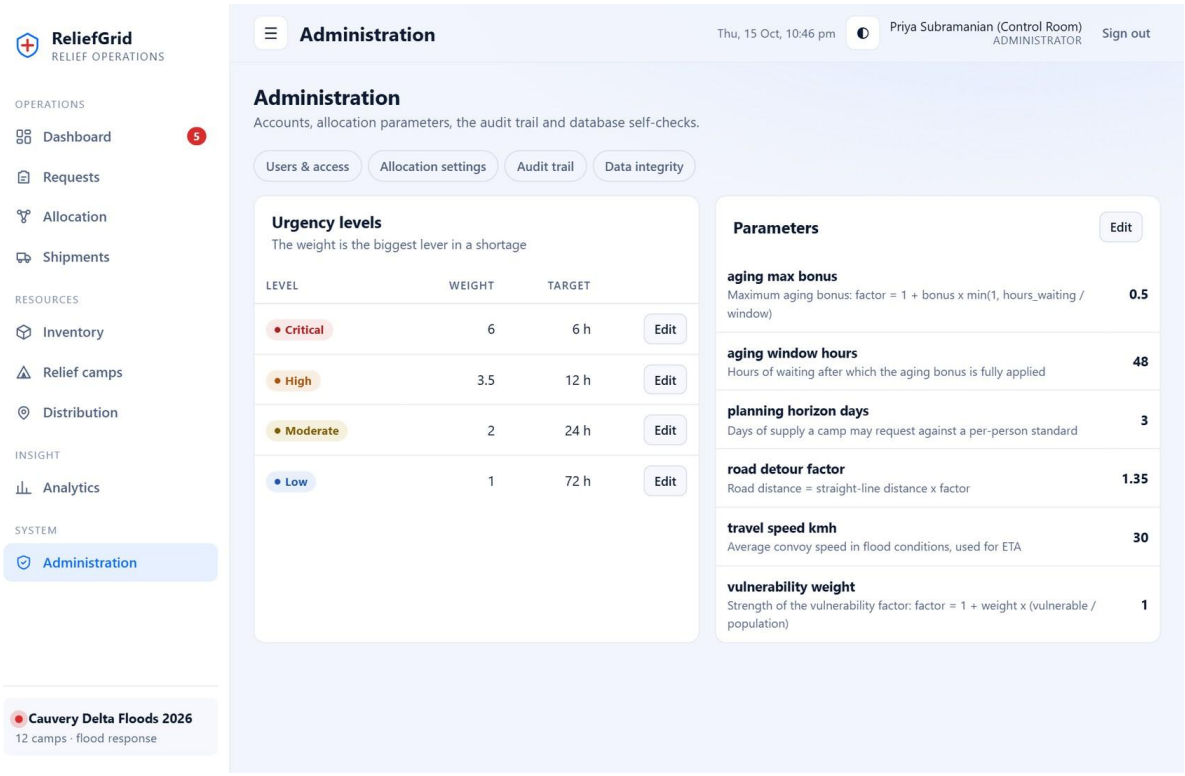


***Figure 36**: Allocation settings: urgency weights and parameters (editable by administrators)*

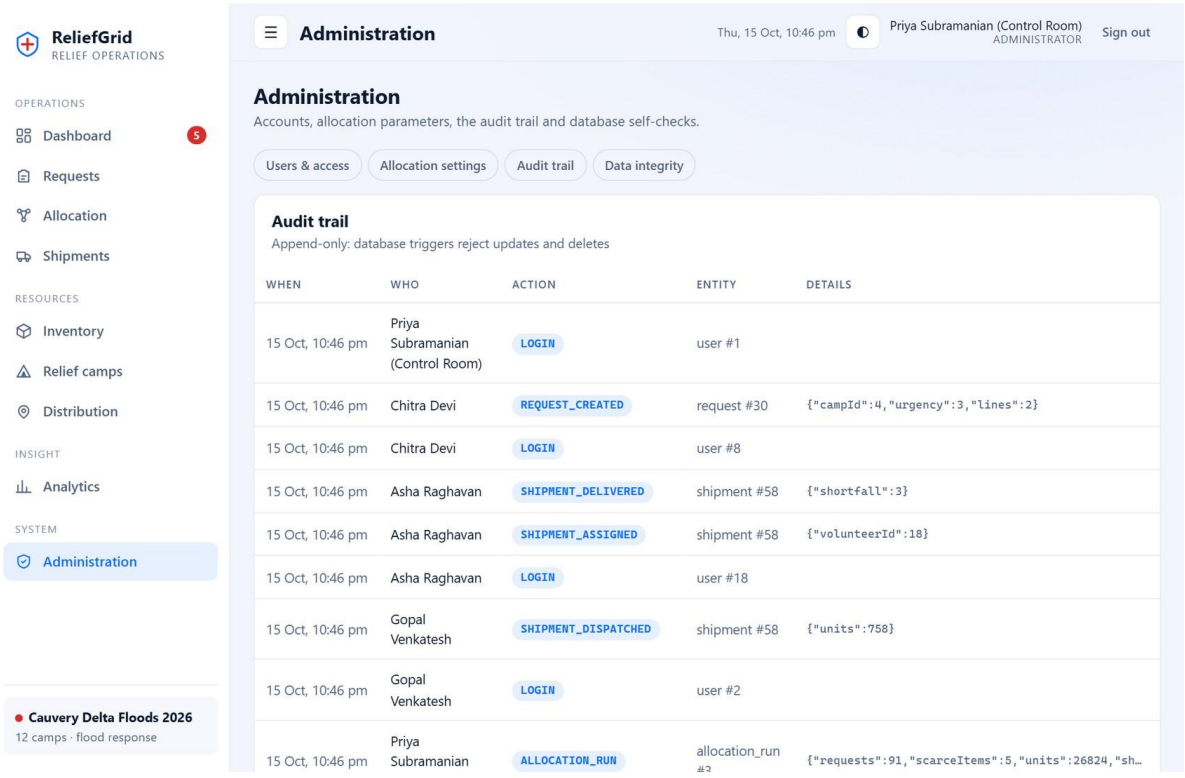


***Figure 37**: Append-only audit trail*

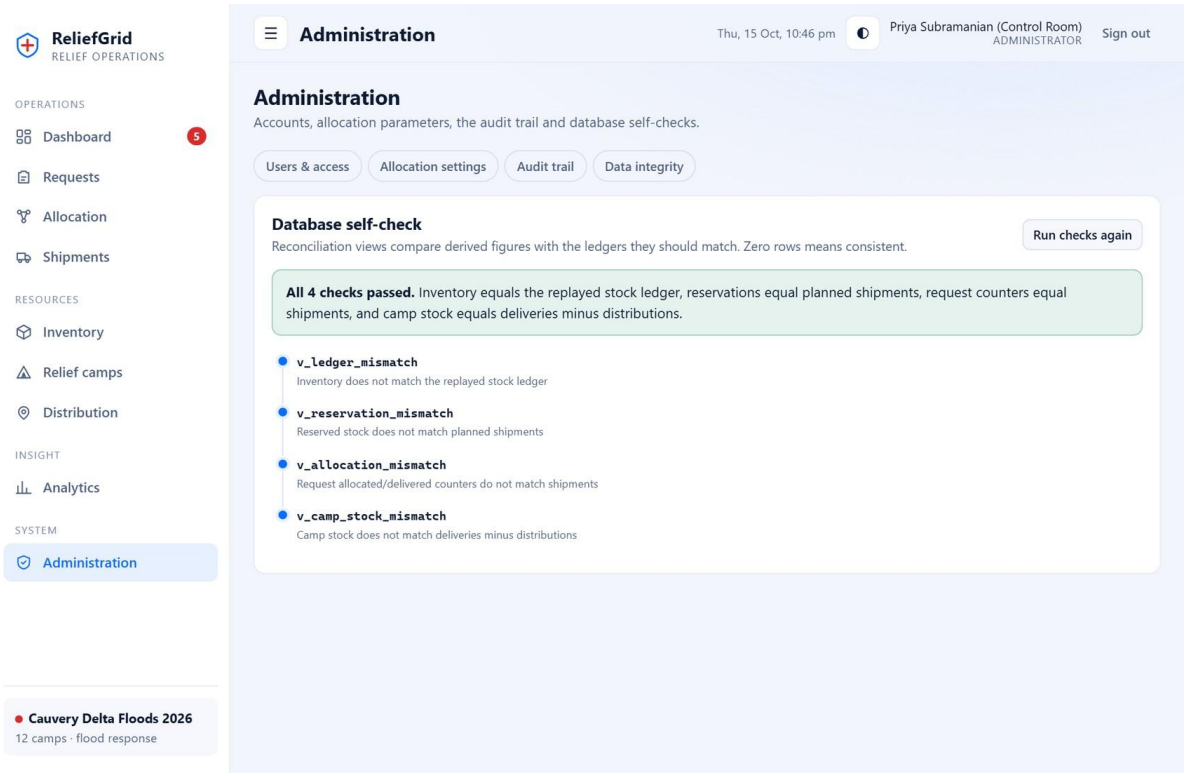
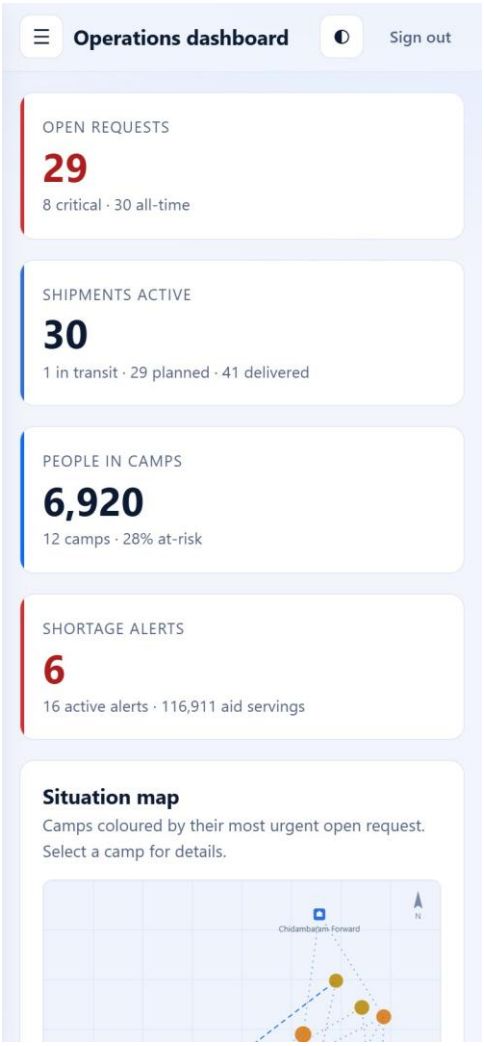
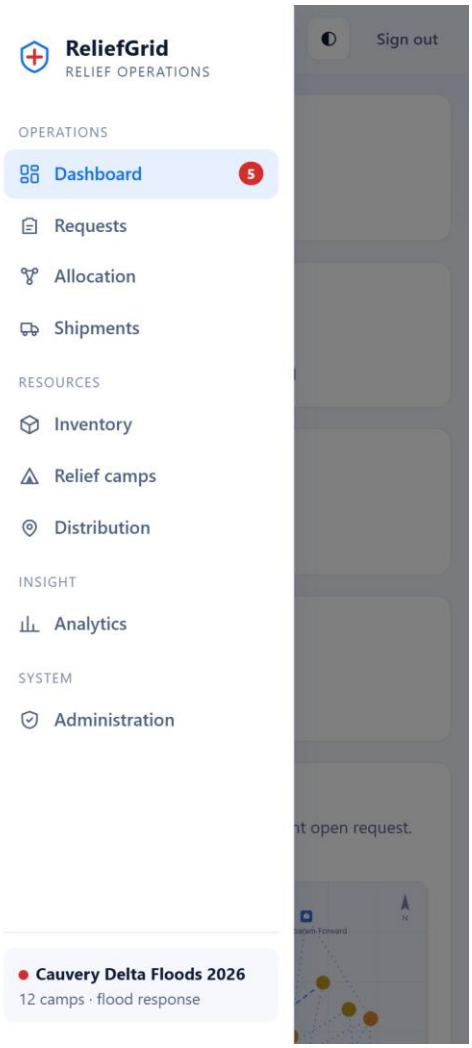


***Figure 38**: Data-integrity self-check: all four reconciliation views report no problems*

A.10 Mobile layout



Dashboard on a 390 px wide screen



Off-canvas navigation menu

APPENDIX B — REPRODUCING THE RESULTS

All verified results in this report can be regenerated with the commands below, run from the project directory (`reliefgrid/`) on Node.js 22.13 or newer. No database server is required.

```
npm install                 # installs the only runtime dependency (express)
npm run seed                # creates data/reliefgrid.db with the fictional demo incident
npm start                   # serves the application at http://localhost:3000
npm test                    # 71 automated tests (about 4-5 seconds, in-memory databases)
npm run scenarios           # allocation scenarios and baselines -> docs/RESULTS.md
npm run queries             # prints the showcase SQL queries from db/queries.sql
```

***Table 40**: Project size (non-blank files in each area)*

Area	Files	Lines
Server (JavaScript)	19	2,241
Frontend (JS, CSS, HTML)	16	1,984
Tests	6	1,025
SQL (4 migrations + showcase queries)	5	753
Scripts	3	317

Line counts include blank lines and comments. Demo accounts (local builds only) all use the password documented in the README; sign-in shortcuts are disabled when `NODE_ENV=production`.

APPENDIX C — AUTOMATED TEST RESULTS

Output of `npm test` (spec reporter), reproduced from the verification run: 71 passed, 0 failed, total duration 4.3 s. Per-test durations are in milliseconds.

File	Test	Result	ms
api.test.js	authentication: login, bad password, missing token, logout, deactivation	Passed	497.0
api.test.js	authentication: repeated failures are rate limited	Passed	562.1
api.test.js	role-based access control	Passed	278.3
api.test.js	request validation and standard-based approval	Passed	288.1
api.test.js	preview changes nothing; run is atomic, explainable and reserves stock	Passed	247.4
api.test.js	over-allocation is impossible: a second run only sees what is left	Passed	249.0
api.test.js	concurrent allocation runs never double-book stock	Passed	264.3
api.test.js	a failure part-way through a run rolls everything back	Passed	248.6
api.test.js	full lifecycle: request -> allocation -> dispatch -> partial delivery -> re-allocation -> distribution	Passed	269.2
api.test.js	cancelling a planned shipment releases its reservation and re-queues the demand	Passed	251.6
api.test.js	inventory: receipts, adjustments and the reserved-stock floor	Passed	258.3
api.test.js	dashboard, alerts, analytics and integrity endpoints respond with consistent data	Passed	244.1
api.test.js	input hardening: malformed JSON, unknown routes, oversized ids	Passed	279.5
database.test.js	migrations are idempotent and record checksums	Passed	8.3
database.test.js	foreign keys are enforced on every connection	Passed	3.1
database.test.js	CHECK constraints reject impossible stock and quantities	Passed	4.7
database.test.js	user scope must match role	Passed	3.5
database.test.js	usernames are unique case-insensitively	Passed	3.4
database.test.js	history tables are append-only	Passed	17.8
database.test.js	shipment state machine and automatic event log	Passed	5.0
database.test.js	receipts can only be recorded while a shipment is in transit	Passed	4.3
database.test.js	business rules in triggers: closed camps, distribution guard, inventory delete guard	Passed	3.6
database.test.js	views: supply versus demand and the allocator work queue	Passed	4.1
database.test.js	reconciliation views detect tampering	Passed	4.8
database.test.js	indexes: partial and composite indexes are used by the hot queries	Passed	3.0
database.test.js	tx(): commits on success, rolls back everything on failure, supports nesting	Passed	3.5
database.test.js	isolation: a second connection cannot start a write transaction while one is open, then sees the committed state	Passed	112.4
engine.test.js	abundance: every line receives its full remaining need	Passed	0.9
engine.test.js	equal priority: proportional split of what exists	Passed	7.9
engine.test.js	priority skew: coverage is proportional to priority, summing exactly to supply	Passed	0.2
engine.test.js	capping: a small high-priority line is fully served and the surplus is redistributed	Passed	0.2
engine.test.js	deficit awareness: a line that already holds more than the fair level waits	Passed	0.2
engine.test.js	deficit awareness: lines are levelled up to the same cumulative coverage	Passed	0.1
engine.test.js	no stock: nothing allocated, flagged scarce	Passed	0.1
engine.test.js	property: 600 random scenarios satisfy the allocation invariants and match an exact solver	Passed	15.5
engine.test.js	property: cumulative coverage never decreases with priority among lines that receive stock	Passed	2.7
engine.test.js	determinism: input order does not change the result	Passed	0.6
engine.test.js	priority score: urgency x vulnerability x aging, with aging capped	Passed	0.2
engine.test.js	standard-based approval caps over-requests and explains why	Passed	0.2
engine.test.js	geo: haversine and road distance	Passed	0.1
engine.test.js	allocate: serves each camp from its nearest warehouse and splits across warehouses when needed	Passed	5.8
engine.test.js	allocate: scarcity favours higher urgency and explains the decision	Passed	0.3
engine.test.js	allocate: no stock gives a clear explanation and no sources	Passed	0.3
engine.test.js	allocate: aging lifts a long-waiting low-priority request above a fresh equal one	Passed	0.2
engine.test.js	allocate: lines with nothing left to allocate are skipped	Passed	0.1
queries.test.js	db/queries.sql contains the documented showcase queries	Passed	1.2
queries.test.js	query q01_shortage_by_item runs on the demo data and returns rows	Passed	0.6

File	Test	Result	ms
queries.test.js	query q02_camps_ranked_by_unmet_need runs on the demo data and returns rows	Passed	0.2
queries.test.js	query q03_requests_past_sla runs on the demo data and returns rows	Passed	0.1
queries.test.js	query q04_network_share_per_warehouse runs on the demo data and returns rows	Passed	0.3
queries.test.js	query q05_nearest_stocked_warehouse runs on the demo data and returns rows	Passed	0.2
queries.test.js	query q06_fairness_by_urgency_per_run runs on the demo data and returns rows	Passed	0.2
queries.test.js	query q07_shipment_timeline_gaps runs on the demo data and returns rows	Passed	0.3
queries.test.js	query q08_losses_by_route runs on the demo data and returns rows	Passed	0.3
queries.test.js	query q09_people_served_per_camp runs on the demo data and returns rows	Passed	0.4
queries.test.js	query q10_ledger_running_balance runs on the demo data and returns rows	Passed	0.3
queries.test.js	query q11_critical_camps_with_nothing_coming runs on the demo data and returns rows	Passed	0.1
queries.test.js	query q12_top_volunteers runs on the demo data and returns rows	Passed	0.2
queries.test.js	query q13_items_low_everywhere runs on the demo data	Passed	0.1
queries.test.js	query q14_deliveries_per_day_with_gaps runs on the demo data and returns rows	Passed	0.1
queries.test.js	query q15_integrity_report runs on the demo data and returns rows	Passed	1.0
queries.test.js	query q16_priority_score_in_sql runs on the demo data and returns rows	Passed	0.3
queries.test.js	query q17_request_pipeline_funnel runs on the demo data and returns rows	Passed	0.1
queries.test.js	q15 reports a consistent database	Passed	1.0
queries.test.js	q10 ledger replay ends at the inventory balance	Passed	0.3
queries.test.js	q06 shows higher urgency receiving higher coverage in scarce runs	Passed	0.2
scenarios.test.js	the demo seed is internally consistent and shaped like a real incident	Passed	6.5
scenarios.test.js	live scenario: scarce items are rationed by priority, abundant items are met in full	Passed	2.3
scenarios.test.js	live scenario: critical requests are covered better than low-urgency ones for water	Passed	4.8
scenarios.test.js	live scenario: committing the run via the API reserves every unit of scarce stock exactly once	Passed	105.9
scenarios.test.js	role journeys on the demo data	Passed	136.1